УНИВЕРЗИТЕТ
У НОВОМ САДУ

ФАКУЛТЕТ
ТЕХНИЧКИХ НАУКА

Трг Доситеја Обрадовића 6, 21000 Нови Сад, Република Србија
Деканат: 021 6350-413; 021 450-810; Централа: 021 485 2000
Рачуноводство: 021 458-220; Студентска служба: 021 6350-763
Телефакс: 021 458-133; e-mail: ftndean@uns.ac.rs

ИНТЕГРИСАНИ
СИСТЕМ
МЕНАЏМЕНТА
СЕРТИФИКОВАН ОД:

Projektovanje složenih digitalnih sistema

NAZIV PROJEKTA:

Razvoj sistema za detekciju lica upotrebom Viola-Jones metodologije sa hardverskom akceleracijom

vremenski kritičnih algoritamskih funkcija

PROJEKAT IZRADILI:

Radivoje Jović EE 202/2021

Vojislav Sandić

Sara Glišević

Sadržaj

Uvod 5

Motivacija i cilj 5

Faze razvoja sistema 6

Informacije o target platformi 6

O algoritmu : Šta je Viola-Jones metodologija? 7

Transformacije koda, optimizacija i arhitekturne promjene 10

Podjela posla: šta se ubrzava ? 10

Referentni (zlatni) model 10

Transformacije koda 22

Pregled najvažnijih transformacija 22

Data packing ROM klasifikatora 22

Kodovanje težina pomjeranjem i sabiranjem 22

Integralna slika po bankama redova 23

Rotacija redova (row_base) 23

Inkrementalni upis redova 24

Režim B: pretraga reda u hardveru 24

Pipelining pravougaonika 25

Bit-width narrowing 26

Manje transformacije vrijedne pomena 26

Odbačena optimizacija: paralelizacija čitanja pravougaonika klasifikatora replikacijom banki 27

Pojašnjenje implementacije algoritma 28

Struktura kaskade i rani izlazak 28

Integralna slika i normalizacija varijansom 28

Izmjerena svojstva podataka klasifikatora 28

Statistika ranog izlaska koja oblikuje cijeli dizajn 29

Specifikacija interfejsa jezgra 29

Registarska mapa 31

Interfejs BRAM_PORTA za masovni upis 31

Dogovorena ograničenja 31

Redoslijed programiranja 32

Controlpath ... 33
Prefetch i preklapanje ... 42
Izbor režima ... 42
Moduli controlpatha ... 43
Datapath ... 46
Memorijske strukture datapatha ... 47
Jedinica varijanse ... 47
Protočna obrada pravougaonika ... 47
Skaliranje težina i akumulatori ... 47
Putanja rezultata i detekcija ... 47
Registarske operacije po stanjima ... 48
Moduli datapatha ... 49
Pakovanje IP jezgra ... 56
Integracija sistema ... 59
Blok dizajn i mapa adresa ... 59
Taktovi i reset ... 60
Konfiguracija periferija procesorskog sistema ... 60
Rezultati sinteze i implementacije ... 62
Raščlanjeno zauzeće resursa ... 62
Kritična putanja ... 62
Kašnjenje i propusna moć ... 63
Oznake ... 63
Kašnjenje jednog prozora ... 64
Propusna moć ... 64
Vrijeme obrade cijele slike ... 64
Poređenje zauzeća resursa sa HLS ... 65
Bitstream ... 65
Verifikacija ... 67
Nivoi koji su ponovo pokrenuti i potvrđeni ... 67
Zašto je dodat nivo 2 ... 67
Namjerno konstruisani slučajevi prihvatanja ... 67

Ograničenja pokrivenosti RTL testbenčeva 67
Poređenje sa virtuelnom platformom 69
Metoda 69
Smanjenje transfera 69
Propusnost 69
Potvrda na hardveru 70
Puštanje u rad na ploči 70
Tok builda i izvoza 70
Aplikacija na procesoru 71
Konfiguracija ploče i softvera 72
Izmjereni rezultati na ploči 73
Kvalitet detekcije 74
Vrijeme režima B u odnosu na predviđanje 75
Dodaci za pomoć pri radu 75
Podešavanja serijskog terminala 75
Referentna konfiguracija procesorskog sistema 76
Multipleksirani izvodi i naponi 76
Memorija 76
Namjerno nepromijenjeno podešavanje takta 76
Literatura 77

Uvod

Ovaj projekat realizuje sistem za detekciju lica korištenjem Viola-Jones metodologije zasnovan na kombinaciji softverske obrade slike i hardverske akceleracije računski intenzivnih algoritamskih funkcija. Realizovani akcelerator je namjensko VHDL IP jezgro povezano sa ARM procesorskim sistemom preko AXI magistrale, kojim upravlja bare-metal aplikacija. Trenutna implementacija vrši obradu pojedinačnih slika kao ulaznih podataka, dok je arhitektura sistema koncipirana sa tendencijom da omogućava proširenje na obradu kontinualnog video feeda preko kamere u realnom vremenu.

Početna verzija algoritma implementirana je u softverskoj formi na ARM procesoru, što je omogućilo funkcionalnu validaciju i analizu performansi. Međutim, vremenska analiza identifikovala je kritične sekcije sa najvećim opterećenjem procesora, zbog čega je izvršena njihova implementacija u FPGA programabilnoj logici radi postizanja većeg ubrzanja i determinističkog izvršavanja.

Ovaj dokument opisuje RTL (Register Transfer Level) implementaciju hardverskog IP jezgra za detekciju lica, razvijenog kao treća faza u višestepenom procesu razvoja sistema. Implementacija je realizovana na razvojnoj ploči Zybo Z7-10, zasnovanoj na Xilinx Zynq-7000 SoC arhitekturi, koja integriše dual-core ARM Cortex-A9 procesorski sistem (PS) i FPGA programabilnu logiku (PL) na jednom čipu.

Akcelerirana je klasifikacija jednog prozora dimenzija 24×24. Na osnovu integralne i kvadrirane integralne slike zone prozora, jezgro izračunava kaskadu od 25 faza sa ukupno 2913 slabih klasifikatora i vraća ishod, uz early exit mehanizam: odbačen prozor vraća negativan indeks faze koja ga je odbacila, tako da softver saznaje ne samo da je prozor odbačen, već i gdje.

Motivacija i cilj

Primarni cilj projekta je demonstracija koncepta hardverske akceleracije kroz implementaciju sekvencijalnog algoritma za obradu slike u obliku FSM-baziranog IP jezgra. Konkretno, zadatak je bio eliminisati softverske petlje i uslove iz algoritma, transformisati ih u registrovanu logiku sa jasno definisanim stanjima i prelazima, te pokazati mjerljivo ubrzanje u odnosu na softversku implementaciju.

Sekundarni cilj je integracija IP jezgra u ARM+FPGA (PS+PL) sistem putem standardnog AXI4-Lite protokola, što je preduslov za industrijsku primjenu IP jezgra. Cijeli tok razvoja - od specifikacije i virtuelne platforme do RTL implementacije, Vivado integracije i Vitis bare-metal verifikacije – prati standardnu metodologiju razvoja embeded sistema.

Faze razvoja sistema

Faza	Naziv	Alat/Jezik	Svrha
1	Softverska implementacija	C/C++	Referentni algoritam profajliranje
2	Virtuelna platforma (VP)	SystemC/TLM	Brza procjena arhitekture I performansi
3	RTL implementacija	VHDL/Vivado/Vitis	Hardverska realizacija na FPGA

Tabela 1. Faze razvoja sistema

Svaka faza nadovezuje se na rezultate prethodne. Virtuelna platforma dala je procjenu performansi koja je poslužila kao referenca za validaciju RTL implementacije. Odstupanja između faza su očekivana i analizirana su u nekom od narednih poglavlja.

Informacije o target platformi

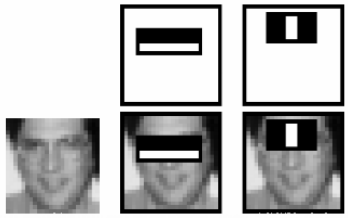
Naziv	Specifikacija
SoC	Xilinx Zynq-7000 (xc7z010clg400-1)
ARM procesor (PS)	Dual-core ARM Cortex-A9 @ 667 MHz
FPGA logika (PL)	Artix-7 serija, 28 nm process
Slice LUT-ovi	17,600
Slice registri	35,200
Block RAM tileovi	60
DSP48E1 blokovi	80
AXI magistrala	AXI4, AXI4-Lite, AXI4-Stream
PL takt (FCLK_CLK0)	50 MHz (konfigurisano)

PS-PL veza	AXI GP0 master port (M_AXI_GP0)

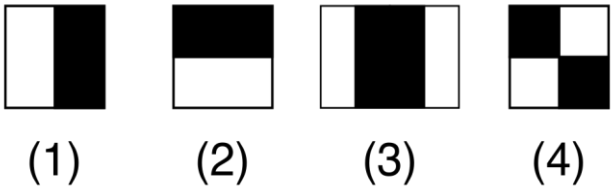
Tabela 2. Osnovni podaci o Zybo Z7-10 ploči

O algoritmu : Šta je Viola-Jones metodologija?

Što se tiče same matematičke pozadine koda, potrebno je osvrnuti se na algoritam koji je detaljno obrađen u sistemskom dijelu projekta. Viola-Jones algoritam baziran je na **Haarovim karakteristikama** digitalne slike. Haarove karakteristike prvi su definisali **Paul Viola** i **Michael Jones**, po kojima je algoritam i dobio ime, kako bi riješili problem računske skupoće rada sa intenzitetima slike. Haarove karakteristike su digitalne karakteristike slike koje se koriste za prepoznavanje objekata. Ime duguju svojoj intuitivnoj sličnosti sa Haarovim talasima. Ukratko, Haarova karakteristika posmatra susjedne pravougaone regione **greyscale** slike na specifičnim lokacijama u detekcijskom prozoru, sumira intenzitete piksela u svakom regionu i računa razliku između ovih suma. Razlika se zatim koristi za kategorizaciju dijelova slike. Ilustracija načina rada, kao i rezultat izvršavanja specifikacije, dat je na sljedećim slikama:



Slika 1. Način izvlačenja Haarovih karakteristika



Slika 2. Haarove karakteristike slike po Violi i Jonesu

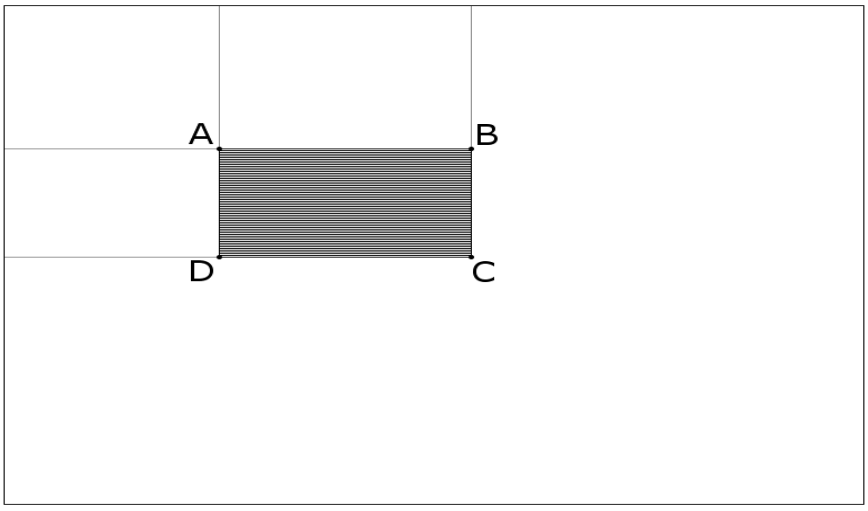
U fazi detekcije, prozor određene veličine se prevlači preko ulazne slike, pa se za svaku podsekciju slike računaju Haarove karakteristike. Ova razlika se zatim poredi sa prethodno naučenim vrijednostima koje razdvajaju objekte (lica u našem slučaju) od onih koji to nisu. Pošto je jedna takva Haarova karakteristika sama po sebi **weak learner** (malo bolja od nasumičnog pogađanja), potreban je veliki broj takvih karakteristika da bi se objekat dovoljno precizno opisao. Zbog toga, u Viola-Jones algoritmu se te osobine kaskadno organizuju tako da postanu jak klasifikator (**strong learner**). Ovakav način je računski brz zbog upotrebe **integralnih slika**, i može vršiti proračune konstantnom brzinom (60 instrukcija za jednu 2-pravougaonu osobinu).

Integralne slike (**summed-area tables**) su zapravo dvodimenzionalni LUT-ovi u formi matrice iste veličine kao i originalna. Integralna slika je način predstavljanja slike gdje svaki piksel

sadrži zbir svih piksela iz originalne slike koji se nalaze **iznad** i **lijevo** od njega (uključujući i taj piksel).
Ovo omogućava brzo izračunavanje zbira piksela unutar bilo kojeg pravougaonog regiona slike.

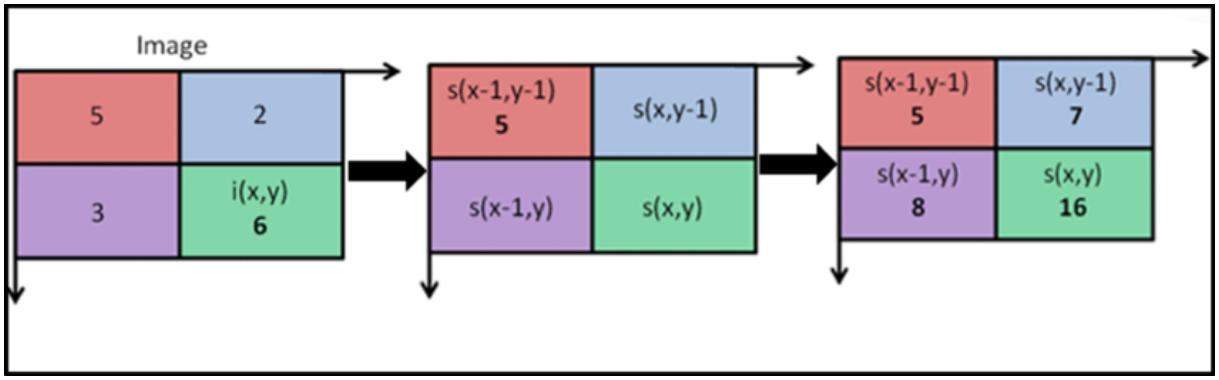
$$S = I(C) + I(A) - I(B) - I(D)$$

gdje tačke A,B,C,D pripadaju integralnoj slici, kao što je prikazano na slici ispod:

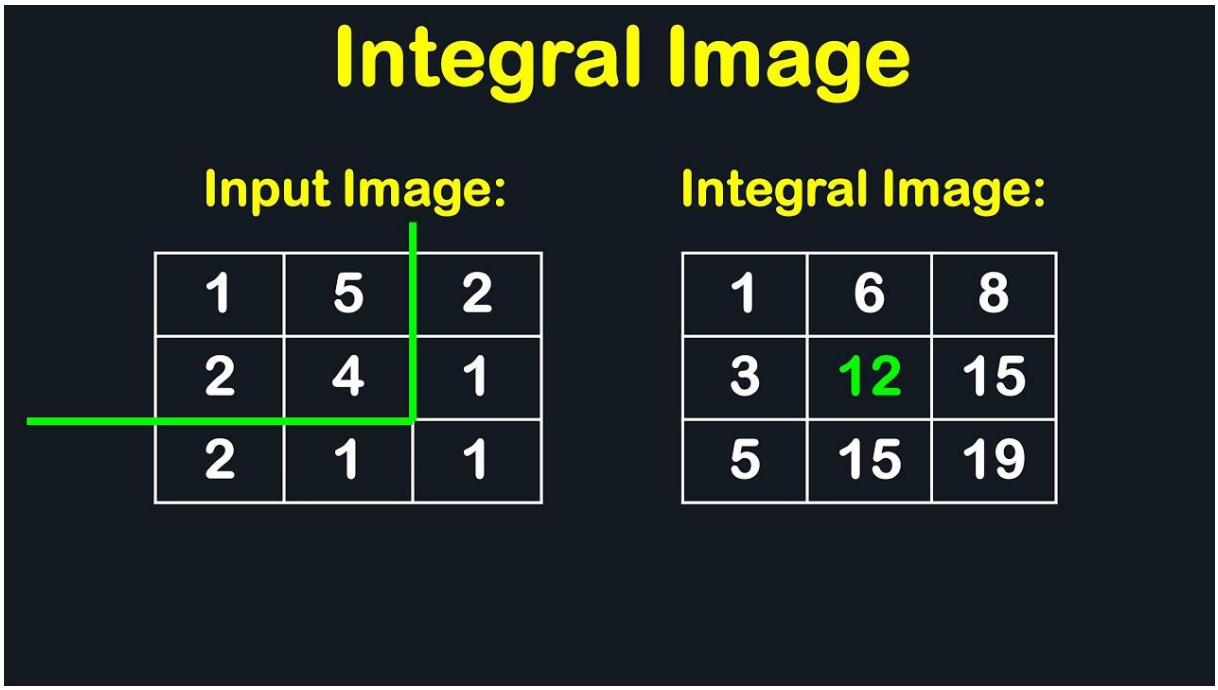


Slika 3. Nalaženje sume označene pravougaone zone

Primjer računanja integralne slike dat je na slikama 4 i 5 :



Slika 4. Ilustracija računanja integralnih slika



Slika 5. Ilustracija računanja integralnih slika

Učenje ovog algoritma vrši se tako što se od svih Haarovih osobina u određenoj oblasti (*M* = 162336 za preporučenu veličinu oblasti potprozora od 24x24 piksela) vrši odabiranje najbitnijih pri treniranju. To odabiranje se vrši upotrebom **AdaBoost** pomoćnog algoritma. **AdaBoost** (skraćeno od **Ada**ptive **Boost**ing) je meta-algoritam za statističku klasifikaciju koji su razvili **Yoav Freund** i **Robert Schapire** 1995. godine, za šta su 2003. dobili Gödelovu nagradu. Može se koristiti u kombinaciji sa različitim tipovima algoritama za učenje radi poboljšanja performansi. Izlazi više slabih klasifikatora kombinuju se u ponderisanu sumu koja predstavlja konačni rezultat ojačanog klasifikatora. Obično se AdaBoost koristi za binarnu klasifikaciju, iako se može generalizovati na više klasa ili ograničene intervale realnih vrednosti. Algoritam funkcioniše po sljedećoj formuli :

$$h(x) = sgn\left(\sum_{j=1}^{M} \alpha_j h_j(x)\right)$$

Gdje su h_j slabi klasifikatori koji se definišu na sljedeći način :

$$h_j(x) = \begin{cases} -s_j, & f_j < \theta_j \\ s_j, drugačije \end{cases}$$

θ_j- granična vrijednost, s_j (može biti ± 1) - polaritet , kao i alfa koeficijenti se definišu tokom treniranja modela. Rezultat ovakvog odabiranja jesu jači klasifikatori, a broj osobina se znatno smanjuje (na oko 6000). Kako bi se poboljšala efikasnost, takvi jaki klasifikatori se međusobno kaskadiraju, te zavisno od broja faza, ukoliko region slike ne prođe provjeru na

prvom klasifikatoru, pri poređenju sljedeći klasifikator taj region slike uopšte neće uzimati u obzir.

Transformacije koda, optimizacija i arhitekturne promjene

Podjela posla: šta se ubrzava ?

U hardver je premještena isključivo evaluacija kaskade. Dekodovanje slike, prevođenje u nivoe sive, izgradnja piramide od 13 skala, inkrementalno ažuriranje integralne slike i završno grupisanje pravougaonika ostaju na procesorskom sistemu.

Istu podjelu koristi i SystemC/TLM virtuelna platforma izrađena za kurs **Projektovanje elektronskih uređaja na sistemskom nivou**, pa obje implementacije obavljaju potpuno isti posao i njihove cijene po prozoru i po klasifikatoru se mogu direktno porediti.

Faza	Izvršava se na	Obrazloženje
Dekodovanje slike, greyscale	PS	Jednokratni trošak, bez unutrašnje petlje
Piramida skala (13 nivoa)	PS	Presampling u fiksnom zarezu; identično kvantizaciji virtuelne platforme
Integralna i kvadrirana slika	PS, inkrementalno	Jedan novi red po pomjeraju prozora
Klasifikacija kaskadom	PL (ovo IP jezgro)	Unutrašnja petlja: do 2913 klasifikatora × 208.506 prozora
Grupisanje pravougaonika	PS	Radi nad nekolicinom detekcija, ne nad prozorima

Tabela 3. Podjela poslova za PS i PL dio

Referentni (zlatni) model

Mjerodavna referenca u cijelom projektu je datoteka `vp/srcs/ip.cpp` iz virtuelne platforme. Sav RTL kod i svako VHDL test okruženje na kraju se provjerava prema izvedenici tog istog izvornog koda.

Ovo je važnije nego što na prvi pogled izgleda. Referentni kod računa varijansu u 32-bitnoj neoznačenoj aritmetici i za pojedine ulaze se stvarno oslanja na overflow. „Čistija" implementacija sa širim akumulatorima dala bi drugačije rezultate baš za te ulaze. Hardver zato vjerno reprodukuje prelivanje umesto da ga izbjegava.

Kritična funkcija oko koje je baziran čitav akcelerator jeste **runCascadeClassifier**, koja se u kodu oslanja na pozive osnovne funkcije **evalWeakClassifier**.Implementacija ovih funkcija za referentni C++ model je data u nastavku:

```
int ip::run_cascade_classifier(uint32_t p_offset, uint32_t width, pl_t& pl,
unsigned char* data,tlm_utils::tlm_quantumkeeper& qk,sc_time& offset)

{
    int haar_counter = 0;
    int w_index = 0;
    int r_index = 0;
    /*
    Korak 1: updatePvalue
```

```
    Cita 4 vrednosti iz int_img_buffer i 4 iz sq_int_buffer iz DDR3.
    Racuna variance_norm_factor koji se koristi za sve slabe klasifikatore
u ovom prozoru.
    */
    update_pvalue(p_offset, width, pl, data, qk, offset);
    // Racunanje variance_norm_factor (isti algoritam kao u originalnom
kodu)
    unsigned int mean =(unsigned int)(p[0] - p[1] - p[2] + p[3]);
    unsigned int variance_norm_factor =(unsigned int)(pq[0] - pq[1] - pq[2]
+ pq[3]);
    variance_norm_factor = variance_norm_factor * (unsigned
int)INV_WINDOW_AREA;
    variance_norm_factor = variance_norm_factor - mean * mean;

    if(variance_norm_factor > 0)
        variance_norm_factor = int_sqrt(variance_norm_factor);
    else
        variance_norm_factor = 1;

    /* Korak 2: Petlja kroz 25 faza kaskade */
    for(int i = 0; i < NUM_STAGES; i++)
    {
        /* Citanje broja klasifikatora u ovoj fazi iz BRAM
(stages_array[i], lokalna adresa: VP_ADDR_BRAM_STAGES + i) */
        uint8_t num_classifiers = 0;
        bram_read(VP_ADDR_BRAM_STAGES + i,
&num_classifiers,sizeof(uint8_t), pl, data, qk, offset);
        int stage_sum = 0;

        /* Petlja kroz sve slabe klasifikatore u ovoj fazi */
        for(int j = 0; j < (int)num_classifiers; j++)
        {
            stage_sum += eval_weak_classifier((int)variance_norm_factor,
(int)p_offset,haar_counter, w_index, r_index,width, pl, data, qk, offset);
            haar_counter++;
            w_index += 3;
            r_index += 12;
        }

        /* Citanje praga faze iz BRAM (stages_thresh_array[i], lokalna
adresa: VP_ADDR_BRAM_STAGES_TH + i) */
        int16_t stage_thresh = 0;
        bram_read(VP_ADDR_BRAM_STAGES_TH + i,
&stage_thresh,sizeof(int16_t), pl, data, qk, offset);

        /* Poredjenje sa pragom (faktor 0.4 = 2/5 celobrojna
aproksimacija), ako stage_sum < 0.4 * stage_thresh, odbijen na fazi i */
        if(stage_sum * 5 < (int)stage_thresh * 2)
            return -i; // early exit: odbijen na fazi i

    } // kraj petlje po fazama
    return 1; // prosao sve faze, lice detektovano
}
```

Listing 1. Softverska implementacija funkcije runCascadeClassifier u VP

```
int ip::eval_weak_classifier(int variance_norm_factor, int p_offset,int
tree_index, int w_index, int r_index,uint32_t width,pl_t& pl, unsigned
char* data,tlm_utils::tlm_quantumkeeper& qk,sc_time& offset)
{
    /* Korak 1: Citanje tree_thresh_array[tree_index] iz BRAM, Lokalna
adresa: VP_ADDR_BRAM_TREE_TH + tree_index */
```

```
    int16_t tree_thresh = 0;
    bram_read(VP_ADDR_BRAM_TREE_TH + tree_index,
&tree_thresh,sizeof(int16_t), pl, data, qk, offset);
    int t = (int)tree_thresh * variance_norm_factor;
    int sum = 0;
    /*
    Korak 2: Obrada 3 pravougaonika (k = 0, 1, 2)
    Za svaki pravougaonik:
      a. Cita 4 char vrednosti (x, y, w, h) iz rectangles_array u BRAM
      b. Racuna 4 linearna ofset indeksa u int_img_buffer (zamena za
scaled_rect)
      c. Cita weight iz weights_array u BRAM
      d. Cita 4 int vrednosti iz int_img_buffer u DDR3
      e. Akumulira sumu
    */
    for(int k = 0; k < 3; k++)
    {
        /*
        a. Citanje raw (x, y, w, h) iz rectangles_array u BRAM
           Svaki pravougaonik je 4 vrednosti, svaka u svom short int slotu.
           Citamo 4 odvojena short int, jer mem[] je short int niz
           (svaka char vrednost zauzima jedan 2-bajtni slot).
        */
        short int sv;
        bram_read(VP_ADDR_BRAM_RECT + r_index + k*4 + 0, &sv, sizeof(short
int), pl, data, qk, offset);
        char rx = (char)sv;
        bram_read(VP_ADDR_BRAM_RECT + r_index + k*4 + 1, &sv, sizeof(short
int), pl, data, qk, offset);
        char ry = (char)sv;
        bram_read(VP_ADDR_BRAM_RECT + r_index + k*4 + 2, &sv, sizeof(short
int), pl, data, qk, offset);
        char rw = (char)sv;
        bram_read(VP_ADDR_BRAM_RECT + r_index + k*4 + 3, &sv, sizeof(short
int), pl, data, qk, offset);
        char rh = (char)sv;

        /* Null provera za treci pravougaonik (k == 2), ako su sva 4 polja
nula, treci pravougaonik ne postoji */
        if(k == 2 && rx == 0 && ry == 0 && rw == 0 && rh == 0)
            break;

        /*
        b. Racunanje linearnih ofset indeksa u int_img_buffer
           Ovo zamenjuje scaled_rectangles_array koji vise ne postoji.
           Formula: linearni_indeks = width * y + x
        */
        int tl = (int)width * (int)ry + (int)rx; // gornji levi
        int tr = (int)width * (int)ry + ((int)rx + (int)rw); // gornji
desni
        int bl = (int)width * ((int)ry + (int)rh) + (int)rx; // donji levi
        int br = (int)width * ((int)ry + (int)rh) + ((int)rx + (int)rw); //
donji desni

        /* c. Citanje weights_array[w_index + k] iz BRAM Lokalna adresa:
VP_ADDR_BRAM_WEIGHTS + w_index + k */
        int16_t weight = 0;
        bram_read(VP_ADDR_BRAM_WEIGHTS + w_index + k,
&weight,sizeof(int16_t), pl, data, qk, offset);
        /*
        d. Citanje 4 int_img vrednosti iz DDR3
```

```
            Lokalna DDR3 adresa: tl/tr/bl/br + p_offset
            (p_offset pomera prozor horizontalno u tekucem redu)
        */
        int img_tl, img_tr, img_bl, img_br;

        ddr3_read(tl + p_offset, &img_tl, sizeof(int), pl, data, qk,
offset);
        ddr3_read(tr + p_offset, &img_tr, sizeof(int), pl, data, qk,
offset);
        ddr3_read(bl + p_offset, &img_bl, sizeof(int), pl, data, qk,
offset);
        ddr3_read(br + p_offset, &img_br, sizeof(int), pl, data, qk,
offset);

        /*
        e. Akumulacija: Haar-like karakteristika = suma po cetiri coska
            (integralna slika formula)
        */
        sum += (img_tl - img_tr - img_bl + img_br) * (int)weight;
    } // kraj k petlje

    /* Korak 3: Poredjenje sa pragom i citanje alpha vrednosti iz BRAM, Ako
sum >= t: vraca alpha2[tree_index], u suprotnom vraca alpha1[tree_index] */
    int16_t alpha = 0;
    if(sum >= t)
        bram_read(VP_ADDR_BRAM_ALPHA2 + tree_index, &alpha,sizeof(int16_t),
pl, data, qk, offset);
    else
        bram_read(VP_ADDR_BRAM_ALPHA1 + tree_index, &alpha,sizeof(int16_t),
pl, data, qk, offset);

    return (int)alpha;
}
```

Listing 2. Softverska implementacija funkcije evalWeakClassifier u VP

Kao međukorak između virtuelne platforme i RTL modela, jezgro je implementirano u HLS-kompatibilnoj formi u programskom jeziku C. Kodni listing istih ovih funkcija, zajedno sa glavnom funkcijom je dat u nastavku:

```
static ap_int<16> eval_weak_classifier(ap_uint<32>
variance_norm_factor,ap_uint<32> p_offset,int tree_index,int w_index,int
r_index,

ap_uint<32> width,ap_int<16> bram[BRAM_SIZE],ap_int<32>
ddr3[DDR3_TOTAL_SIZE]){

//Korak 1: Citanje tree_thresh_array[tree_index] iz BRAMa
    ap_int<16> tree_thresh= bram[VP_ADDR_BRAM_TREE_TH + tree_index];
    ap_int<32> t= (ap_int<32>)tree_thresh *
(ap_int<32>)variance_norm_factor;
    ap_int<40> sum= 0;

//Korak 2: Obrada 3 pravougaonika (k=0,1,2)
    for(int k=0;k<3;k++){
        /*a. Citanje raw podataka o pravougaonicima(x,y,w,h) iz
rectangles_array u BRAMu
        Svaka short int vrijednost zauzima jedan ap_int<16> slot.
        */
        ap_int<8> rx = (ap_int<8>)bram[VP_ADDR_BRAM_RECT + r_index + k*4 +
0];
```

```
        ap_int<8> ry = (ap_int<8>)bram[VP_ADDR_BRAM_RECT + r_index + k*4 +
1];
        ap_int<8> rw = (ap_int<8>)bram[VP_ADDR_BRAM_RECT + r_index + k*4 +
2];
        ap_int<8> rh = (ap_int<8>)bram[VP_ADDR_BRAM_RECT + r_index + k*4 +
3];

        /* Null provjera za treci pravougaonik */
        if(k == 2 && rx == 0 && ry == 0 && rw == 0 && rh == 0)
            break;

        /* b. Racunanje linearnih ofset indeksa u int_img_buffer
        Formula:linearni_indeks = width * y + x*/
        ap_uint<14> tl = (ap_uint<14>)width * (ap_uint<14>)ry +
(ap_uint<14>)rx;
        ap_uint<14> tr = (ap_uint<14>)width * (ap_uint<14>)ry +
(ap_uint<14>)(rx + rw);
        ap_uint<14> bl = (ap_uint<14>)width * (ap_uint<14>)(ry + rh) +
(ap_uint<14>)rx;
        ap_uint<14> br = (ap_uint<14>)width * (ap_uint<14>)(ry + rh) +
(ap_uint<14>)(rx + rw);

        /* c. Citanje weights_array[w_index + k] iz BRAMa */
        ap_int<16> weight = bram[VP_ADDR_BRAM_WEIGHTS + w_index + k];

        // d. Citanje 4 int_img vrijednosti iz DDR3 */
        ap_int<32> img_tl = ddr3[(ap_uint<32>)tl + p_offset];
        ap_int<32> img_tr = ddr3[(ap_uint<32>)tr + p_offset];
        ap_int<32> img_bl = ddr3[(ap_uint<32>)bl + p_offset];
        ap_int<32> img_br = ddr3[(ap_uint<32>)br + p_offset];

        /* e. Akumulacija: Haarova karakteristika = suma po cetiri coska */
        sum += (ap_int<40>)(img_tl - img_tr - img_bl + img_br) *
(ap_int<40>)weight;
    } // kraj k petlje

    //Korak 3: Poredjenje sa pragom i citanje alpha vrijednosti iz BRAM
    ap_int<16> alpha;
    if(sum >= t)
        alpha = bram[VP_ADDR_BRAM_ALPHA2 + tree_index];
    else
        alpha = bram[VP_ADDR_BRAM_ALPHA1 + tree_index];
    return alpha;
}
```

Listing 3. Implementacija funkcije evalWeakClassifier u HLS C formatu

```
static ap_int<32> run_cascade_classifier(ap_uint<32> p_offset,ap_uint<32>
width,ap_int<32> p[4],ap_int<32> pq[4],ap_int<16>
bram[BRAM_SIZE],ap_int<32> ddr3[DDR3_TOTAL_SIZE]){
//inicijalizacija indeksa i brojacke promjenljive
ap_uint<12> haar_counter = 0;
ap_uint<14> w_index = 0;
ap_uint<16> r_index = 0;
/*Korak 1: update_pvalue (preveden u inline funkciju)
Cita vrijednosti 4 coska integralne slike iz int_img_buffer i 4 coska
kvadrirane integralne slike iz sq_int_buffer iz DDR3.*/
    ap_uint<14> corners[4];
    corners[0] = (ap_uint<14>)p_offset;
    corners[1] = (ap_uint<14>)(p_offset + ORIG_WIN_WIDTH - 1);
    corners[2] = (ap_uint<14>)(p_offset + (ap_uint<32>)width *
(ORIG_WIN_HEIGHT - 1));
```

```
    corners[3] = (ap_uint<14>)(p_offset + (ap_uint<32>)width *
(ORIG_WIN_HEIGHT - 1) + ORIG_WIN_WIDTH - 1);
    for(int c = 0; c < 4; c++)
        p[c]  = ddr3[corners[c]];
    for(int c = 0; c < 4; c++)
        pq[c] = ddr3[DDR3_SQ_INT_OFFSET + corners[c]];
    /*Korak 2: Racunanje variance_norm_factor
    Koristi 32-bitnu unsigned aritmetiku , kao i IP iz VPa koji koristi
unsigned int*/
    ap_uint<32> mean32=(ap_uint<32>)(p[0] - p[1] - p[2] + p[3]);
    ap_uint<32> pq_sum32=(ap_uint<32>)(pq[0] - pq[1] - pq[2] + pq[3]);
    ap_uint<32> vnf32= pq_sum32 * (ap_uint<32>)INV_WINDOW_AREA;
    vnf32=vnf32-mean32*mean32;
    ap_uint<32> variance_norm_factor;
    if(vnf32 > 0)
        variance_norm_factor = int_sqrt(vnf32);
    else
        variance_norm_factor = 1;
    /*Korak 3: Petlja kroz 25 faza kaskade*/
    for(int i = 0; i < NUM_STAGES; i++){
        /* Citanje broja klasifikatora u ovoj fazi iz BRAM */
        ap_uint<8> num_classifiers = (ap_uint<8>)bram[VP_ADDR_BRAM_STAGES +
i];
        ap_int<24> stage_sum = 0;
        /* Petlja kroz sve slabe klasifikatore u ovoj fazi */
        for(int j = 0; j < (int)num_classifiers; j++){
            stage_sum += eval_weak_classifier(variance_norm_factor,
p_offset,(int)haar_counter, (int)w_index, (int)r_index,width, bram, ddr3);
            haar_counter++;
            w_index += 3;
            r_index += 12;
        }
        /* Citanje praga faze iz BRAM */
        ap_int<16> stage_thresh = bram[VP_ADDR_BRAM_STAGES_TH + i];
        /* Poredjenje sa pragom: stage_sum < 0.4 * stage_thresh - odbijen
na i fazi */
        if(stage_sum * 5 < (ap_int<24>)stage_thresh * 2)
            return (ap_int<32>)(-i);
    } // kraj petlje po fazama
    return ap_int<32>(1);
}
```

Listing 4. Implementacija funkcije runCascadeClassifier u HLS C formatu

```
// top_function
void top_function(ap_uint<8> *cfg_val,ap_uint<32> p_offset,ap_uint<32>
width,ap_int<16> bram[BRAM_SIZE],ap_int<32> ddr3[DDR3_TOTAL_SIZE],bool
*ip_interrupt,ap_int<32> *result){
static VJCore IP;
IP.cfg_reg = *cfg_val;
//promjena stanja zavisno od stanja konfiguracionog registra
    if(IP.cfg_reg[IP_RESET_BIT])
    IP.state = RESET;
    else if(IP.cfg_reg[IP_START_BIT])
    IP.state = START;
    else
    IP.state = IDLE;
//reagovanje na promjene stanja
    switch(IP.state){
    case IDLE:
    {
    IP.ip_interrupt = false;
```

```
    break;
    }
    case RESET:
    {
    IP.ip_interrupt=false;
    IP.result_reg=0;
    IP.p_offset_reg=0;
    IP.width_reg=0;
    reset_l:for(int i = 0; i < 4; i++)
        IP.p[i] = IP.pq[i] = 0;
    IP.cfg_reg = ap_uint<8>(0);
    break;
    }
    case START:
    {
    IP.ip_interrupt = false;
    IP.p_offset_reg = p_offset;
    IP.width_reg = width;
    IP.result_reg = run_cascade_classifier(IP.p_offset_reg, IP.width_reg,
IP.p, IP.pq,bram, ddr3);
    IP.cfg_reg.set(IP_DONE_BIT, 1);
    IP.cfg_reg.set(IP_START_BIT, 0);
    IP.ip_interrupt = true;
    break;
    }
   }
    *cfg_val = IP.cfg_reg;
    *ip_interrupt = IP.ip_interrupt;
    *result = IP.result_reg;
}
```

Listing 5. Top funkcija za HLS C implementaciju akceleratora

U nastavku će dodatno biti objašnjena implementacija samog algoritma, ali i arhitektonske promjene koje je bilo neophodno izvesti kako bi algoritam bio prilagođen za target platformu.

Uklanjanje petlji

Kako bi se kod mogao mapirati na hardver, neophodno je izvršiti uklanjanje petlji. Ovo je neophodno zato što se FOR petlja značajno teže mapira u hardveru nego u softveru, i to iz tri razloga:

1. U C/C++ kodu, iteratori petlje postoje isključivo na stacku, što u hardveru naravno nije implementabilno na isti način, pa se takva funkcionalnost dobija tako što se iteratori smještaju i registar koji „preživljava“ taktne cikluse i inkrementira se pri svakom narednom.
2. Testovi u softveru dešavaju se „između iteracija“, dok bi u hardverskom smislu to bilo ili zasebno stanje, ili testni blok u određenom stanju.
3. U softveru se tijelo petlje izvršava „atomski“, bez vidljive cijene izvršavanja, dok bi u hardveru to zahtijevalo jedan ili više ciklusa, uz vidljivo memorijsko kašnjenje.

Prije nastavljanja diskusije, neophodno je napomenuti da je uočeno da se baferi integralne slike mogu i moraju smjestiti u BRAM blokove, što predstavlja prvu svojevrsnu arhitekturnu optimizaciju i na njoj će se temeljiti sva dalja analiza. Zaključak je proistekao iz diskusije sa predmetnim asistentom, jer je razmatrana implementacija sa DMA kontrolerom, koja je

također odbačena, bila jedino rješenje za pristup podacima u DDR3, ali da zbog načina skladištenja i upisa u bafere integralne slike nije bila moguća za realizaciju, čak ni korištenjem „scatter-gather“ načina rada. Kod sa odmotanom petljom i RTL model u nastavku temeljeni su na ovoj arhitekturnoj promjeni, koju je bilo bitno napomenuti.

C implementacija ima 5 ugniježđenih petlji:
run_cascade_classifier

```
 for c in 0..3             //read p[]
 for c in 0..3             //read pq[]
 for i in 0..24            //faze
    for j in 0..n_cls-1   //klasifikatori
      for k in 0..2     //pravougaonici (unutar evalWeakClassifier)
 for i in 0..15            //int_sqrt
```

Listing 6. Prikaz bitnih petlji u kritičnoj funkciji

i ove petlje treba eliminisati. Primjenom sljedećih transformacija koda, od kojih su najbitnije detaljnije opisane u narednom poglavlju, kod kritične funkcije runCascadeClassifier i pomoćne evalWeakClassifier (preimenovane u **vj_core_step** radi boljeg razumijevanja načina rada IP-a), petlje se eliminišu i sistem je spreman za dalju razradu:

```
void vj_core_step(vj_core_t *IP, const vj_mem_t *mem)
{
    switch (IP->state) {


    case S_IDLE:
        IP->ip_interrupt = 0;
        if (IP->cfg_reg & (1u << IP_START_BIT))
            IP->state = S_WINDOW_INIT;
        break;


    case S_WINDOW_INIT:
        IP->haar_counter = 0;
        IP->i            = 0;
        IP->j            = 0;
        IP->hit_count    = 0;
        IP->state        = S_PV_ADDR;
        break;


    case S_PV_ADDR:
        IP->corner_row[0] = 0;                    IP->corner_col[0] = IP-
>p_offset_reg;
        IP->corner_row[1] = 0;                    IP->corner_col[1] = IP-
>p_offset_reg + ORIG_WIN_WIDTH - 1;
        IP->corner_row[2] = ORIG_WIN_HEIGHT - 1;  IP->corner_col[2] = IP-
>p_offset_reg;
        IP->corner_row[3] = ORIG_WIN_HEIGHT - 1;  IP->corner_col[3] = IP-
>p_offset_reg + ORIG_WIN_WIDTH - 1;
        IP->state = S_PV_FETCH;
        break;


    case S_PV_FETCH:
```

```
        IP->p[0] = mem->int_img[image_row(IP->corner_row[0], IP-
>row_base)][IP->corner_col[0]];
        IP->p[1] = mem->int_img[image_row(IP->corner_row[1], IP-
>row_base)][IP->corner_col[1]];
        IP->p[2] = mem->int_img[image_row(IP->corner_row[2], IP-
>row_base)][IP->corner_col[2]];
        IP->p[3] = mem->int_img[image_row(IP->corner_row[3], IP-
>row_base)][IP->corner_col[3]];
        IP->pq[0] = mem->sq_int[0][IP->corner_col[0]];
        IP->pq[1] = mem->sq_int[0][IP->corner_col[1]];
        IP->pq[2] = mem->sq_int[1][IP->corner_col[2]];
        IP->pq[3] = mem->sq_int[1][IP->corner_col[3]];
        IP->state = S_PV_LATCH;
        break;


    case S_PV_LATCH:
        IP->state = S_VNF_SUMS;
        break;


    case S_VNF_SUMS:
        IP->mean32   = (IP->p[0]  + IP->p[3])  - (IP->p[1]  + IP->p[2]);
        IP->pq_sum32 = (IP->pq[0] + IP->pq[3]) - (IP->pq[1] + IP->pq[2]);
        IP->state    = S_VNF_MUL;
        break;


    case S_VNF_MUL:
        IP->vnf32     = (IP->pq_sum32 << 9) + (IP->pq_sum32 << 6);  /* *576
*/
        IP->mean_sq32 = IP->mean32 * IP->mean32;
        IP->state     = S_VNF_SUB;
        break;


    case S_VNF_SUB:
        IP->vnf32 = IP->vnf32 - IP->mean_sq32;
        if (IP->vnf32 == 0) {
            IP->variance_norm_factor = 1;
            IP->sqrt_pending = 0;
        } else {
            IP->sqrt_pending = 1;
        }
        IP->state = S_VNF_SQRT;
        break;


    case S_VNF_SQRT:
        if (IP->sqrt_pending) {
            IP->variance_norm_factor = int_sqrt(IP->vnf32);
            IP->sqrt_pending = 0;
        }
        IP->state = S_STAGE_ENTER;
        break;


    case S_STAGE_ENTER:
        IP->num_classifiers  = mem->stages_array[IP->i];
        IP->j                = 0;
        IP->stage_sum        = 0;
```

```
        IP->classifier_next  = mem->classifier[IP->haar_counter];
        IP->state            = S_CLS_FETCH;
        break;


    case S_CLS_FETCH:
        IP->classifier = IP->classifier_next;
        IP->state      = S_RECT_0;
        break;


    case S_RECT_0: {
        haar_rect_t r = rectangles_array(IP->classifier, 0);
        IP->t = (int32_t)tree_thresh_array(IP->classifier)
              * (int32_t)IP->variance_norm_factor;
        IP->classifier_next = mem->classifier[IP->haar_counter + 1];
        IP->sum = -haar_rect_sum(IP, mem, r);
        IP->state = S_RECT_1;
        break;
    }

    /* k = 1*/
    case S_RECT_1: {
        haar_rect_t r  = rectangles_array(IP->classifier, 1);
        int32_t     rs = haar_rect_sum(IP, mem, r);
        IP->sum += weight1_is_3(IP->classifier) ? ((rs << 1) + rs) : (rs <<
1);
        if (third_rect_present(IP->classifier)) {
            IP->state = S_RECT_2;                       /* iteratorski
hold*/
        } else {
            accumulate_alpha(IP);
            IP->classifier = IP->classifier_next;
            IP->haar_counter++;
            IP->j++;
            IP->state = (IP->j >= IP->num_classifiers) ? S_CLS_FLUSH :
S_RECT_0;
        }
        break;
    }

    /* k = 2 */
    case S_RECT_2: {
        haar_rect_t r = rectangles_array(IP->classifier, 2);
        IP->sum += (haar_rect_sum(IP, mem, r) << 1);   /* weight 2 je 8192
*/
        accumulate_alpha(IP);
        IP->classifier = IP->classifier_next;
        IP->haar_counter++;
        IP->j++;
        IP->state = (IP->j >= IP->num_classifiers) ? S_CLS_FLUSH :
S_RECT_0;
        break;
    }

    /* dodatno stanje za praznjenje pipelinea*/
    case S_CLS_FLUSH:
        IP->state = S_STAGE_TEST;
        break;

    /**/
```

```
    case S_STAGE_TEST:
        if (IP->stage_sum * 5 < (int32_t)mem->stages_thresh_array[IP->i] *
2)
            IP->state = S_RETURN_REJECT;
        else if (IP->i == NUM_STAGES - 1)
            IP->state = S_RETURN_ACCEPT;
        else {
            IP->i++;
            IP->state = S_STAGE_ENTER;
        }
        break;

    /*return -i*/
    case S_RETURN_REJECT:
        IP->result_reg = -(int32_t)IP->i;
        IP->state      = S_WINDOW_END;
        break;

    /*return 1*/
    case S_RETURN_ACCEPT:
        IP->result_reg = 1;
        if (IP->hit_count < HIT_LIST_LEN)
            IP->hit_list[IP->hit_count++] = (uint16_t)IP->p_offset_reg;
        IP->state = S_WINDOW_END;
        break;

    /**/
    case S_WINDOW_END:
        if ((IP->cfg_reg & (1u << IP_MODE_BIT)) &&
            IP->p_offset_reg + IP->p_offset_step <= IP->p_offset_last) {
            IP->p_offset_reg += IP->p_offset_step;
            IP->i            = 0;
            IP->j            = 0;
            IP->haar_counter = 0;
            IP->state        = S_PV_ADDR;      /* bypass S_WINDOW_INIT*/
        } else {
            IP->state = S_DONE;
        }
        break;


    case S_DONE:
        IP->cfg_reg |= (1u << IP_DONE_BIT);
        IP->cfg_reg &= (uint8_t)~(1u << IP_START_BIT);
        IP->ip_interrupt = 1;
        IP->state = S_IDLE;
        break;
    }
}

/*
 */
int32_t run_cascade_classifier(vj_core_t *IP, const vj_mem_t *mem, uint32_t
p_offset)
{
    IP->p_offset_reg = p_offset;
    IP->cfg_reg     |= (1u << IP_START_BIT);
    IP->ip_interrupt = 0;
    do {
        vj_core_step(IP, mem);
    } while (!IP->ip_interrupt);
```

```
    IP->cfg_reg &= (uint8_t)~(1u << IP_DONE_BIT);
    return IP->result_reg;
}
```

Listing 7. Prikaz transformisane kritične funkcije akceleratora

Razlike u odnosu na prethodnu implementaciju C koda jesu sljedeće:
Promocija iteratora. Svaka promjenljiva u petlji postaje polje strukture stanja,i i j ostaju i i j ali se pomjeraju sa steka u strukturu; iterator k se stapa u stanje, dok se c potpuno eliminiše jer četiri čitanja ćoškova postaju jedno paralelno.

Promocija promjenljive. num_classifiers postaje registar, jer se test j >= num_classifiers provjerava u kasnijem ciklusu u odnosu na učitani.

Razbijanje petlji u stanja. Najugnježđenija petlja se izvršava tačno tri puta sa ranim izlazom koji je data-dependent, pa se razbija u stanja S_RECT_0, S_RECT_1, S_RECT_2. Iterator ovim postupkom u potpunosti nestaje i postaje stanje.

Skidanje early exita. Break i return ne mogu postojati u RTL modelu pošto mašina stanja nema stack. Break se pretvara u S_RECT_1 - S_CLS_FLUSH / S_RECT_1 - S_RECT_2 grananje, dok return postaje terminalna stanja S_RETURN_REJECT / S_RETURN_ACCEPT praćena stanjem S_WINDOW_END.

Inlining poziva. evalWeakClassifier se poziva unutar dvije petlje i vraća vrijednost koju koristi spoljašnja petlja. Pošto ne postoji stek poziva, može se inlineovati, pa lokalne promjenljive sum i t postaju registri, dok njegova povratna vrijednost alpha postaje accumulate u stage_sum.

Podjela pristupa memoriji. Svako čitanje iz bafera dijeli se u izdavanje adrese i potrošnju podataka, odakle dolaze stanja S_PV_ADDR / S_PV_FETCH / S_PV_LATCH i S_STAGE_ENTER / S_CLS_FETCH.

Jedan poziv po stanju. IP se izvršava do kraja; svaki poziv izvršava jedno stanje.

Ostale adaptacije koje su dio arhitekturnih promjena jesu : dijeljenja starog ddr3[] niza za bafere integralne slike u jedan bafer po redu slike, koji pravi osnovu za banke redova koje će u nastavku biti objašnjene, zatim smanjenje broja bafera za kvadriranu integralnu sliku na 2, uvođenje **cls_word_t** promjenljive kao osnove za data packing optimizaciju, promjena akumulatora tako da drži sum/4096 (pošto je 4096 zajednički težinski faktor), što je osnova za strength reduction optimizaciju, promjena načina provjere prisutnosti trećeg pravougaonika tako da se smanji broj čitanja, i implementacija hit-buffera za sweep čija će svrha kasnije biti objašnjena.

Transformacije koda

Pregled najvažnijih transformacija

Transformacija	Cijena u VP modelu	Poslije transformacije	Osnov za transformaciju
Raspored podataka klasifikatora	17 skalarnih čitanja po klasifikatoru	1 čitanje pakovanog ROM-a (112 bita)	Zajednički faktor
Množenje težinama	3×(16×32) + 1×(16×16) množenja	0 množenja; pomjeranje / negacija / pomjeraj-sabiranje	Zajednički faktor
Adresiranje integralne slike	width·ry + rx, 12 množenja po klasifikatoru	Izbor banke + 9-bitno sabiranje	Geometrija pravougaonika
Smještanje kvadrirane slike	25 redova	2 reda (banke 25 i 26)	Način čitanja slike
Pomjeranje redova kliznog prozora	2 × 24 × width kopiranja riječi po koraku	1 uvećanje brojača (row_base)	Rotacija (objašnjeno u nastavku)
Propusnost upisa redova	18.000 riječi po koraku	1.080 riječi po koraku (16,7×)	Inkrementalni upis
Handshaking PS/PL	1 prekid po prozoru (208.506)	1 prekid po redu (986)	Implementacija režima B

Tabela 5. Tabelarni pregled transformacija

Data packing ROM klasifikatora

Sva polja jednog slabog klasifikatora - geometrije tri pravougaonika, dva koda težine, prag stabla i dvije alpha konstante - pakuju se u jednu riječ od 112 bita. Klasifikacioni ROM sadrži 2913 takvih riječi i inicijalizuje se iz tekstualne datoteke pri elaboraciji.

Širine polja slijede neposredno iz izmjerenih podataka: geometrijska polja imaju 5 bita (0-24), kodovi težine 2 bita, prag stabla 13 bita, a alpha1 i alpha2 po 14 bita (opsezi −7680..1190 i −7680..2560). Pakovanje smanjuje podatke klasifikatora sa 102,5 kB na 56 kB i, što je važnije, pretvara sedamnaest uzastopnih čitanja u jedno.

```
constant CLS_COUNT : natural := 2913;   -- klasifikatori
constant CLS_AW    : natural := 12;     -- 4096 >= 2913
constant CLS_DW    : natural := 112;    -- sirina pakovane rijeci

constant KCODE_NEG1 : "00";  -- tezina  -4096
constant KCODE_2    : "01";  -- tezina   8192
constant KCODE_3    : "10";  -- tezina  12288
constant WEIGHT_SHIFT : natural := 12;  -- 4096 = 1 << 12
```

Listing 8. Implementacija pakovane riječi u vj_pkg.vhd

Kodovanje težina pomjeranjem i sabiranjem

Pošto je svaka težina oblika 4096*k sa k ∈ {−1, 2, 3}, težinska suma pravougaonika računa se ovako: k = −1 je negacija, k = 2 je pomjeranje ulijevo za jedno mjesto, a k = 3 je pomeranje ulijevo za jedno mesto uvećano za nepomjerenu vrednost. Zajednički činilac 4096 primjenjuje se jednom, kao pomjeranje za WEIGHT_SHIFT, na kraju akumulacije umjesto po pravougaoniku.

Posljedica je vidljiva u kasnijem izvještaju o resursima: dizajn koristi ukupno četiri DSP48E1 bloka i nijedan od njih nije na putanji po klasifikatorima. Oni služe računanju varijanse koje se obavlja jednom po prozoru.

Integralna slika po bankama redova

Klizni integralni bafer sadrži 25 redova dužine do 360 piksela. Umjesto jedne široke memorije adresirane izračunatim pomjerajem, on je podijeljen na 25 nezavisnih blokovskih memorija sa dva puna porta, dimenzija 512×32 bita, po jedna za svaki logički, uz još dvije banke za dva stvarno korišćena reda kvadrirane slike.

Konstanta	Vrijednost	Značenje
MAX_IMG_WIDTH	360	Najšira skala piramide (faktor 1)
WIN_SIZE	24	Prozor detekcije je 24 × 24
INT_ROWS	25	Broj redova u kliznom baferu
BANK_DEPTH / BANK_AW	512 / 9	≥ MAX_IMG_WIDTH, zaokruženo na stepen dvojke
N_INT_BANKS	25	Banke integralnih redova
N_BANKS	27	25 integralnih + 2 kvadrirane
SQ0_BANK / SQ23_BANK	25 / 26	Kvadrirana slika, logički redovi 0 i 23
HIT_DEPTH / HIT_AW	512 / 9	Bafer detekcija u režimu B

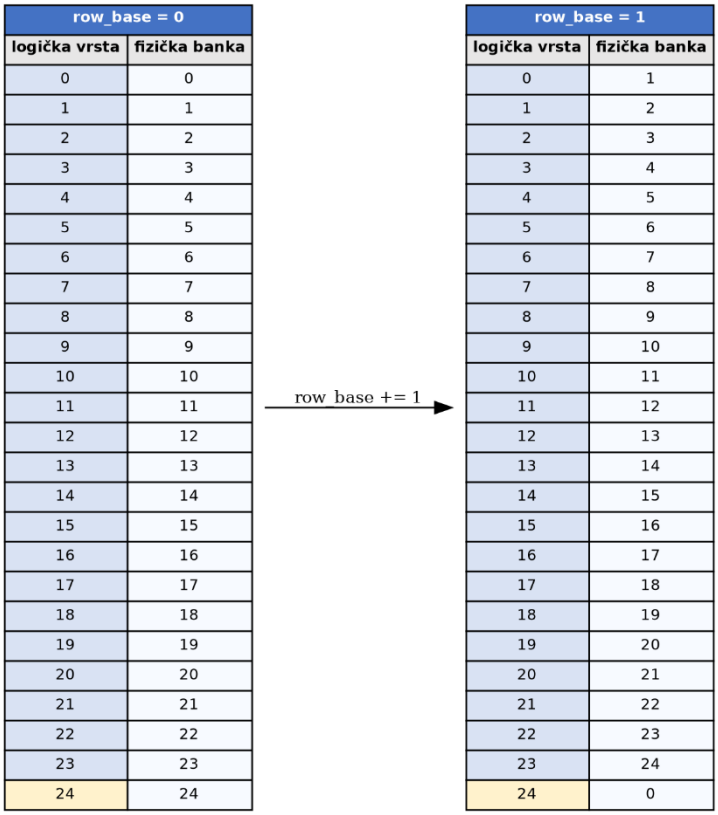
Tabela 6. Implementacija pokavane riječi u vj_pkg.vhd

Pošto je svaka banka nezavisno adresibilna i ima dva porta, dva čitanja čoškova u gornjem i dva u donjem redu pravougaonika izdaju se istovremeno: sva četiri ćoška jednog pravougaonika stižu u jednom taktu, bez ikakve adresne aritmetike osim 9-bitnog sabiranja kolone. Konfiguracija 512×32 sa dva puna porta ne staje u RAMB18, pa svaka banka troši cijeli RAMB36 blok. Tih 27 banki, zajedno sa ROM-om klasifikatora, razlog su što je blokovska memorija najuže grlo u dizajnu, sa zauzećem od 65%.

Rotacija redova (row_base)

Pomjeranje prozora naniže za jedan red u VP bi značilo kopiranje 24 reda oba bafera za jedno mjesto naviše. Umjesto toga, banke se adresiraju preko funkcije rotacije definisane u datoteci `vj_pkg.vhd`:

```
rot_bank(logicki_red, row_base) = (logicki_red + row_base) mod 25
```



Slika 6. Objašnjenje rotacije banki

Pomjeranje prozora tada je jedno uvećanje veličine row_base, a red koji izlazi iz prozora postaje red koju naredni upis prepisuje. Kopiranje složenosti O(width) postaje uvećanje brojača složenosti O(1). Rotacija se na isti način primenjuje u hardveru i u upravljačkom programu, a oba se unakrsno provjeravaju prema referenci bez rotacije.

Inkrementalni upis redova

Za svaki korak prozora naniže potrebno je prenijeti samo jedan novi integralni red i dva kvadrirana reda. VP pristup ponovo prenosi oba kompletna bafera.

Pristup	Riječi po koraku	Upisa u BRAM, cijela slika
Ponovni upis oba bafera (pristup virtuelne platforme)	18.000	11.151.750
Inkrementalni upis (ovaj dizajn)	1.080	714.753
Smanjenje	16,7×	15,6×

Tabela 7. Poređenje dobiti smanjenjem broja upisa u integralne redove

Bez ove transformacije samo vrijeme prenosa sa procesora dominiralo bi ukupnim vremenom i akcelerator bi bio besmislen.

Režim B: pretraga reda u hardveru

U režimu A procesorski sistem klasifikuje jedan prozor po pozivu: upiše koordinatu x, generiše impuls start, sačeka done i pročita rezultat. To vjerno preslikava način programiranja virtuelne platforme i zadržano je zbog poređenja, ali za cijelu sliku zahtijeva 208.506

handshakeova, od kojih svaki košta oko dvije mikrosekunde vremena na AXI magistrali - daleko više od same klasifikacije, koja u prosjeku traje 0,62 µs.

U režimu B softver upisuje prvu i posljednju koordinatu x i korak, pa jednom generiše impuls start. Konačni automat zatim samostalno pretražuje cijeli red, upisuje detekcije u FIFO na čipu i podiže jedan prekid kada završi. Za cijelu sliku potrebno je 986 handshakeova umesto 208.506.

FIFO bafer detekcija ima 512 mjesta. Najšira skala ima 336 položaja prozora po redu, pa je prekoračenje strukturno nemoguće, a ne samo malo vjerovatno.

Pipelining pravougaonika

stanje	r0	r1	r0	r1	r2	r0	r1	drain	drain	drain	drain	drain	eval	init	
s0 izdavanje/adresa	k0 r0	k0 r1	k1 r0	k1 r1	k1 r2	k2 r0	k2 r1								k0,k1,k2-klasifikatori (uzastopni, slabi) r0,r1,r2-rectovi klasifikatora
s1 BRAM output reg		k0 r0	k0 r1	k1 r0	k1 r1	k1 r2	k2 r0	k2 r1							
s2 corner mux reg			k0 r0	k0 r1	k1 r0	k1 r1	k1 r2	k2 r0	k2 r1						
s3 rs_r = (a+d) - (b+c)				k0 r0	k0 r1	k1 r0	k1 r1	k1 r2	k2 r0	k2 r1					
s4 weight + s_acc					k0 r0	k0 r1	k1 r0	k1 r1	k1 r2	k2 r0	k2 r1				
s5 finalizacija i suma						k0 r0	k0 r1	k1 r0	k1 r1	k1 r2	k2 r0	k2 r1			
takt	t+0	t+1	t+2	t+3	t+4	t+5	t+6	t+7	t+8	t+9	t+10	t+11	t+12	t+13	
	CIKLUS IZDAVANJA PRAVOUGAONIKA (II =1)							FLUSH OD 5 CIKLUSA							

Tabela 8. Prikaz implementiranog pipelinea kroz taktove

Prefetch za pipelining pravougaonika implementiran je tako što se izraz ctl_rom_addr = haar + 1 nalazi u stanju ST_R0, pa se sljedeći klasifikator fetchuje dok se za trenutni izdaju pravougaonici. Bez njega, svaki klasifikator bi koštao dodatan ciklus kašnjenja iz klasifikacione memorije, što je 2913 dodatnih ciklusa po potpuno pređenom prozoru, što je povećanje od 40% u odnosu na mjerenu vrijednost ciklusa.

Umjesto implementacije kontrolera za svaku fazu pipelinea, kontrola je implementirana tako što se u naredne faze protočne obrade uz podatke prenose signali (valid,first,last,kcode,t,a1,a2) u tag_d1-tag_d4 registrima. Jedan od razloga ovakve implementacije jeste to što se registar t_reg prepisuje sljedećim klasifikatorom jedan ciklus nakon izdavanja posljednjeg pravougaonika za trenutni klasifikator, ali se poređenje pragova

dešava 4 ciklusa kasnije. Čitanje t_reg registra direktno bi bio write-after-read (WAR) hazard na svakom klasifikatoru, međutim ovakvom implementacijom pipelinea i prenošenjem promjenljive t u tagu hazard je strukturno nemoguć.

Bit-width narrowing

Poljima slabog klasifikatora i datapath registara smanjuju se bitske širine na sljedeći način:
- polja geometrije pravougaonika (x,y,w,h) iz **int** tipa prelaze u 5-bitne vrijednosti, jer se sva geometrija pravougaonika dimenzija 24*24 može opisati sa 5 bita.

- indeksi kolona i redova iz **int** tipa prelaze u 9-bitne, odnosno 5-bitne vrijednosti (maksimalno 360 kolona i 25 redova, što se može predstaviti tim vrijednostima).

- tree_thresh,alpha1 i alpha2 postaju redom 13-bitni, 14-bitni i 14-bitni označeni tipovi, jer su njihove vrijednosti u .dat fajlovima takve da se mogu predstaviti tim brojem bitova. Slično, maksimalna vrijednost za stage_sum može biti predstavljena sa 24 bita, pa se tako i tu vrši smanjenje.

- vnf iz **float** tipa postaje 16-bitni neoznačeni tip, jer je rezultat cjelobrojnog korijena

- indeks klasifikatora postaje 9-bitna vrijednost, jer je ukupan broj klasifikatora 2913.

Na ovaj način se klasifikator koji bi zauzeo sedam 32-bitnih riječi smanjuje na 112-bitova, što nam omogućava prethodno opisanu data-packing transformaciju.

Manje transformacije vrijedne pomena

Loop-invariant hoisting

Dvije veličine se izračunavaju na najvišem nivou na kome su konstantne:

vnf (faktor normalizacije varijanse) - jednom po prozoru, u stanjima ST_VAR_*

t_reg = tree_thresh * vnf - jednom po slabom klasifikatoru, u stanju ST_R0

U uobičajenoj formulaciji, poređenje sa pragom se zapisuje kao sum ≥ tree_thresh * vnf unutar petlje po pravougaonicima. Izdizanjem tog izraza iz petlje, množenje koje bi se izvršavalo za svaki pravougaonik svodi se na jedno množenje po klasifikatoru.

Loop folding za operaciju cjelobrojnog korijena

vj_isqrt funkcija odrađuje dvije referentne iteracije po taktu, pa se tako korijen iz originalnog koda od 16 iteracija završava u 8 ciklusa. Ovo nije pipelining, pošto je jedinica blokirajuća, sa kašnjenjem od 8 taktova i intervalom inicijalizacije od 8 taktova. Stanje ST_VAR_WAIT traje 11 ciklusa pri pokretanju korijena, odnosno jedan ciklus kada je varijansa nulta.

Odbačena optimizacija: paralelizacija čitanja pravougaonika klasifikatora replikacijom banki

Replikacija banki redova omogućila bi da se sva tri pravougaonika klasifikatora čitaju paralelno. Ta mogućnost je razmotrena i odbačena na osnovu dokaza:

Na osnovu izvršavanja, samo 1,5% klasifikatora koji se stvarno izvrše ima treći pravougaonik. Stepeni 0 i 1 - koji zajedno odlučuju o ishodu za 80,6% svih prozora - uopšte ne sadrže klasifikatore sa tri pravougaonika.

Svaka replika košta dodatnih 25 RAMB36 blokova. Dizajn već koristi 39 od 60 raspoloživih na uređaju. Ni jedna jedina replika ne staje.

Druga tvrdnja potvrđena je sintezom, a na većem sistemu odnos bi trebalo ponovo razmotriti, a tada bi prva tvrdnja - da je dobit ionako zanemarljiva - postala presudna.

Pojašnjenje implementacije algoritma

Struktura kaskade i rani izlazak

Klasifikator predstavlja rezultat izvršavanja kaskade slabijih klasifikatora. Svaka od 25 skala sadrži izvjestan broj slabih klasifikatora; svaki slabi klasifikator računa Haarovu karakteristiku nad dva ili tri pravougaonika, poredi normalizovani odziv sa sopstvenim pragom i doprinosi jednoj od dvije konstante (alpha1 ili alpha2) tekuće sume faze. Na kraju faze suma se poredi sa pragom faze. Ako je manja, prozor se odmah odbacuje i naredne faze se uopšte ne računaju.

Kaskada zato nije proračun fiksne cijene. Njena cijena po prozoru varira više od dva reda veličine u zavisnosti od sadržaja slike, i upravo je ta promjenljivost najvažnije svojstvo za arhitekturu.

Integralna slika i normalizacija varijansom

Suma nad pravougaonikom dobija se iz integralne slike pomoću četiri čitanja ćoškova i dva sabiranja odnosno oduzimanja. Prije računanja bilo kog obilježja prozor se normalizuje varijansom: iz integralne i kvadrirane integralne slike računaju se srednja vrijednost i srednja vrijednost kvadrata nad prozorom 24×24, a dobijena standardna devijacija skalira prag svakog klasifikatora.

Faktor normalizacije zahtijeva cjelobrojni kvadratni koren. On je realizovan u datoteci `vj_classifier_rom.vhd` kao jedinica koja sažima dvije referentne iteracije u jedan takt, pa se 16 referentnih iteracija završava za 8 taktova. Pošto se izvršava jednom po prozoru, a ne jednom po klasifikatoru, taj trošak se amortizuje nad cijelim prozorom i nije na kritičnoj putanji propusnosti.

Izmjerena svojstva podataka klasifikatora

Tri svojstva klasifikacionih podataka utvrđena su neposrednim pregledom svih 2913 klasifikatora. To nisu pretpostavke, nego dodatne olakšice pri dizajniranju načina rada sistema, a cijela hardverska arhitektura počiva na njima.

Svojstvo 1 - svaka težina je umnožak broja 4096

Za svaki klasifikator važi `weight[0] = −4096` - konstanta, pa se nikada ne čuva. Dalje, `weight[1] ∈ {8192, 12288}` (jedan bit informacije) i `weight[2] ∈ {0, 8192}` (jedan bit, pri čemu je vijrednost različita od nule tačno ekvivalentna tvrdnji „postoji treći pravougaonik" - provjereno i tačno za svih 2913 klasifikatora).

Ako se svaka težina zapiše kao 4096*k gdje k ∈ {−1, 2, 3}, težinska suma se razlaže na pomjeranja i sabiranja. Posljedica je da u unutrašnjoj petlji po klasifikatorima nema nijednog množača.

Svojstvo 2 - geometrija pravougaonika je ograničena prozorom

Sva geometrijska polja leže u opsegu 0-24, a uvijek važi `ry+rh ≤ 24` i `rx+rw ≤ 24`. U virtuelnoj platformi, adresa u integralnoj slici računa se kao `width·ry + rx`, što je jedno množenje po ćošku - dvanaest množenja po klasifikatoru sa tri pravougaonika. Pošto ry može izabrati samo jedan od 25 logičkih redova, to množenje se svodi na izbor banke i sabiranje 9-bitne kolone. Odatle potiče arhitektura memorije po bankama.

Svojstvo 3 - kvadrirana integralna slika čita se na tačno jednom mjestu

Kvadrirana integralna slika sampluje se isključivo u četiri ćoška samog prozora, koji poslije pomenutog redizajna uvijek leže u logičkim redovima 0 i 23. Referentni model iz VP čuva svih 25 redova; 23 se nikada ne pročitaju, dok naš hardverski čuva dva.

Statistika ranog izlaska koja oblikuje cijeli dizajn

Mjereno nad svih 13 skala piramide i 9.640 testnih prozora:

Mjerena veličina	**Vrijednost**	**Posljedica po arhitekturu**
Prozori odbačeni u fazi 0	80,6%	Dominira cijena pripreme prozora, a ne propusnost unutrašnje petlje
Prozori odbačeni u prve tri faze	99,2%	Pipelining prozora bi bio uzaludan, potencijalni starvation
Prosječan broj klasifikatora po prozoru	14,6	Prema najgorem slučaju od 2913 - raspon 200×
Klasifikatori sa trećim pravougaonikom (statički)	19%	Putanja sa trećim pravougaonikom je rijedak slučaj
Klasifikatori sa trećim pravougaonikom (sa ovakvim izvršavanjem)	1,5%	Njena akceleracija potpuno besmislena

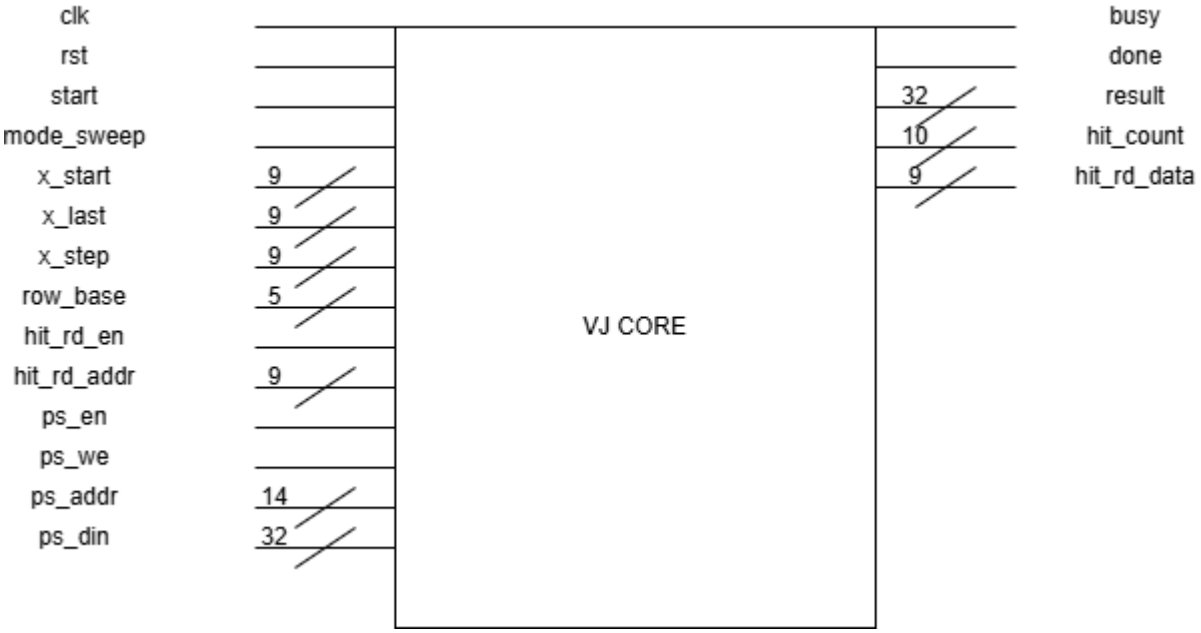
Tabela 9. Statistički parametri kaskadne klasifikacije

Zaključak za projektovanje je neposredan: fiksni troškovi po prozoru znače daleko više od propusnosti po klasifikatoru. Smanjenje cijene pripreme prozora i smanjenje broja handshakeova sa procesorom donose mnogo više od ubrzanja petlje koja se u prosjeku izvrši samo 14 puta.

Specifikacija interfejsa jezgra

IP jezgro nudi procesorskom sistemu dva interfejsa: AXI4-Lite slave interfejs za upravljanje i status, i BRAM port A za masovni upis redova, kojim upravlja AXI BRAM kontroler. Razdvajanje je namjerno: upravljački transfer čini nekoliko pojedinačnih transakcija po redu,

dok je upis reda burst od preko hiljadu riječi koji bi pojedinačni AXI4-Lite upisi obesmislili.



Slika 7. IP jezgro prije pakovanja, sa interfejsima

Interfejsi se mogu podijeliti u nekoliko grupa:
- **clk i reset** predstavljaju zaseban interfejs i standardni su signali u RTL dizajnu ovakvih sistema.

- **start i mode_sweep** čine komandni interfejs, gdje start pokreće rad IP jezgra, a mode_sweep određuje režim rada IP jezgra.

- Interfejs ulaznih podataka čine signali **x_start** , **x_last** , **x_step i row_base**. Oni služe kao pomoć za upis ulaznih podataka vezanih za operacije s bankama redova.

- **hit_rd_en** i **hit_rd_addr** predstavljaju signale koji se koriste za pristupanje bufferu pogodaka pri detekciji.

- **busy i done** predstavljaju statusni interfejs IP-ja.

- **result**, **hit_count i hit_rd_data** predstavljaju interfejs za izlazne podatke, koji sadrže podatke o rezultatu detekcije, broju pogodaka i podacima sa adrese u FIFO baferu za pogotke.

- **ps_en, ps_we, ps_addr i ps_din** predstavljaju interfejs za upis banki redova.

Za razliku od reference, memorije se instanciraju unutar modula, pa s tog aspekta ne postoji eksterni memorijski interfejs za njih, što se ogleda u tome što su jedini memorijski portovi koje imamo zapravo putanje za upis redova slika, a klasifikaciona memorija uopšte nema eksterni port jer se inicijalizuje iz ROM_INIT_FILE pri konfiguraciji. Mod A upravlja signalom

x_start i očitava rezultat, dok mod B dodatno upravlja signalima mode_sweep, x_last i x_step, pa zatim čita hit_count i prelazi hit buffer do hit_rd_addr/hit_rd_data. Signal row_base bira koja fizička banka drži logički red 0, kako bi red prozora mogao da se mijenja bez ponovog upisa svih 25 banki.

Registarska mapa

Offset	Naziv	Pristup	Opis
0x00	CTRL	RW	b0 RESET (sam se briše); b1 START (sam se briše); b2 MODE, 0 = jedan prozor, 1 = pretraga po x; b4 DONE, postavlja hardver, briše se upisom jedinice
0x04	X_START	RW	Koordinata x prozora (režim A) / prva koordinata x pretrage (režim B)
0x08	WIDTH	RW	Širina skale. Zadržan radi saglasnosti sa virtuelnom platformom; datapath ga više ne koristi jer je adresiranje po bankama uklonilo množenje širinom
0x0C	RESULT	RO	Ishod u režimu A: 1 = lice; −i = odbačeno u fazi i
0x10	X_LAST	RW	Režim B: posljednja koordinata x
0x14	X_STEP	RW	Režim B: korak pomjeraja između prozora
0x18	HIT_COUNT	RO	Režim B: broj detekcija u baferu
0x1C	HIT_DATA	RO	Režim B: element bafera na adresi HIT_ADDR
0x20	ROW_BASE	RW	Rotacija 25 integralnih banki, 0..24
0x24	STATUS	RO	b0 BUSY
0x28	HIT_ADDR	RW	Indeks u baferu detekcija koji koristi HIT_DATA

Tabela 10. Registarska mapa IP jezgra

Interfejs BRAM_PORTA za masovni upis

Port za masovni upis implementiran u upakovanom jezgru čine signali `bram_clk_a`, `bram_en_a`, `bram_we_a[3:0]`, `bram_addr_a[31:0]`, `bram_wrdata_a[31:0]` i `bram_rddata_a[31:0]`. Sa stanovišta procesorskog sistema port je samo za upis; signal pročitanih podataka vezan je na nulu.

Dekodovanje adrese je sljedeće:

```
adresa rijeci = bram_addr_a(15 downto 2)

  biti 13..9   izbor banke   0..24 = integralni redovi
                             25    = kvadrirana slika, logicki red 0
                             26    = kvadrirana slika, logicki red 23
  biti  8..0   kolona        0..511
```

Odgovarajući pogled iz softvera je flat adresa riječi, `banka × 512 + kolona`, što i računa funkcija bare-metal aplikacije `vj_upload_row()`.

Dogovorena ograničenja

Dva ograničenja obezbjeđena su dogovorom, a ne hardverom. Oba daju pogrešne rezultate bez ikakve poruke o grešci ako se prekrše, pa su zato zabilježena i u zaglavlju RTL koda i ovdje.

Signal bram_clk_a mora biti isti takt kao s00_axi_aclk. Banke redova taktuje AXI takt, a bram_clk_a je unutar jezgra namjerno ostavljen nepovezan. Pobuđivanje BRAM porta drugim taktom pokvarilo bi podatke bez vidljivog simptoma. U blok dizajnu oba potiču iz FCLK_CLK0.

Softver ne smije upisivati podatke redova dok je jezgro zauzeto. Port A svake banke dijeli se između putanje upisa i čitanja nulte kolone iz jezgra, uz pretpostavku da se to dvoje nikada ne preklapa. Redovi se smiju upisivati samo poslije signala DONE, nikada tokom rada.

Redoslijed programiranja

Režim A — jedan prozor

```
vj_set_width(width);          /* saglasnost sa VP */
vj_set_row_base(base);
for (r = 0; r < 25; r++)      /* pun upis, samo za prvi prozor */
    vj_upload_row(r, &sum[r*width], width);
vj_upload_row(25, &sqsum[0],       width);
vj_upload_row(26, &sqsum[23*width], width);

result = vj_run_window(x);    /* upis X_START, impuls START,
                                 ispitivanje STATUS.BUSY, citanje RESULT */
/* result ==  1  -> lice                                          */
/* result == -i  -> odbaceno u fazi i                          */
```

Režim B — pretraga reda

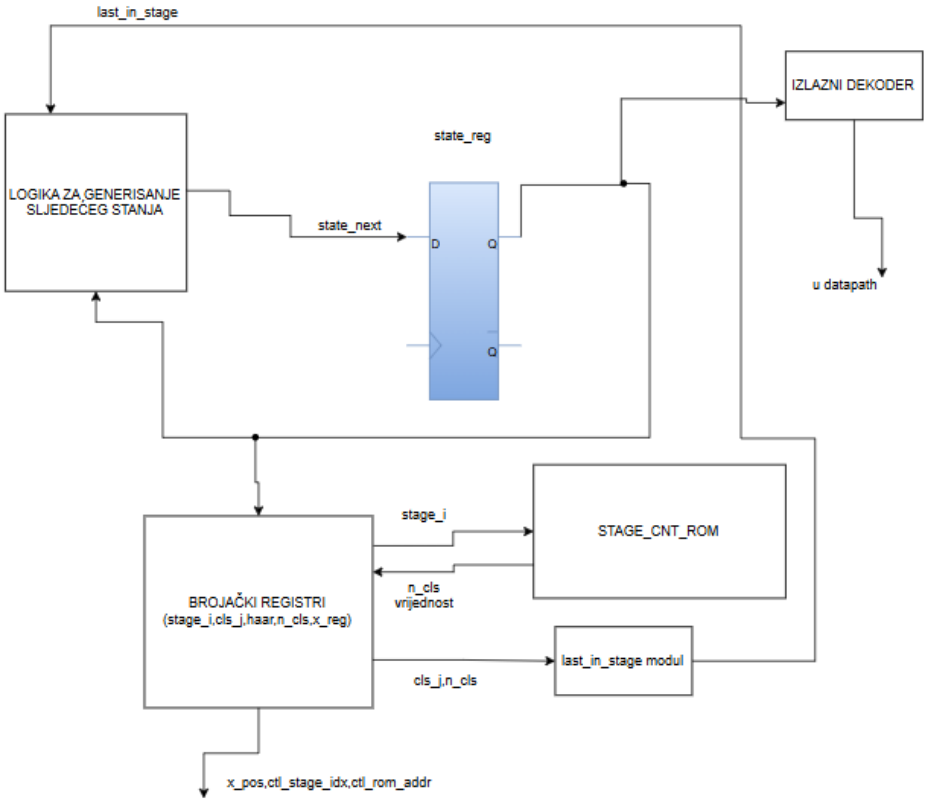
```
vj_set_row_base(base);
vj_upload_row(rot_bank(24, base), new_row, width);
vj_upload_row(25, sq_row0,  width);
vj_upload_row(26, sq_row23, width);

n = vj_run_sweep(x0, x_last, x_step);   /* jedan START po redu */
for (i = 0; i < n; i++)
    packed = vj_hit(i);                 /* HIT_ADDR pa HIT_DATA */
```

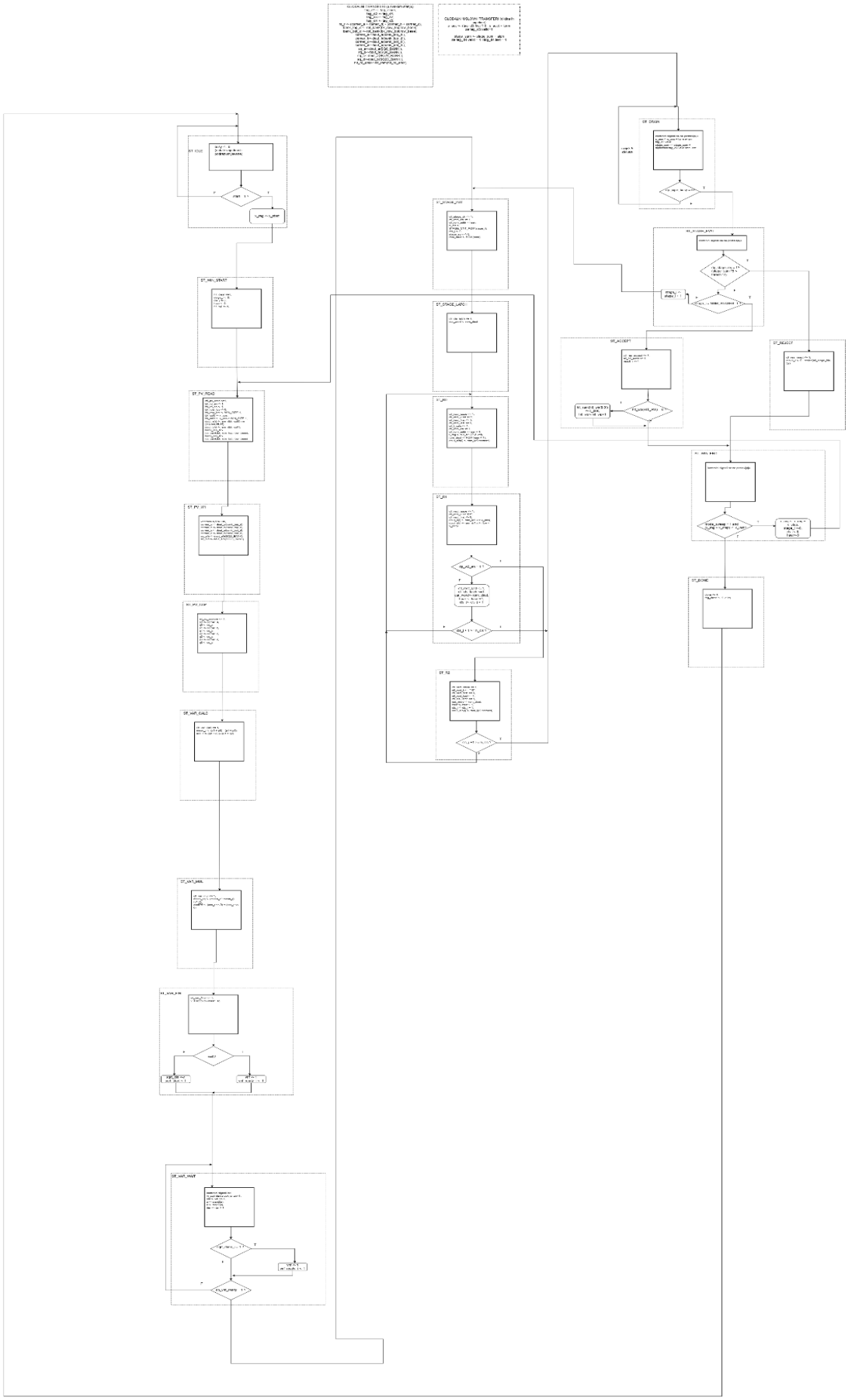
Upravljački program koji nudi ove funkcije predstavlja bare-metal Zynq aplikaciju.

Controlpath

Controlpath čini konačni automat sa 20 stanja u datoteci `vj_controlpath.vhd`, zajedno sa brojačima prozora, faze, klasifikatora i pravougaonika i dekodovanjem izlaza. U njemu nema nikakve aritmetike nad podacima slike; sav takav proračun pripada datapathu.



Slika 8. Unutrašnja struktura controlpatha



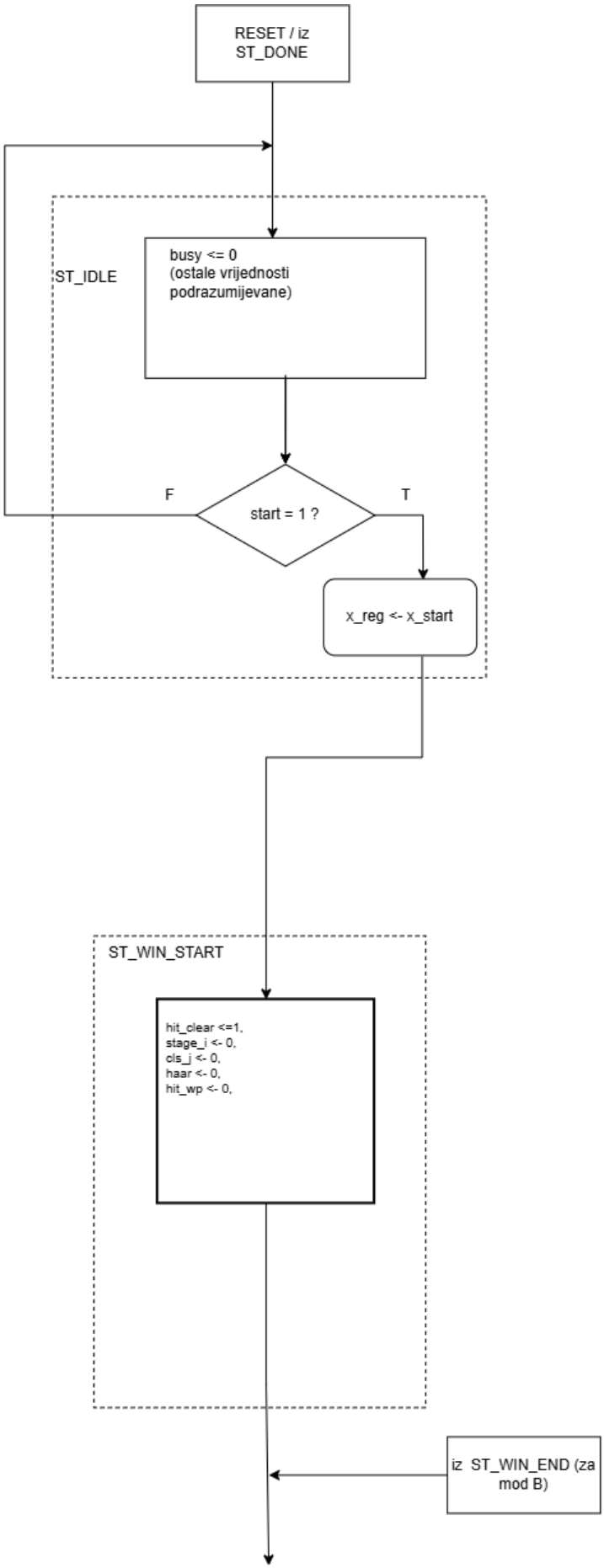
GLOBALNI TRANSFERI (u svakom stanju)

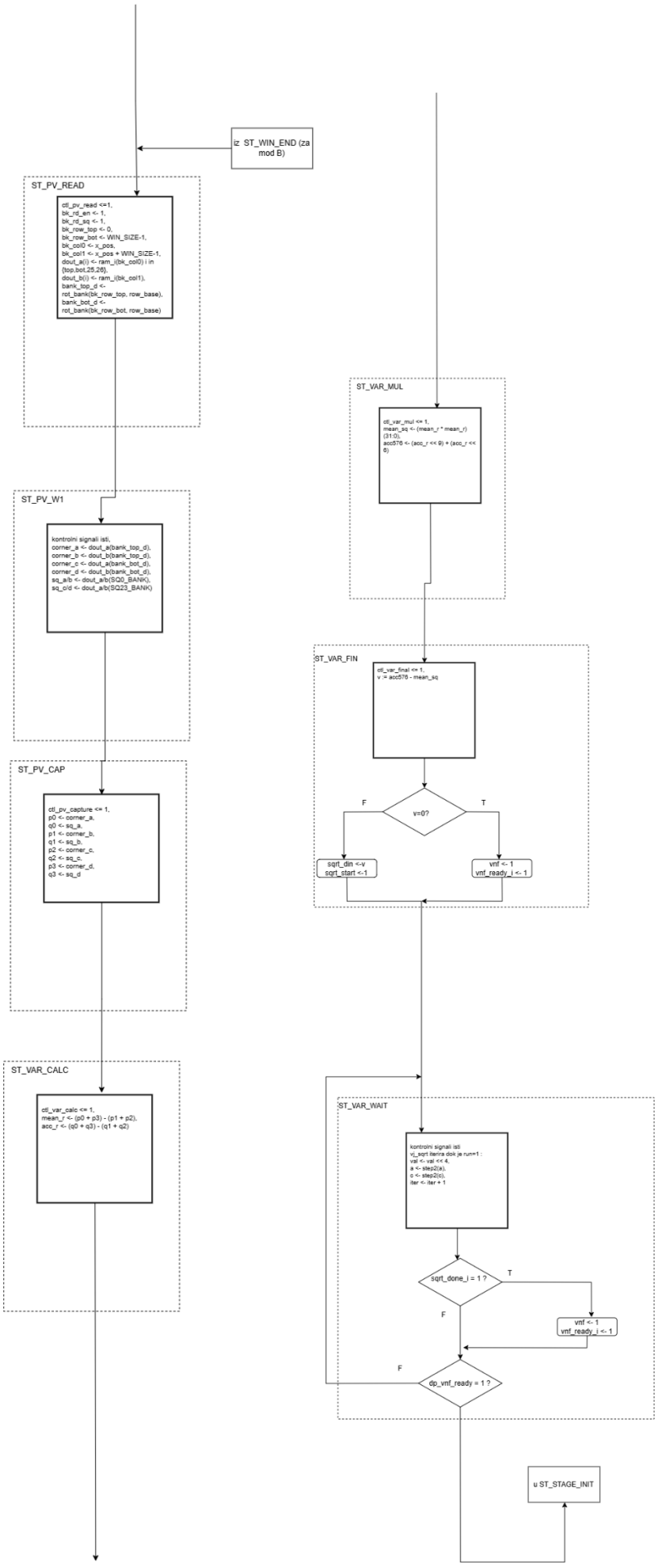
tag_d1 <- tag_issue,
tag_d2 <- tag_d1,
tag_d3 <- tag_d2,
tag_d4 <- tag_d3,
rs_r <- (corner_a + corner_d) - (corner_b + corner_c),
bank_top_d <- rot_bank(bk_row_top,row_base),
bank_bot_d <- rot_bank(bk_row_bot,row_base),
corner_a<-dout_a(bank_top_d),
corner_b<-dout_b(bank_top_d),
corner_c<-dout_a(bank_bot_d),
corner_d<-dout_b(bank_bot_d),
sq_a<-dout_a(SQ0_BANK),
sq_b<-dout_b(SQ0_BANK),
sq_c<-dout_a(SQ23_BANK),
sq_d<-dout_b(SQ23_BANK),
hit_rd_data<-hit_ram(hit_rd_addr)

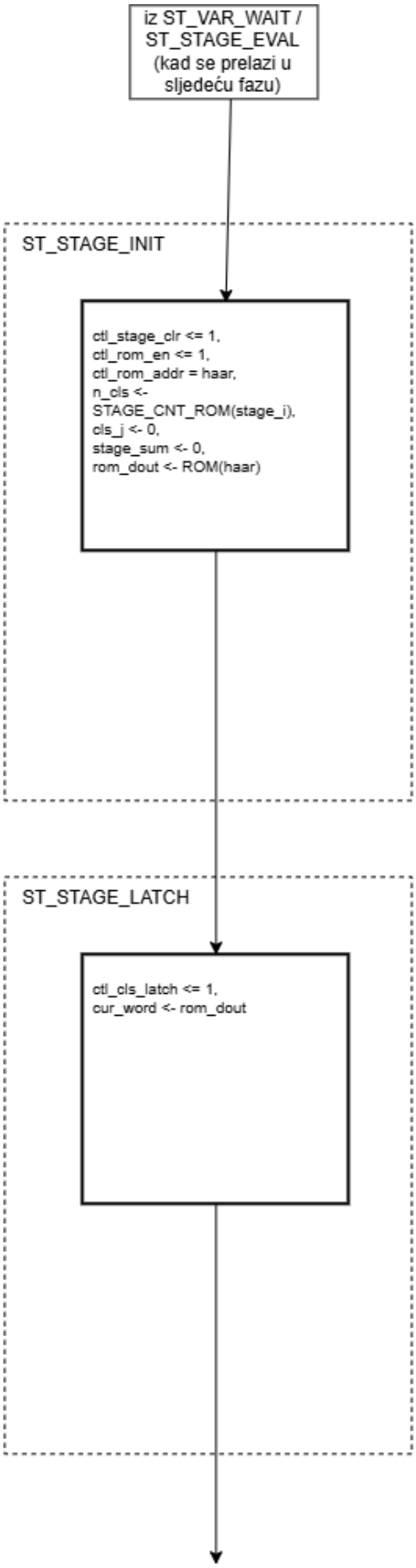
GLOBALNI USLOVNI TRANSFERI (diktira ih pipeline)

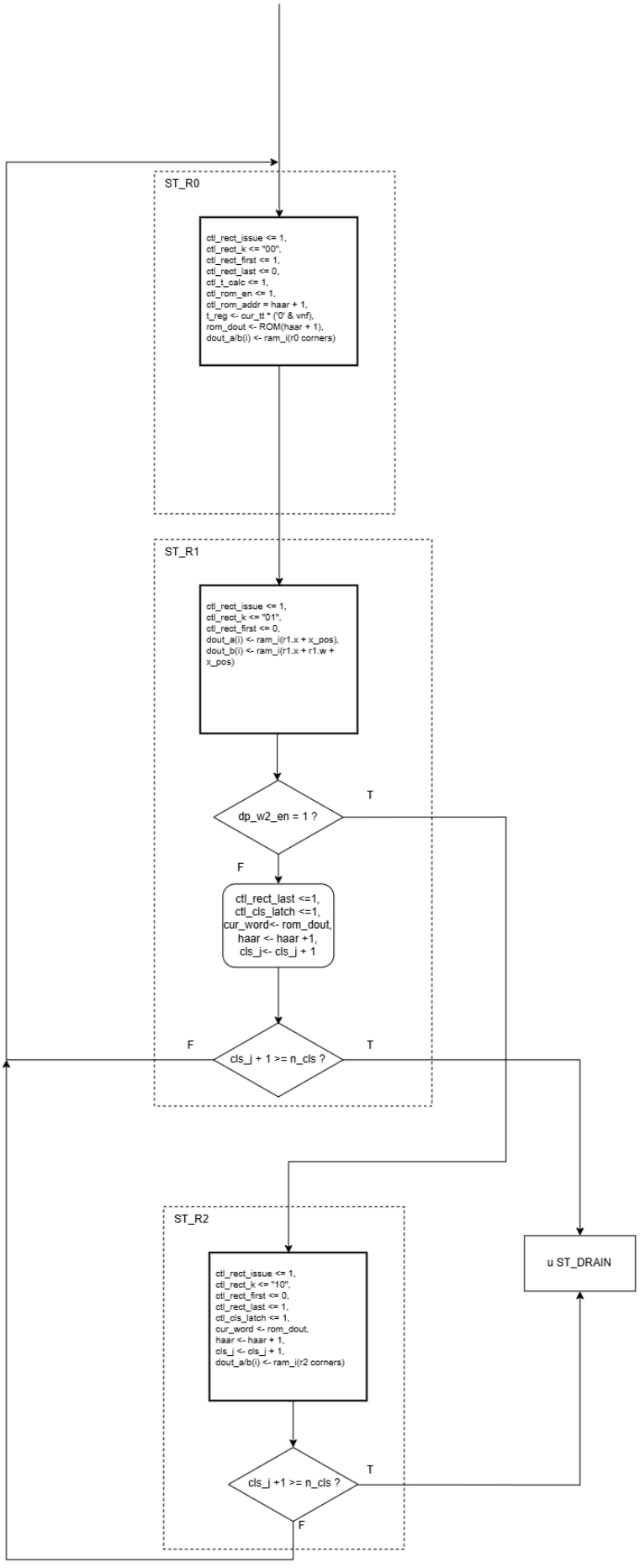
s_acc <- (tag_d3.first ? 0 : s_acc) + term
za tag_d3.valid=1

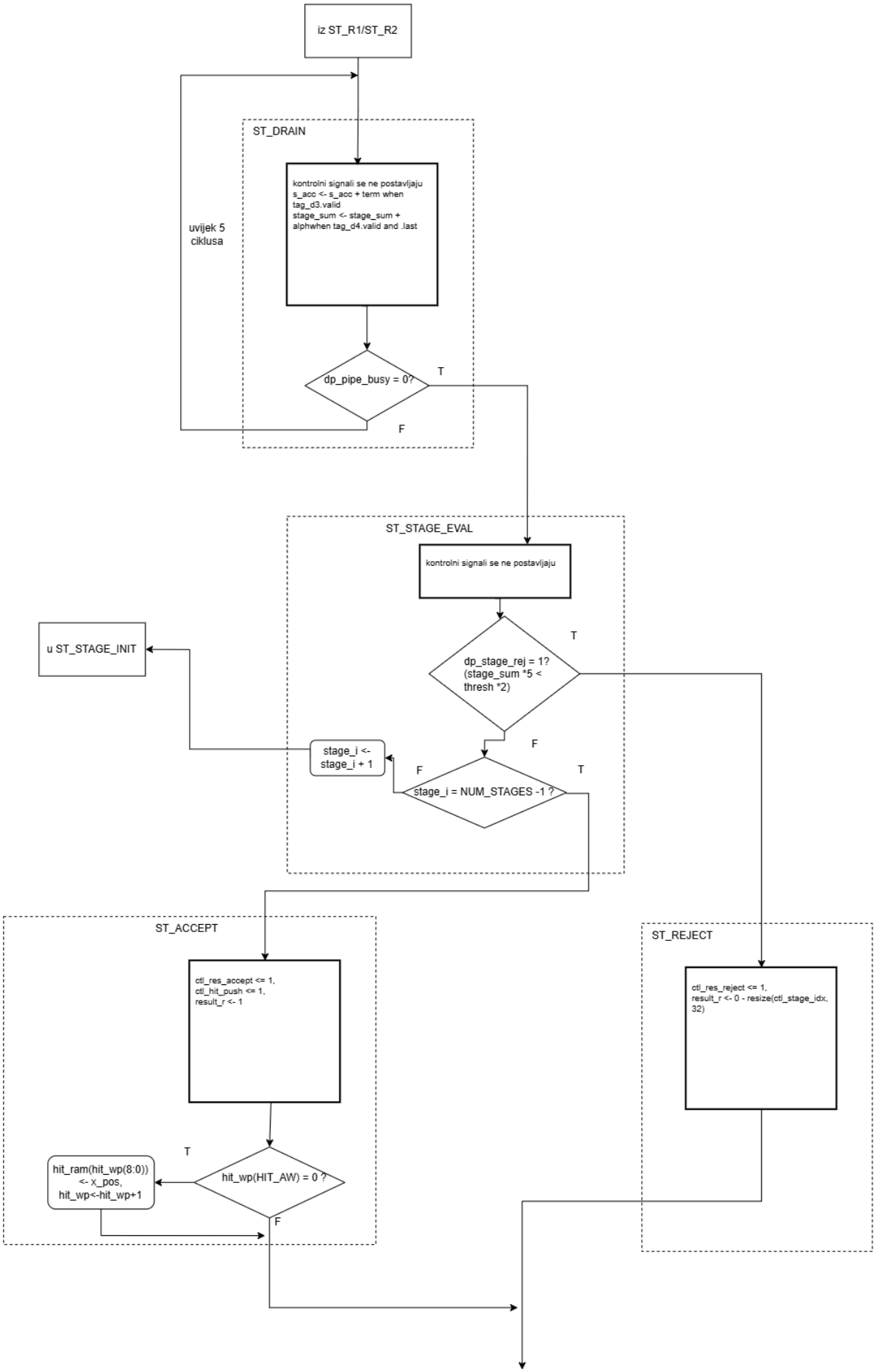
stage_sum <- stage_sum + alph
za tag_d4.valid = 1 i tag_d4.last = 1

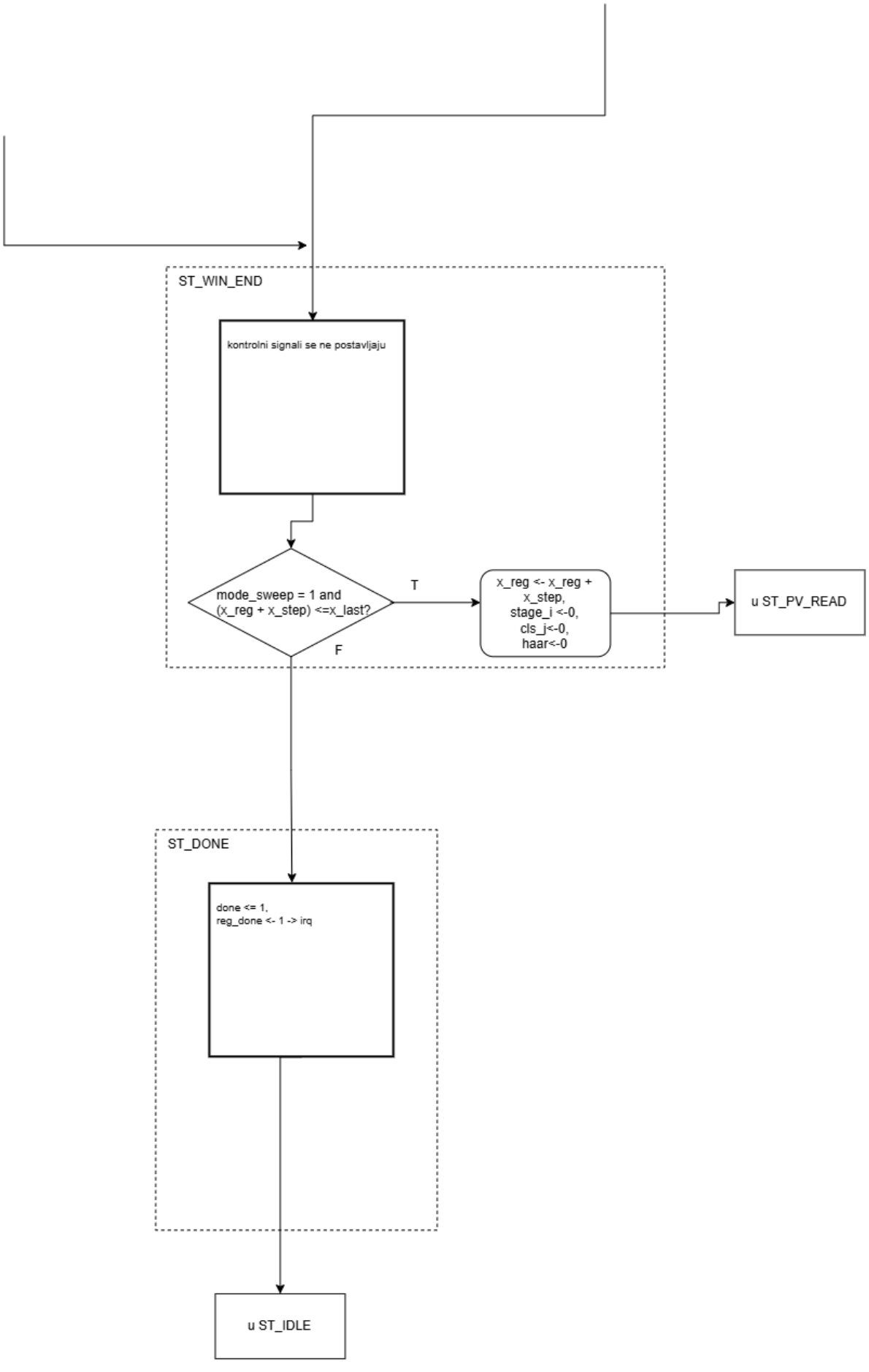

RESET / iz
ST_DONE
ST_IDLE
busy <= 0
(ostale vrijednosti
podrazumijevane)
F
start = 1 ?
T
x_reg <- x_start
ST_WIN_START
hit_clear <=1,
stage_i <- 0,
cls_j <- 0,
haar <- 0,
hit_wp <- 0,
iz ST_WIN_END (za
mod B)


iz ST_WIN_END (za mod B)
ST_PV_READ
ctl_pv_read <=1,
bk_rd_en <- 1,
bk_rd_sq <- 1,
bk_row_top <- 0,
bk_row_bot <- WIN_SIZE-1,
bk_col0 <- x_pos,
bk_col1 <- x_pos + WIN_SIZE-1,
dout_a(i) <- ram_i(bk_col0) i in (top,bot,25,26),
dout_b(i) <- ram_i(bk_col1),
bank_top_d <-
rot_bank(bk_row_top, row_base),
bank_bot_d <-
rot_bank(bk_row_bot, row_base)
ST_PV_W1
kontrolni signali isti,
corner_a <- dout_a(bank_top_d),
corner_b <- dout_b(bank_top_d),
corner_c <- dout_a(bank_bot_d),
corner_d <- dout_b(bank_bot_d),
sq_a/b <- dout_a/b(SQ0_BANK),
sq_c/d <- dout_a/b(SQ23_BANK)
ST_PV_CAP
ctl_pv_capture <= 1,
p0 <- corner_a,
q0 <- sq_a,
p1 <- corner_b,
q1 <- sq_b,
p2 <- corner_c,
q2 <- sq_c,
p3 <- corner_d,
q3 <- sq_d
ST_VAR_CALC
ctl_var_calc <= 1,
mean_r <- (p0 + p3) - (p1 + p2),
acc_r <- (q0 + q3) - (q1 + q2)
ST_VAR_MUL
ctl_var_mul <= 1,
mean_sq <- (mean_r * mean_r) (31:0),
acc576 <- (acc_r << 9) + (acc_r << 6)
ST_VAR_FIN
ctl_var_final <= 1,
v := acc576 - mean_sq
v=0?
F
T
sqrt_din <-v
sqrt_start <-1
vnf <- 1
vnf_ready_i <- 1
ST_VAR_WAIT
kontrolni signali isti
vi_sqrt iterira dok je run=1 :
val <- val << 4,
a <- step2(a),
c <- step2(c),
iter <- iter + 1
sqrt_done_i = 1 ?
T
F
vnf <- 1
vnf_ready_i <- 1
F
dp_vnf_ready = 1 ?
u ST_STAGE_INIT


iz ST_VAR_WAIT / ST_STAGE_EVAL (kad se prelazi u sljedeću fazu)
ST_STAGE_INIT
ctl_stage_clr <= 1,
ctl_rom_en <= 1,
ctl_rom_addr = haar,
n_cls <- STAGE_CNT_ROM(stage_i),
cls_j <- 0,
stage_sum <- 0,
rom_dout <- ROM(haar)
ST_STAGE_LATCH
ctl_cls_latch <= 1,
cur_word <- rom_dout


ST_R0
ctl_rect_issue <= 1,
ctl_rect_k <= "00",
ctl_rect_first <= 1,
ctl_rect_last <= 0,
ctl_t_calc <= 1,
ctl_rom_en <= 1,
ctl_rom_addr = haar + 1,
t_reg <- cur_tt * ('0' & vnf),
rom_dout <- ROM(haar + 1),
dout_a/b(i) <- ram_i(r0 corners)
ST_R1
ctl_rect_issue <= 1,
ctl_rect_k <= "01",
ctl_rect_first <= 0,
dout_a(i) <- ram_i(r1.x + x_pos),
dout_b(i) <- ram_i(r1.x + r1.w +
x_pos)
dp_w2_en = 1 ?
T
F
ctl_rect_last <=1,
ctl_cls_latch <=1,
cur_word<- rom_dout,
haar <- haar +1,
cls_j<- cls_j + 1
F
cls_j + 1 >= n_cls ?
T
ST_R2
ctl_rect_issue <= 1,
ctl_rect_k <= "10",
ctl_rect_first <= 0,
ctl_rect_last <= 1,
ctl_cls_latch <= 1,
cur_word <- rom_dout,
haar <- haar + 1,
cls_j <- cls_j + 1,
dout_a/b(i) <- ram_i(r2 corners)
cls_j +1 >= n_cls ?
T
F
u ST_DRAIN


iz ST_R1/ST_R2
ST_DRAIN
kontrolni signali se ne postavljaju
s_acc <- s_acc + term when
tag_d3.valid
stage_sum <- stage_sum +
alphwhen tag_d4.valid and .last
uvijek 5
ciklusa
dp_pipe_busy = 0?
T
F
ST_STAGE_EVAL
kontrolni signali se ne postavljaju
u ST_STAGE_INIT
dp_stage_rej = 1?
(stage_sum *5 <
thresh *2)
T
F
stage_i <-
stage_i + 1
F
stage_i = NUM_STAGES -1 ?
T
ST_ACCEPT
ctl_res_accept <= 1,
ctl_hit_push <= 1,
result_r <- 1
T
hit_ram(hit_wp(8:0))
<- x_pos,
hit_wp<-hit_wp+1
hit_wp(HIT_AW) = 0 ?
F
ST_REJECT
ctl_res_reject <= 1,
result_r <- 0 - resize(ctl_stage_idx,
32)


ST_WIN_END
kontrolni signali se ne postavljaju
mode_sweep = 1 and
(x_reg + x_step) <=x_last?
T
F
x_reg <- x_reg +
x_step,
stage_i <-0,
cls_j<-0,
haar<-0
u ST_PV_READ
ST_DONE
done <= 1,
reg_done <- 1 -> irq
u ST_IDLE

Slika 9. ASM dijagram (sa uvećanim isječcima)

Stanje	Funkcija
ST_IDLE	Čekanje na START. Samo je u ovom stanju BUSY neaktivan.
ST_WIN_START	Prihvatanje koordinate x prozora; inicijalizacija brojača faze i klasifikatora.
ST_PV_READ / ST_PV_W1 / ST_PV_CAP	Izdavanje i prihvatanje četiri čitanja ćoškova prozora iz integralnih i kvadriranih banki, za računanje varijanse.
ST_VAR_CALC / ST_VAR_MUL / ST_VAR_FIN / ST_VAR_WAIT	Računanje srednje vrijednosti, njenog kvadrata, varijanse i cjelobrojnog kvadratnog korena. Stanje ST_VAR_WAIT pokriva kašnjenje korijena (8 taktova za 16 referentnih iteracija).
ST_STAGE_INIT	Fetch prvog klasifikatora faze i brisanje akumulatora faze.
ST_STAGE_LATCH	Prihvatanje pakovane riječi klasifikatora i dekodovanje njenih polja.
ST_R0	Izdavanje pravougaonika 0 i računanje skaliranog praga t = tree_thresh × vnf. Ujedno unaprijed dohvata narednu riječ klasifikatora (rom_addr = haar + 1).
ST_R1	Izdavanje pravougaonika 1. Za 81% klasifikatora sa samo dva pravougaonika upravljanje se odavdje vraća pravo u ST_R0.
ST_R2	Izdavanje pravougaonika 2. Dostiže se samo za 19% klasifikatora koji imaju treći pravougaonik.
ST_DRAIN	Pražnjenje protočne obrade pravougaonika pre evaluacije sume faze.
ST_STAGE_EVAL	Poređenje sume faze sa pragom faze; grananje ka narednoj fazi ili ka odbacivanju.
ST_REJECT	Upis negativnog indeksa faze kao rezultata.
ST_ACCEPT	Bilježenje lica; u režimu B upis detekcije u FIFO.
ST_WIN_END	U režimu B uvećanje x za X_STEP i povratak u ST_WIN_START dok je x ≤ X_LAST; inače nastavak.
ST_DONE	Postavljanje signala DONE i prekida; povratak u ST_IDLE.

Tabela 11. Funkcionalni opis stanja ASM dijagrama

Prefetch i preklapanje

U red stanja ugrađena su dva preklapanja i ona su razlog niske prosječne cijene po klasifikatoru:

Adresa naredne riječi klasifikatora izdaje se već tokom evaluacije tekućeg klasifikatora, pa se između uzastopnih klasifikatora ne gubi nijedan takt.

Čitanja ćoškova narednog pravougaonika izdaju se dok se ćoškovi tekućeg još koriste, pa se ni između pravougaonika unutar klasifikatora ne gubi takt.

Jedini zastoj u ustaljenom režimu je kašnjenje kvadratnog korijena, koje se javlja jednom po prozoru, a ne po klasifikatoru, pa se amortizuje na 14,6 klasifikatora koliko prosječan prozor izračuna.

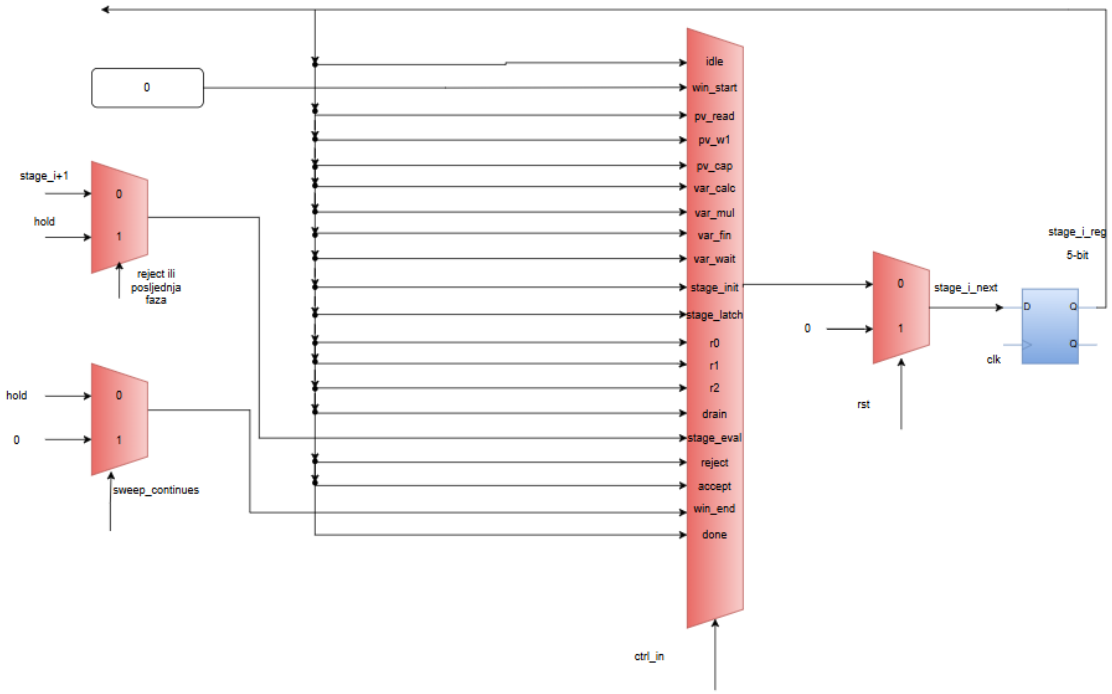
Izbor režima

Režimi A i B dijele cijeli red stanja sve do ST_WIN_END. U režimu A stanje ST_WIN_END bezuslovno prelazi u ST_DONE. U režimu B ponovo ulazi u ST_WIN_START sa uvećanim x sve

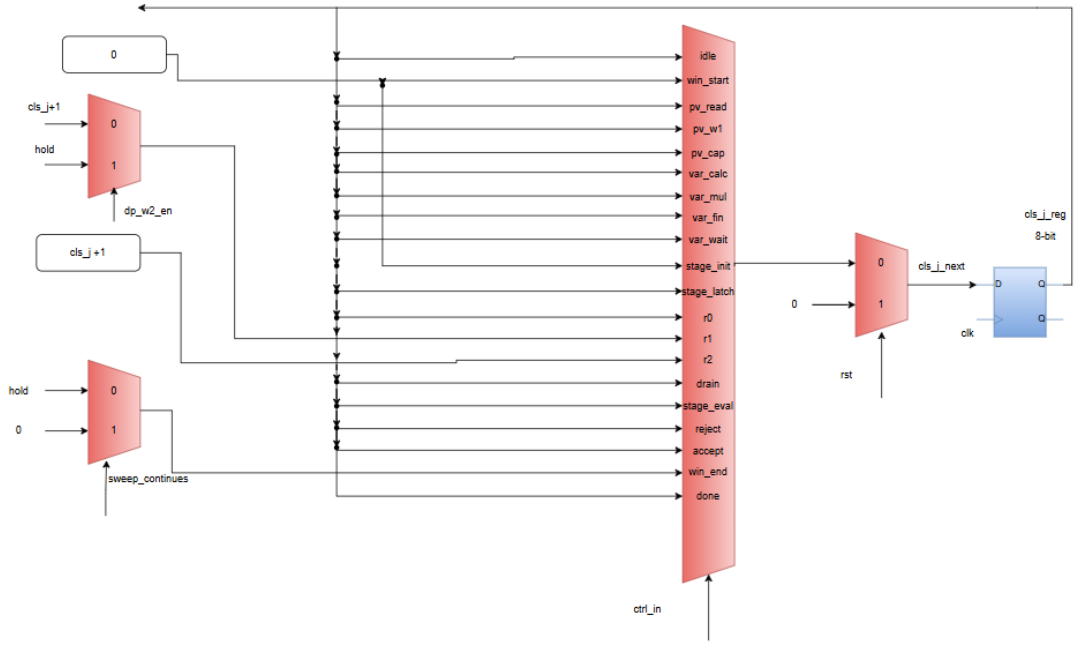
dok se pretraga ne iscrpi. Ne postoji zaseban datapath niti udvojena upravljačka logika za dva režima; razlika je jedan uslov grananja.

Moduli controlpatha

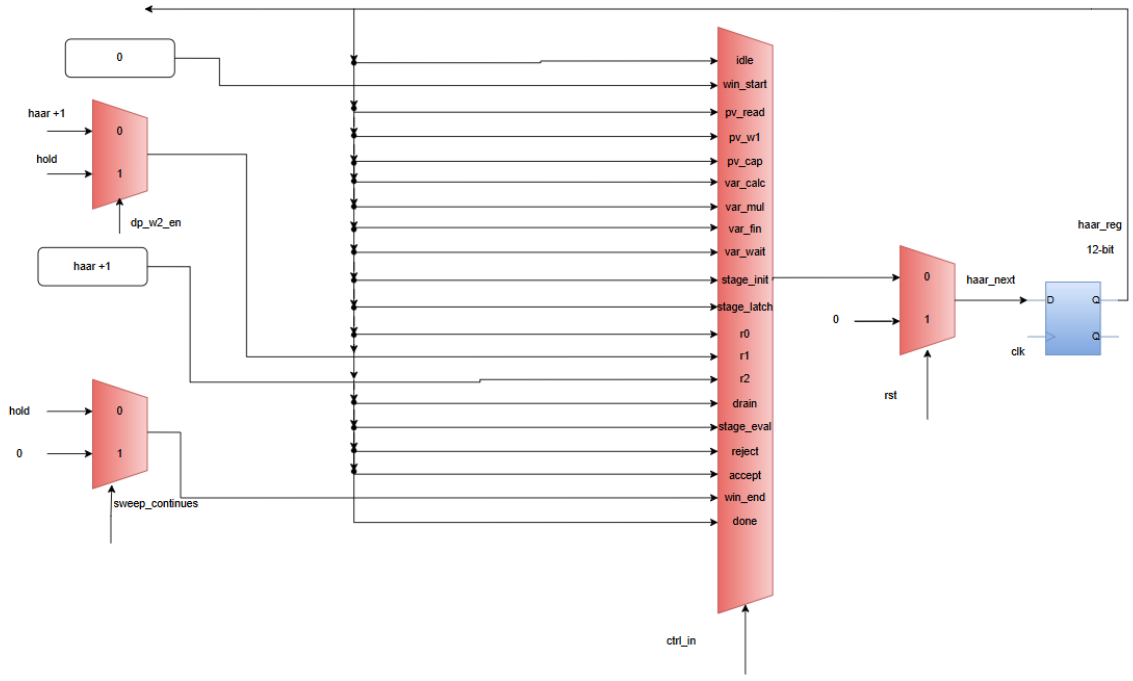
U nastavku su dati dijagrami modula na nivou komponenti koji čine controlpath.



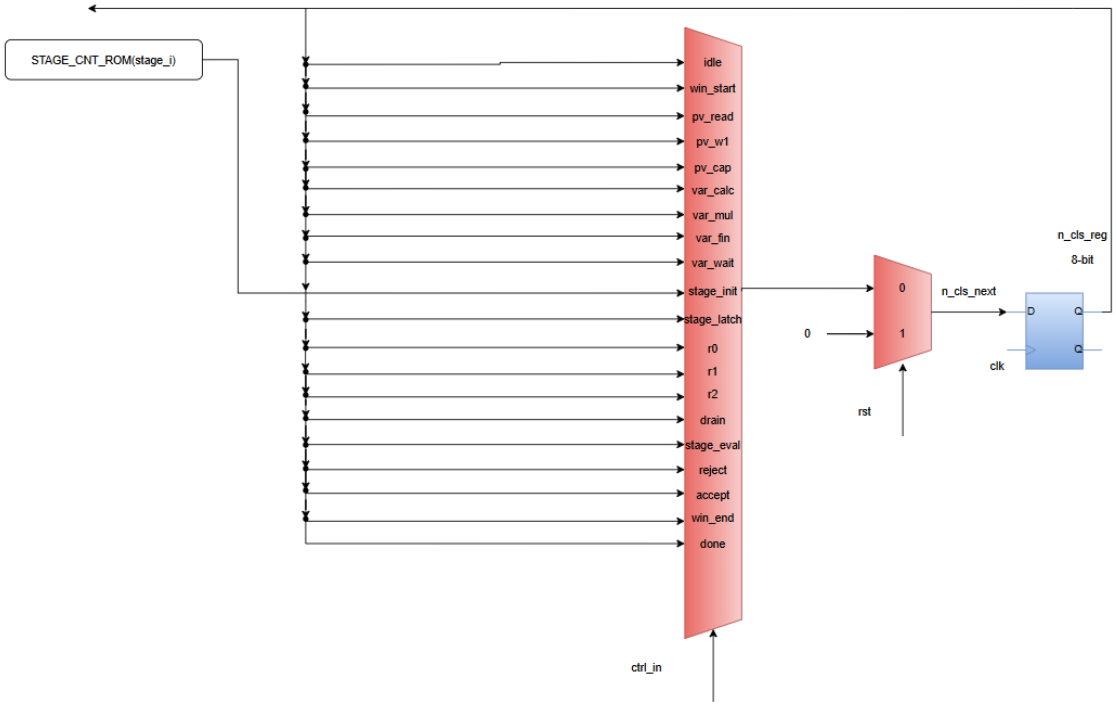
Slika 10. stage_i modul



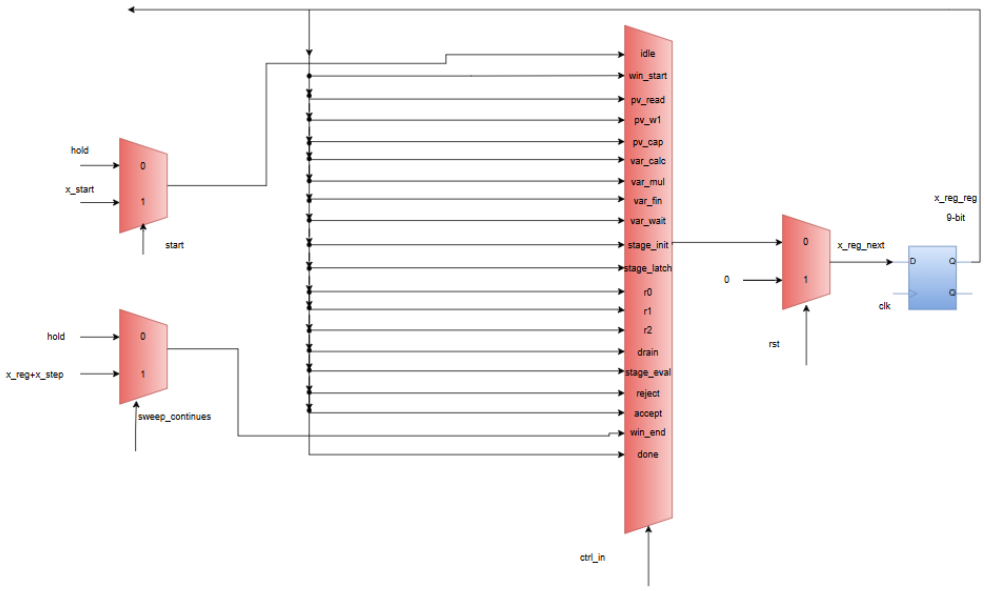
Slika 11. cls_j modul



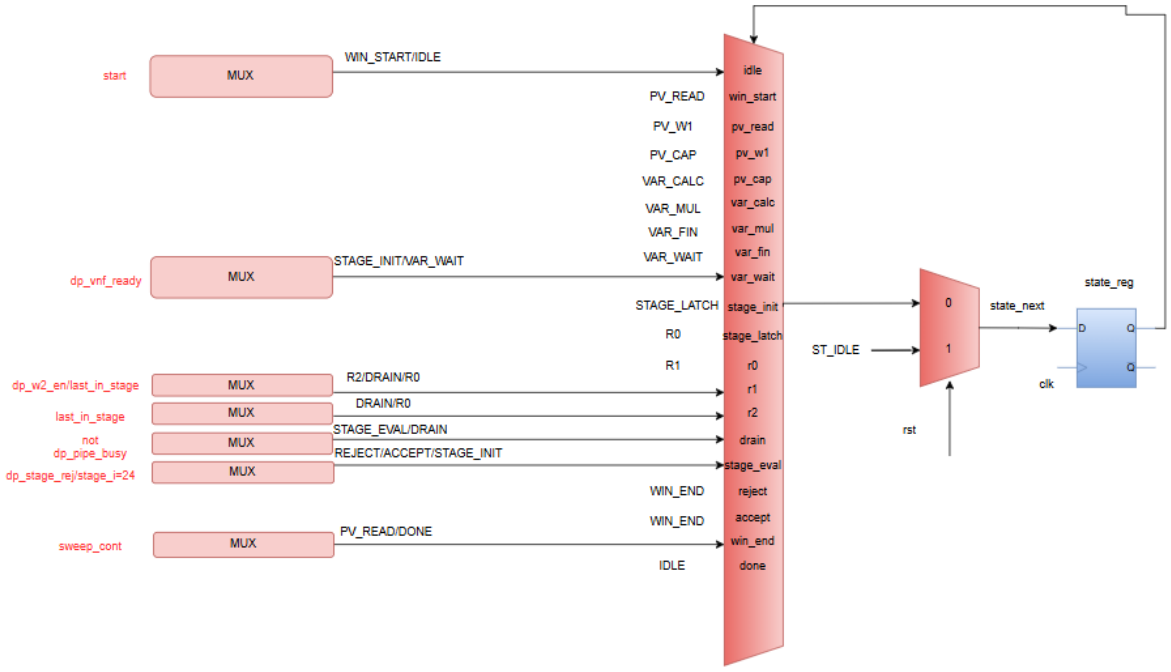
Slika 12. haar modul



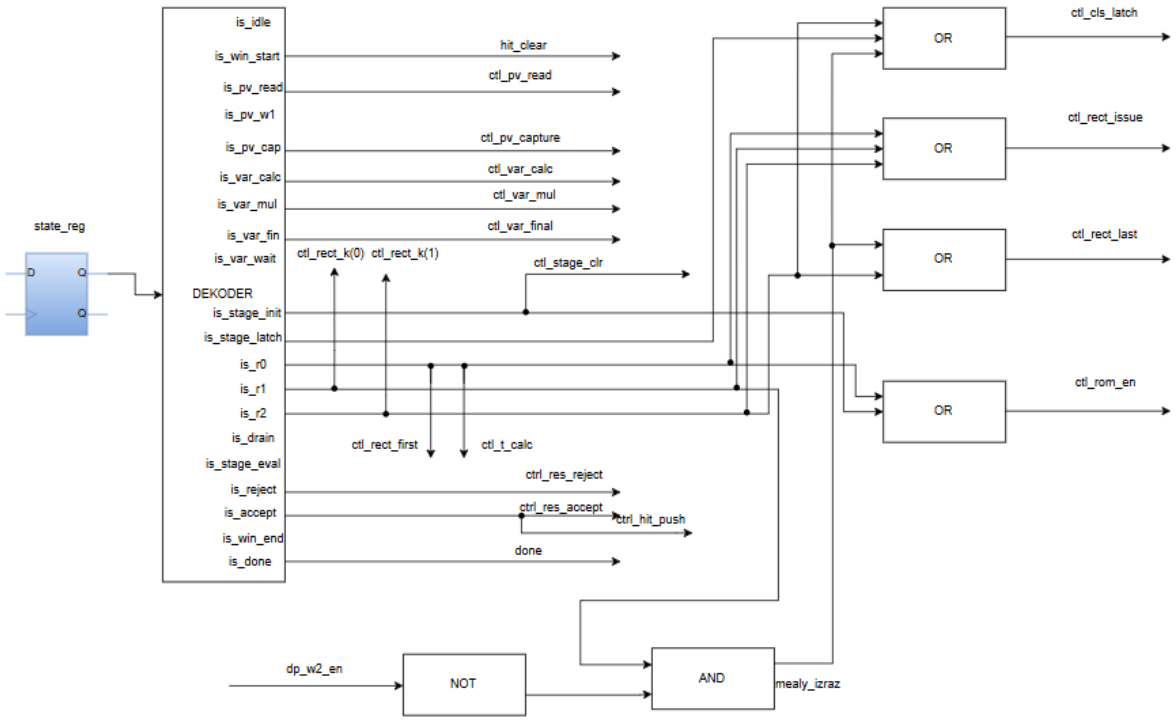
Slika 13. n_cls modul



Slika 14. x_reg modul



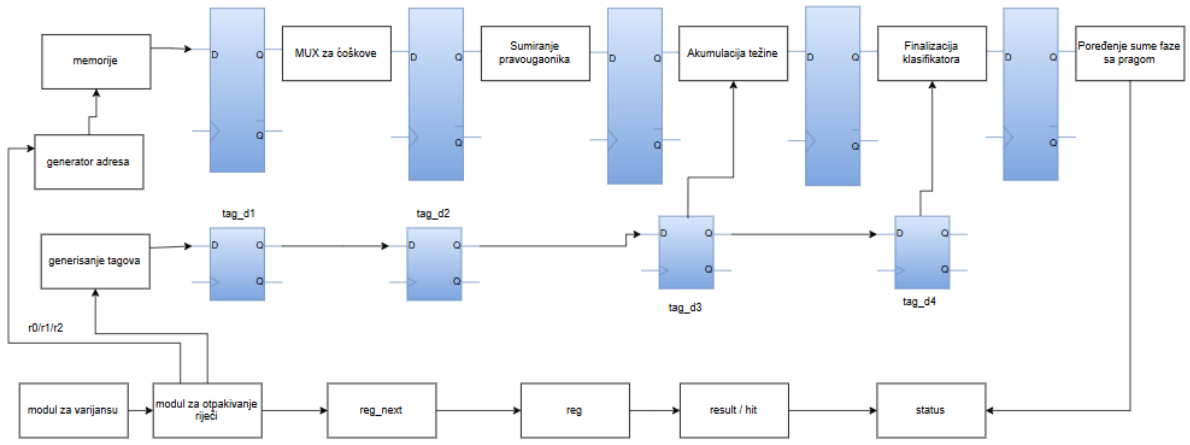
Slika 15. Logika za generisanje narednog stanja



Slika 16. Izlazni dekoder

Datapath

Datapath (vj_datapath.vhd, sa nizom banki i ROM-om klasifikatora kao podmodulima) čine memorije, jedinica varijanse, protočna obrada pravougaonika, skaliranje težina, akumulatori i bafer rezultata odnosno detekcija.



Slika 17. Unutrašnja struktura datapatha

Memorijske strukture datapatha

Struktura	Organizacija	Namjena
Banke integralnih redova	25 × 512 × 32 b, dva puna porta	Bafer kliznog prozora, adresiran rotacijom
Banke kvadrirane slike	2 × 512 × 32 b, dva puna porta	Samo logički redovi 0 i 23
ROM klasifikatora	2913 × 112 b, inicijalizovan iz datoteke	Jedna pakovana riječ po slabom klasifikatoru
Bafer detekcija	512 mjesta	Detekcije u režimu B; prekoračenje strukturno nemoguće

Tabela 12. Objašnjenje organizacije i namjene memorija unutar datapatha

Niz banki (`vj_bank_array.vhd`) sadrži i logiku rotacije i dekodovanje putanje upisa iz procesorskog sistema, tako da se rotacija dosljedno primjenjuje i na čitanja iz jezgra i na upise sa procesora, iz jedne jedine implementacije.

Jedinica varijanse

Jedinica varijanse iz četiri pročitana ćoška računa srednju vrijednost i srednju vrijednost kvadrata nad prozorom, formira varijansu i preko cjelobrojnog kvadratnog korijena dobija faktor normalizacije. Množenje kvadrata srednje vrednosti jedino je mjesto u dizajnu gde se koriste DSP48E1 blokovi - četiri komada, jednom po prozoru.

Širina akumulatora fiksirana je na 32 bita i takva mora ostati. VP se u ovom proračunu za pojedine ulaze oslanja na overflow 32-bitne neoznačene aritmetike, a proširenje akumulatora bi neprimjetno promijenilo rezultate baš za te ulaze.

Protočna obrada pravougaonika

Za svaki pravougaonik četiri vrijednosti ćoškova čitaju se iz odgovarajućih banki redova - dvije banke, po dva porta - i kombinuju u sumu pravougaonika. Obrada je uređena tako da se čitanja ćoškova pravougaonika n+1 izdaju dok se pravougaonik n kombinuje, pa uzastopni pravougaonici u ustaljenom režimu koštaju po jedan takt, umjesto da svaki uvede zastoj zbog kašnjenja čitanja.

Skaliranje težina i akumulatori

Suma pravougaonika skalira se kodom težine pomoću pomjeranja, negacije i pomjeraj-sabiranja, i akumulira u sumu klasifikatora. Suma klasifikatora poredi se sa skaliranim pragom stabla i odgovarajuća alpha konstanta dodaje se sumi faze. Suma faze široka je 24 bita, čime je najgori slučaj od 211 klasifikatora * 7680 što je mnogo ispod 2^{21}.

Putanja rezultata i detekcija

U režimu A ishod se upisuje u registar RESULT: 1 za lice ili negativan indeks faze koja je odbacila prozor. U režimu B prihvaćeni prozori upisuju se u FIFO detekcija zajedno sa

koordinatom x, a softver ih poslije završetka pretrage čita upisom u HIT_ADDR i čitanjem HIT_DATA.

Registarske operacije po stanjima

Datapath nije proizvoljan raspored blokova: on slijedi neposredno iz spiska registarskih operacija koje algoritam zahtijeva, grupisanih najprije po stanju koje ih izdaje, a zatim po registru u koji upisuju.

Operacije po stanjima

Stanje	Registarske operacije
ST_IDLE	ako start: x_reg ← x_start
ST_WIN_START	hit_wp ← 0; stage_i ← 0; cls_j ← 0; haar ← 0
ST_PV_READ	(bez upisa u registar — adrese banki su kombinacione: row_top ← 0, row_bot ← 23, col0 ← x_pos, col1 ← x_pos+23)
ST_PV_W1	(pokriva dvotaktno kašnjenje čitanja banki)
ST_PV_CAP	p0..p3 ← corner_a..corner_d; q0..q3 ← sq_a..sq_d
ST_VAR_CALC	mean_r ← (p0+p3) − (p1+p2) acc_r ← (q0+q3) − (q1+q2)
ST_VAR_MUL	mean_sq ← mean_r × mean_r acc576 ← (acc_r ≪ 9) + (acc_r ≪ 6)
ST_VAR_FIN	v := acc576 − mean_sq; ako v = 0: vnf ← 1 inače: sqrt_start ← 1, sqrt_din ← v
ST_VAR_WAIT	na sqrt_done: vnf ← sqrt_dout; vnf_ready ← 1
ST_STAGE_INIT	n_cls ← STAGE_CNT_ROM(stage_i); cls_j ← 0; stage_sum ← 0
ST_STAGE_LATCH	cur_word ← rom_dout
ST_R0	t_reg ← cur_tt × vnf; tag_issue ← {k=00, first=1, valid=1}; rom_addr ← haar + 1
ST_R1	tag_issue ← {k=01, valid=1, last=¬w2_en} ako ¬w2_en: cur_word ← rom_dout; haar ← haar+1; cls_j ← cls_j+1
ST_R2	tag_issue ← {k=10, last=1, valid=1}; cur_word ← rom_dout; haar ← haar+1; cls_j ← cls_j+1
(svaki takt)	tag_d1 ← tag_issue; tag_d2 ← tag_d1; tag_d3 ← tag_d2; tag_d4 ← tag_d3 rs_r ← (corner_a+corner_d) − (corner_b+corner_c) s_acc ← (tag_d3.first ? 0 : s_acc) + težina(tag_d3.kcode, rs_r) ako tag_d4.last: stage_sum ← stage_sum + (s_acc≪12 ≥ tag_d4.t ? tag_d4.a2 : tag_d4.a1)
ST_DRAIN	(čeka na dp_pipe_busy)
ST_STAGE_EVAL	ako nije odbačeno i nije posljednja faza: stage_i ← stage_i + 1
ST_REJECT	result_r ← 0 − stage_i
ST_ACCEPT	result_r ← 1; hit_ram(hit_wp) ← x_pos; hit_wp ← hit_wp + 1
ST_WIN_END	ako pretraga: x_reg ← x_reg + x_step; stage_i ← 0; cls_j ← 0; haar ← 0
ST_DONE	(impuls done; reg_done se postavlja u AXI omotaču)

Tabela 13. Dijagram RT operacija po stanju

Operacije po odredišnom registru

Pregrupisavanje istog spiska po odredištu daje inventar registara, a svaki red je jedan blok datapatha: sam registar i kombinaciona mreža koja proizvodi njegovu narednu vrijednost.

Registar	Širina	Izrazi za narednu vrijednost	Upisuje se u
x_reg	COL_W = 9	x_start; x_reg + x_step	ST_IDLE, ST_WIN_END
stage_i	5	0; stage_i + 1	ST_WIN_START, ST_STAGE_EVAL, ST_WIN_END
cls_j	STAGE_CNT_BW = 8	0; cls_j + 1	ST_STAGE_INIT, ST_R1, ST_R2
n_cls	8	STAGE_CNT_ROM(stage_i)	ST_STAGE_INIT
haar	CLS_AW = 12	0; haar + 1	ST_WIN_START, ST_R1, ST_R2, ST_WIN_END
p0..p3	4 × ACC_W = 32	corner_a..corner_d	ST_PV_CAP
q0..q3	4 × 32	sq_a..sq_d	ST_PV_CAP

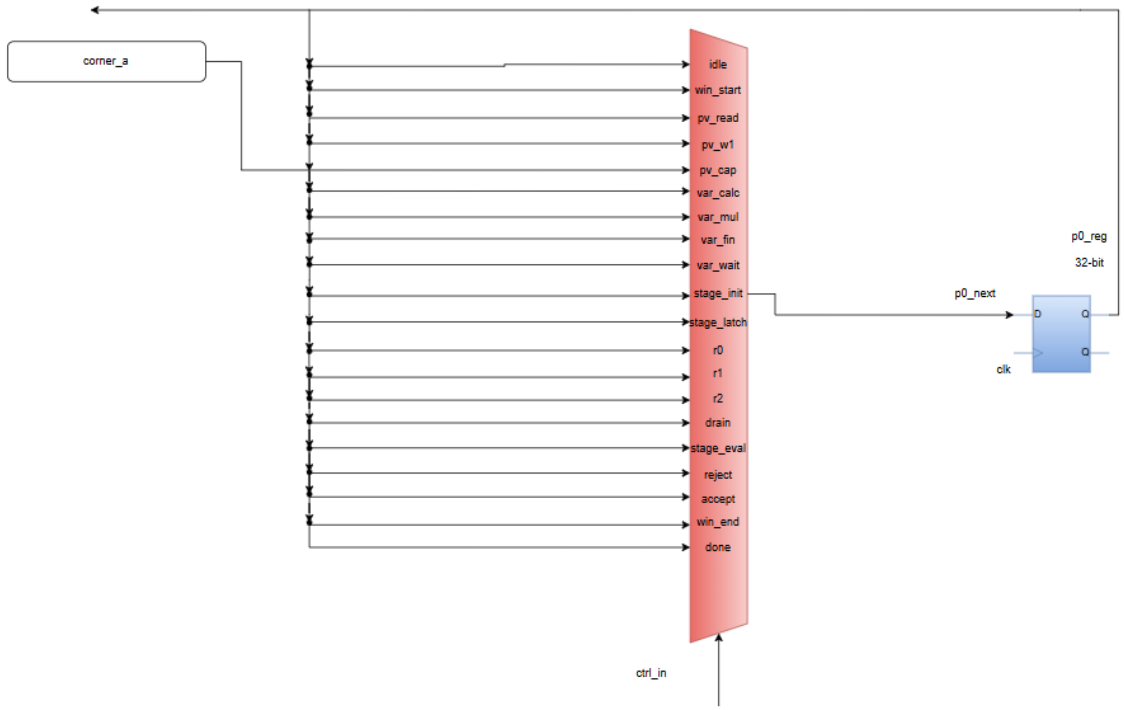
mean_r	32	(p0+p3) − (p1+p2)	ST_VAR_CALC
acc_r	32	(q0+q3) − (q1+q2)	ST_VAR_CALC
mean_sq	32	mean_r × mean_r (DSP48E1, overflow)	ST_VAR_MUL
acc576	32	(acc_r≪9) + (acc_r≪6) (overflow)	ST_VAR_MUL
vnf	VNF_W = 16	1; sqrt_dout	ST_VAR_FIN, ST_VAR_WAIT
cur_word	CLS_DW = 112	rom_dout	ST_STAGE_LATCH, ST_R1, ST_R2
t_reg	29	cur_tt × vnf (DSP48E1)	ST_R0
tag_d1..tag_d4	4 × tag_t	prethodni stepen oznake	svaki takt
rs_r	32	(corner_a+corner_d) − (corner_b+corner_c)	svaki takt
s_acc	32	negacija / pomjeranje / pomjeraj-sabiranje nad rs_r, akumulirano	svaki takt
stage_sum	STAGE_SUM_W = 24	0; stage_sum + alpha1|alpha2	ST_STAGE_INIT, stepen E protočne obrade
result_r	32	1; 0 − stage_i	ST_ACCEPT, ST_REJECT
hit_wp	HIT_AW+1 = 10	0; hit_wp + 1 (saturation)	ST_WIN_START, ST_ACCEPT
hit_ram	512 × 9	x_pos na adresi hit_wp	ST_ACCEPT

Tabela 14. Dijagram RT operacija po target registru

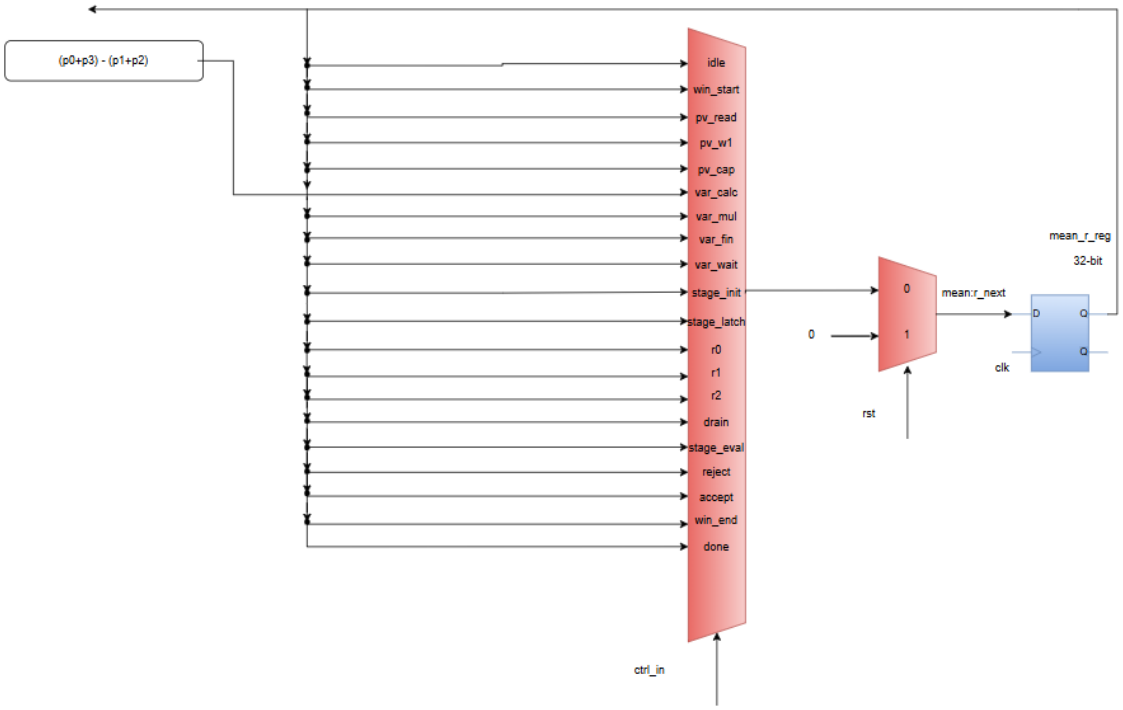
Dva reda zaslužuju pažnju. Registri mean_sq i acc576 označeni su kao „overflow": široki su tačno 32 bita jer ih referenca računa u 32-bitnom cijelom broju i zaista se prelivaju (ip.cpp, linije 249-253). A registri tag_d1..tag_d4 jedini se ne upisuju iz imenovanog stanja - pomjeraju se bezuslovno svakog takta, što i omogućava da aritmetička protočna obrada radi bez sopstvenog automata.

Moduli datapatha

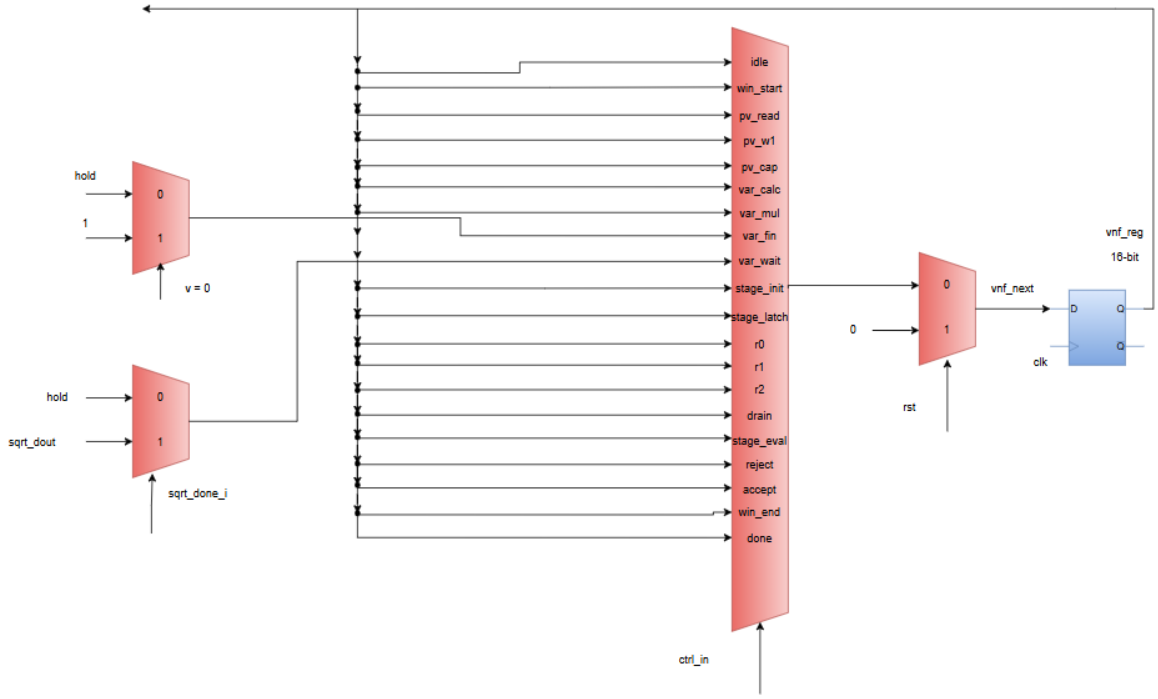
U nastavku su dati moduli koji čine datapath sistema:



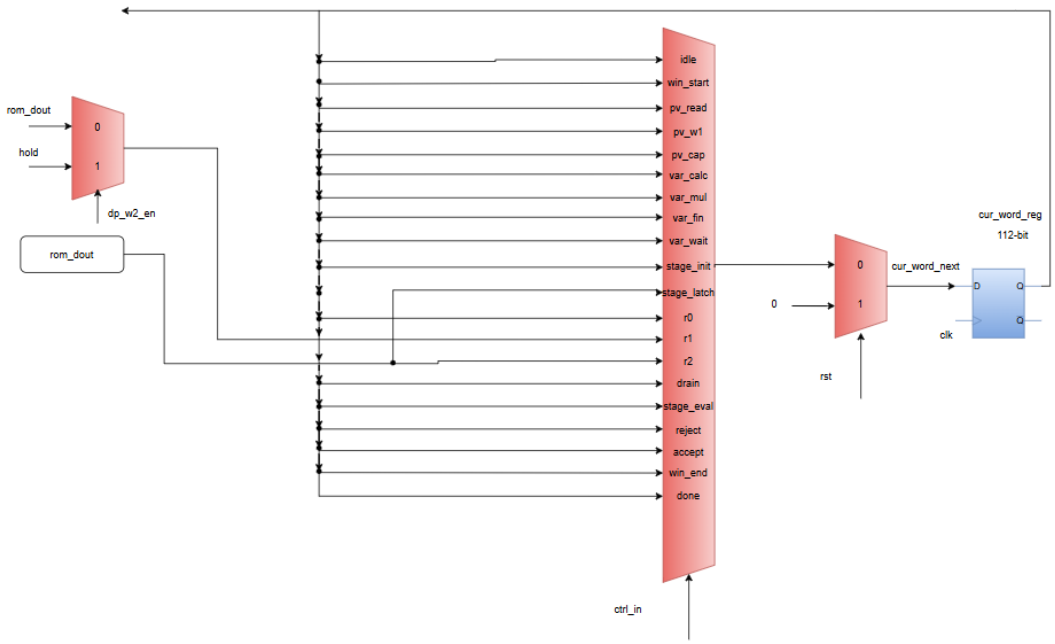
Slika 18. p0-p7 moduli (8 identično konstruisanih modula)



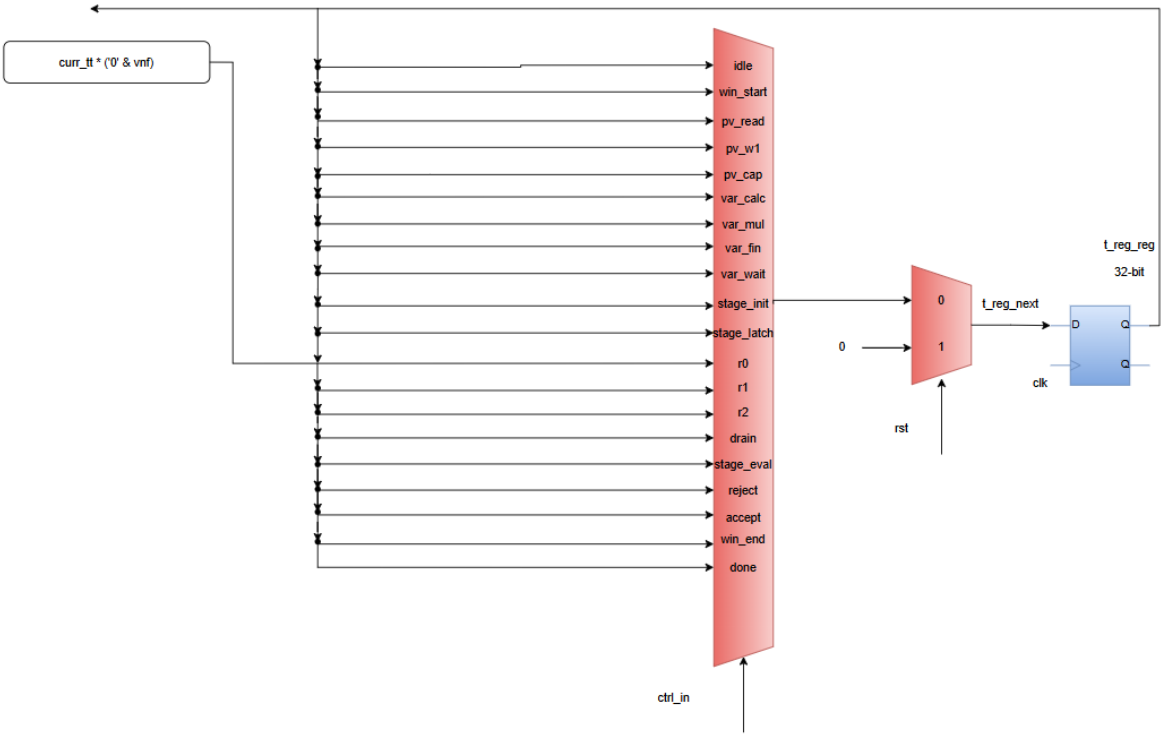
Slika 19. mean_r modul



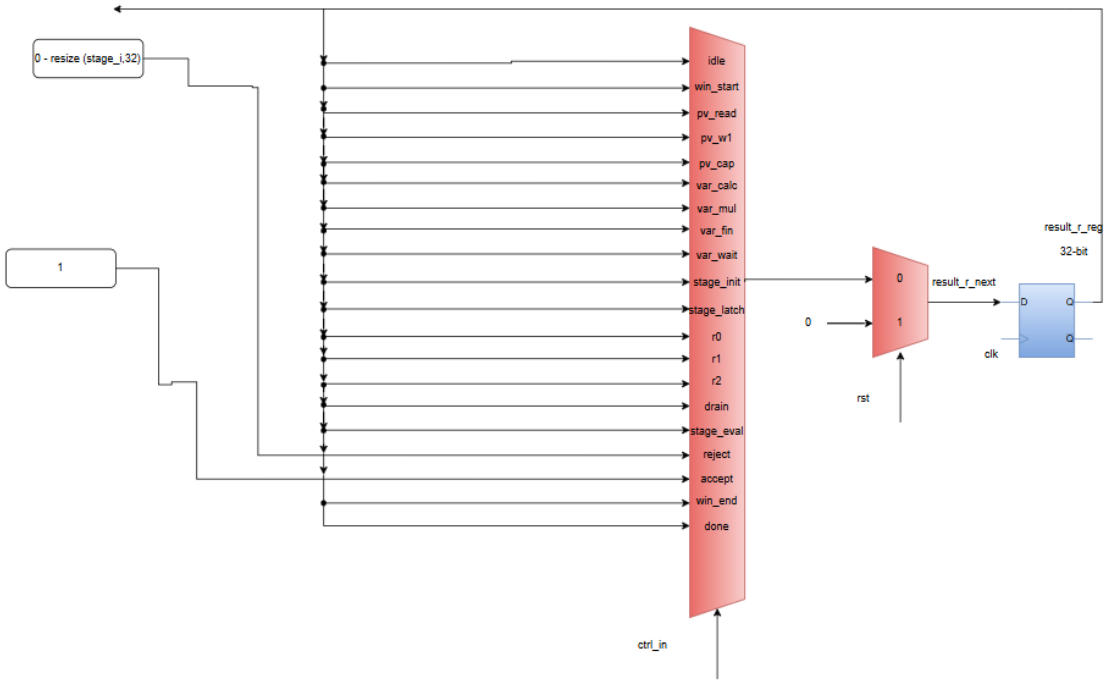
Slika 20. vnf modul



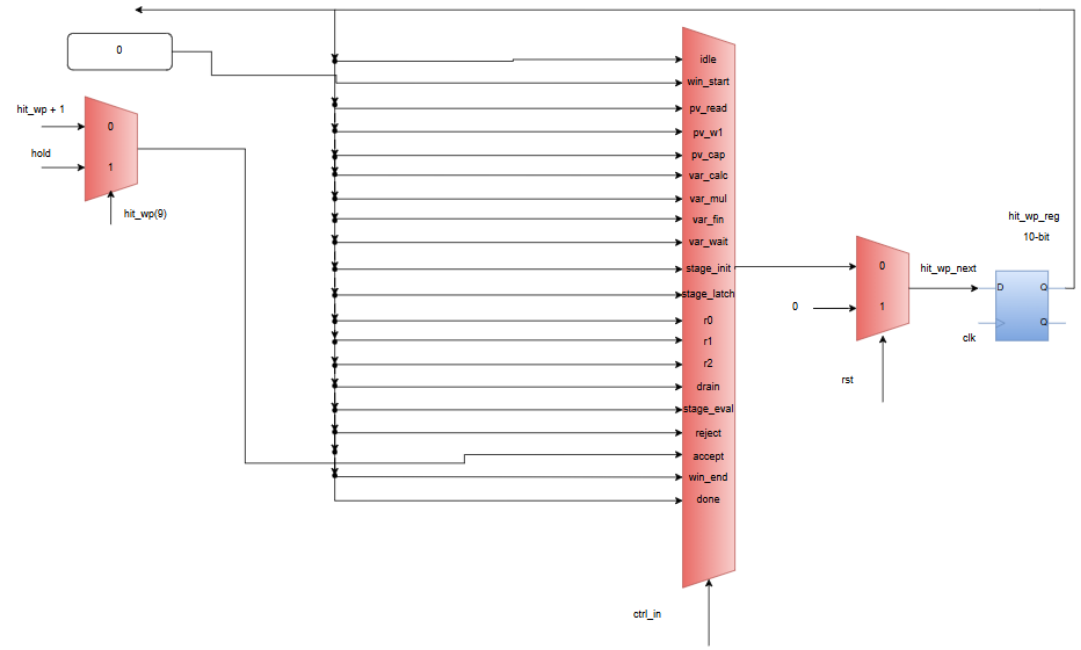
Slika 21. curr_word modul



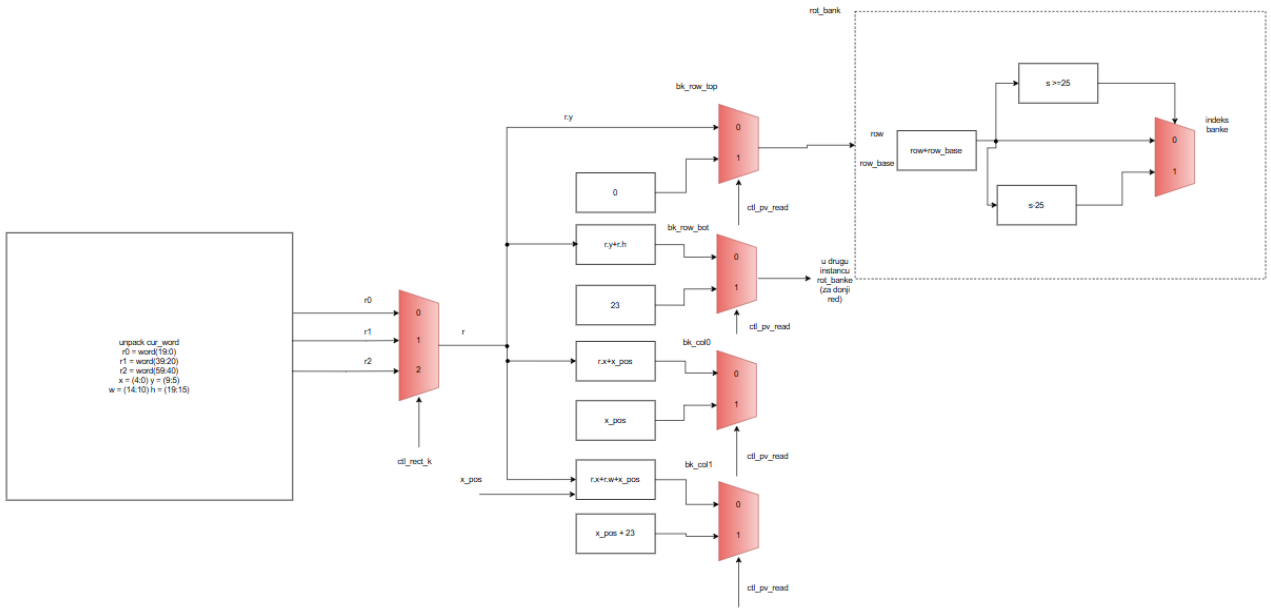
Slika 22. t_reg modul



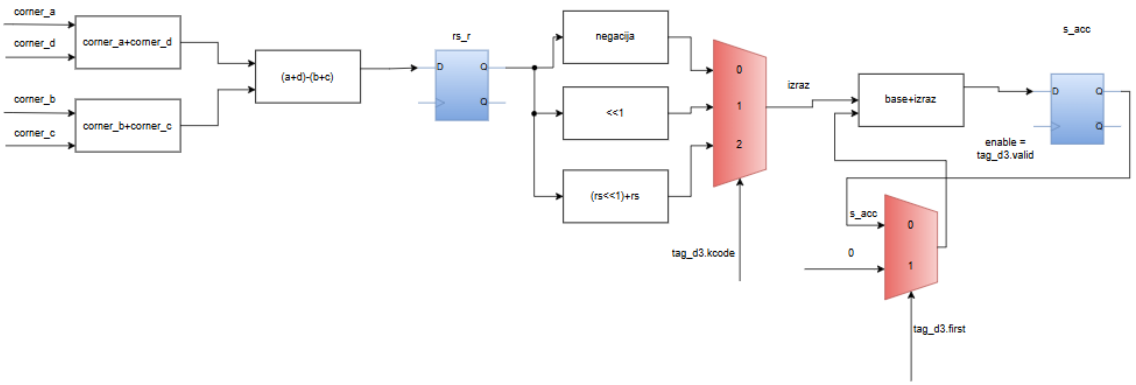
Slika 23. result_r modul



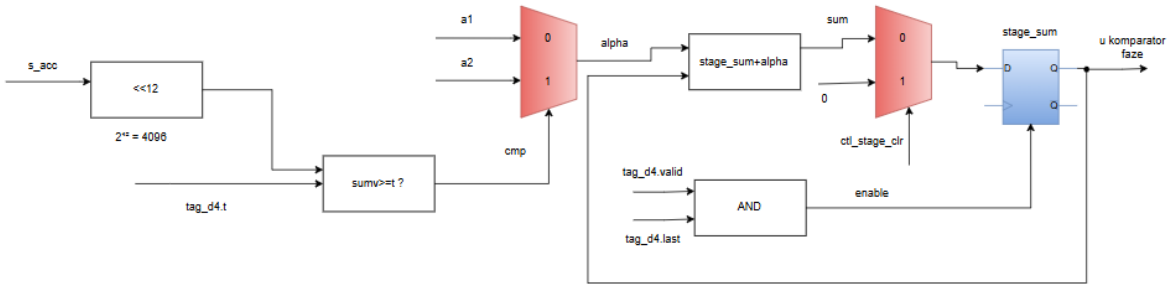
Slika 24. hit_wp modul



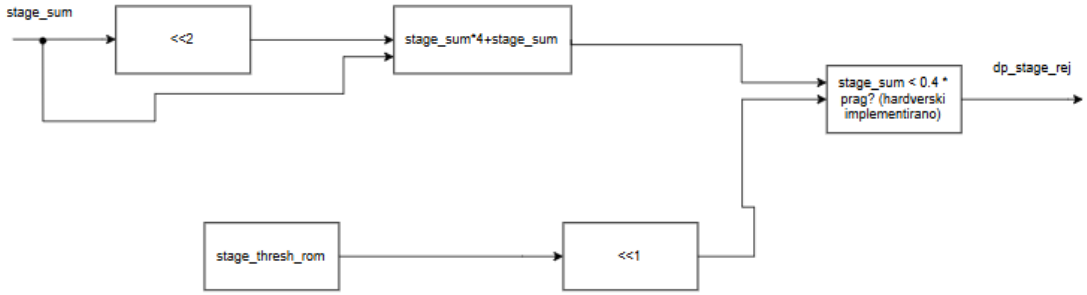
Slika 25. Mreža za generisanje adresa



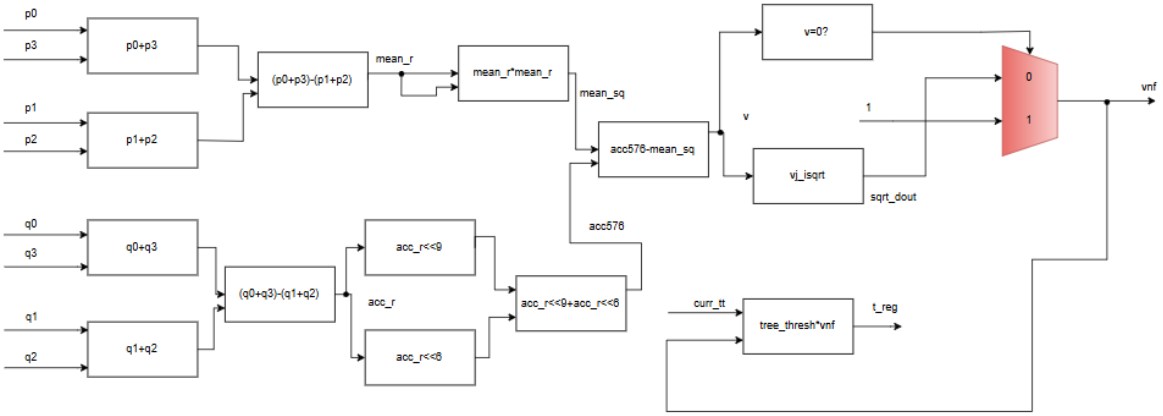
Slika 26. Mreža za sumiranje pravougaonika,skaliranje težina i akumulaciju sume faze



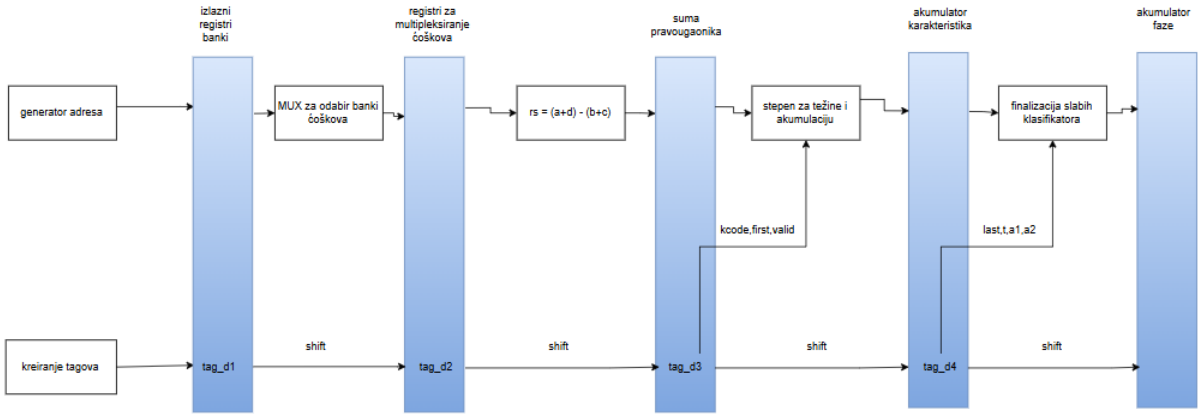
Slika 27. Mreža za finalizaciju klasifikatora



Slika 28. Mreža za poređenje sa pragom faze



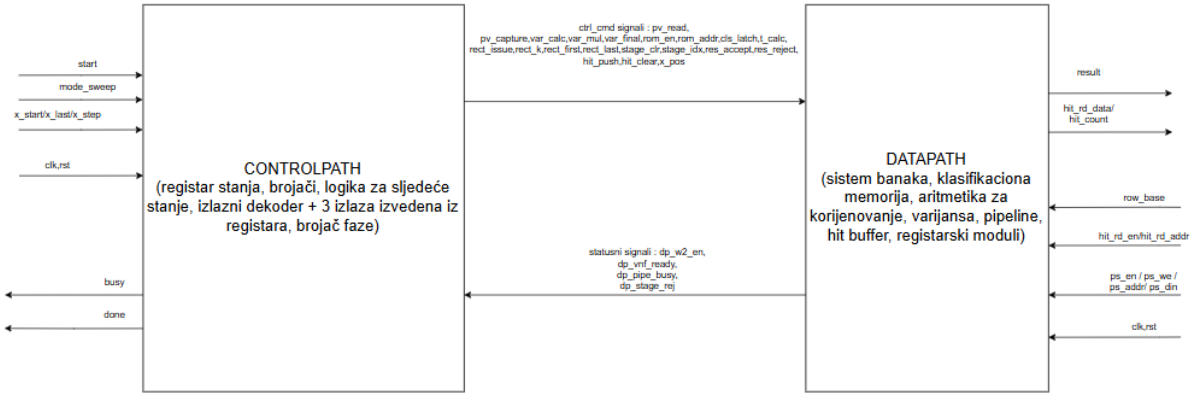
Slika 29. Mreža za normalizaciju varijanse



Slika 30. Protočna obrada pravougaonika u datapathu

Pakovanje IP jezgra

Controlpath i datapath sa svojim međusignalima i spoljašnjim interfejsima čine IP jezgro. Na narednoj slici dat je unutrašnji pogled na IP jezgro koji opisuje međuodnose, a koji će biti važan za pakovanje.



Slika 31. Unutrašnji izgled IP jezgra i interfejsing datapatha i signala

Controlpath u datapath prenosi komandne i kontrolne signale, dok datapath controlpathu vraća statusne signale. Statusne signale je potrebno dodatno objasniti i oni su sljedeći:

dp_w2_en – indikator da li trenutni klasifikator ima treći pravougaonik.
Bit 61 registra **cur_word**, žica bez ikakve dodatne logike. Ispituje se u stanju **ST_R1** i to je jedini ulaz na koji je izlazni dekoder osjetljiv: on uslovljava signale **ctl_rect_last** i **ctl_cls_latch,** čime **ST_R1** postaje jedino Milijevo stanje u automatu. Kada je na '1', automat prelazi u **ST_R2**, a brojači klasifikatora se ne uvećavaju; kada je na '0', klasifikator ima dva pravougaonika i nakon njih se završava.

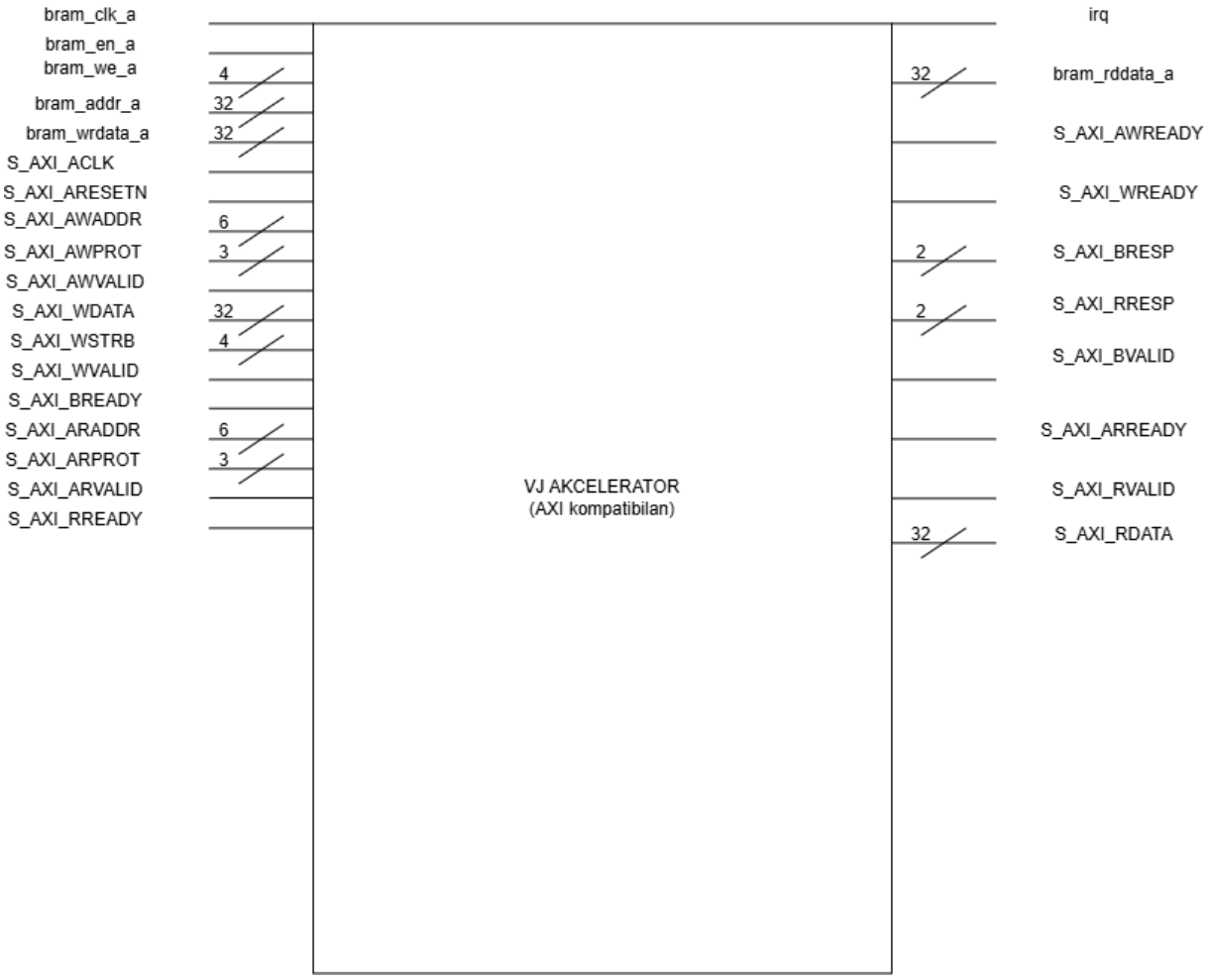
dp_vnf_ready – indikator za važeći faktor normalizacije varijanse.
Registrovan (**vnf_ready_i**). Postavlja se ili u stanju **ST_VAR_FIN**, kada je varijansa jednaka nuli pa se kvadratni korijen preskače, ili jedan takt nakon signala **sqrt_done_i**, kada iterativna jedinica završi rad. Ispituje se u stanju **ST_VAR_WAIT**, koje u prvom slučaju traje 1 takt, a u drugom 11. Automat ga očitava jedan takt nakon upisa u **vnf,** pa je time zagarantovano da je **vnf** stabilan prije nego što ga **ctl_t_calc** iskoristi u stanju **ST_R0**.

dp_pipe_busy – indikator da u protočnom sistemu za pravougaonike još uvijek ima podataka.
Logičko ILI sa pet ulaza: **tag_issue.valid** OR **tag_d1.valid** OR … OR **tag_d4.valid**. Ispituje se u stanju **ST_DRAIN**. Pošto je **ctl_rect_issue** u tom stanju na '0', prvi član odmah opada, pa pražnjenje deterministički traje 5 taktova, koliko iznosi i dubina protočne obrade. Time se

dobija tačno jedan takt rezerve između posljednjeg ažuriranja registra **stage_sum** i trenutka kada ga stanje **ST_STAGE_EVAL** čita.

dp_stage_rej – indikator da akumulirani zbir faze ne zadovoljava prag te faze.
Kombinaciono se dobija iz registra **stage_sum**:
(**stage_sum**<<2) + **stage_sum < STAGE_THRESH_ROM**(**ctl_stage_idx**) << 1,
što je cjelobrojni oblik uslova **stage_sum** < 0.4 * prag. Ispituje se u stanju **ST_STAGE_EVAL**, i to prije provjere posljednje faze, tako da odbacivanje na posljednjoj fazi ispravno vodi u **ST_REJECT**, a ne u **ST_ACCEPT**. Pošto se dobija čisto kombinaciono iz registra, stanju **ST_STAGE_EVAL** je dovoljan samo jedan takt.

Nakon pakovanja u AXI-kompatibilan oblik, jezgro ima izgled kao na slici u nastavku:



Slika 32. Spoljašnji interfejs IP jezgra nakon pakovanja

Promjene koje treba napraviti nad automatski generisanim Vivado AXI4-Lite slaveom jesu sljedeće:
slv_reg0…slv_regN uklonjeni. Generički niz registara iz templejta zamijenjen je imenovanim signalima odgovarajućih tipova (na primjer:
reg_x_start : unsigned(COL_W-1 downto 0)
i tako dalje), tako da širine dolaze iz **vj_pkg.vhd**, umjesto da budu dopunjene na 32 bita.

Dodat BRAM port A. **bram_clk_a, bram_en_a, bram_we_a, bram_addr_a, bram_wrdata_a i bram_rddata_a** nisu dio originalnog templejta. Dekodovanje
ps_adr <= bram_addr_a(PS_AW+1 downto 2)
pretvara bajt adresu u banku i kolonu.

Dodat irq izlaz, koji je direktno povezan sa **reg_done**, tako da je prekid na visokom nivou sve dok softver ne obriše **DONE**.

START i RESET su jednociklusni impulsi. Na početku **reg_write** se bezuslovno brišu, a postavljaju se samo prilikom upisa. Za razliku od toga, registri iz templejta zadržavaju svoju vrijednost.

DONE koristi princip „write-1-to-clear“ (brisanje upisom jedinice). Softversko brisanje se vrši u **CASE** grani, dok se hardversko postavljanje vrši nakon nje, tako da kasnija dodjela ima prednost ako se završetak operacije dogodi u istom ciklusu u kojem softver pokušava da obriše **DONE**. Na taj način završetak operacije nikada ne može biti izgubljen.

reg_x_step se resetuje na 1, a ne na 0. Nulti korak bi doveo do toga da uslov za provjeru skeniranja (x_reg + x_step) <= x_last zauvijek ostane ispunjen.

Multiplekser za čitanje dekodira i izlaze jezgra, pored registara. RESULT, HIT_COUNT, HIT_DATA i STATUS čitaju se direktno iz jezgra, a ne iz sačuvanih registara.

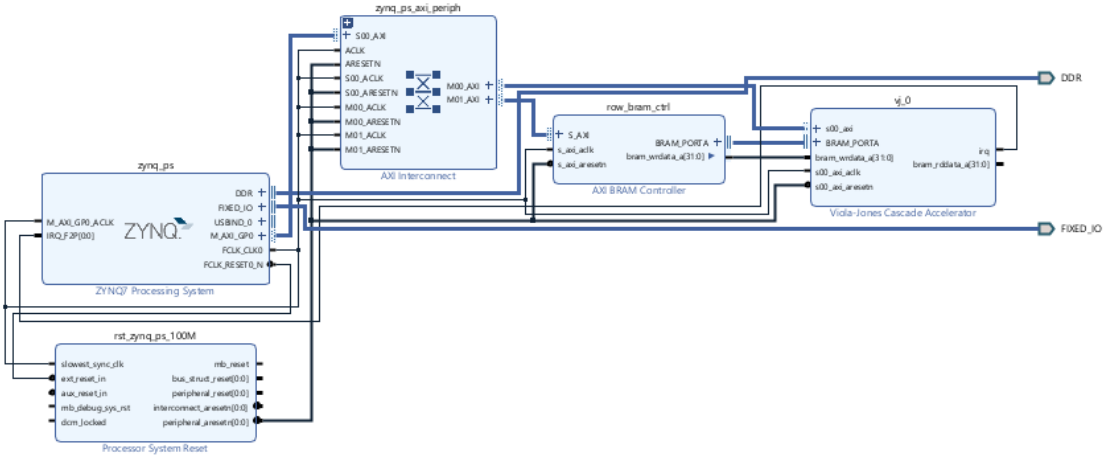
Jezgro je instancirano unutar S00_AXI fajla, umjesto u top-level wrapperu;
vj_cascade_v1_0.vhd je samo pass-through sloj.

core_rst <= (not S_AXI_ARESETN) or soft_reset - soft reset aktivira sinhroni reset jezgra, ali namjerno ne briše AXI registarski fajl, tako da konfiguracija ostaje sačuvana i nakon soft reseta.

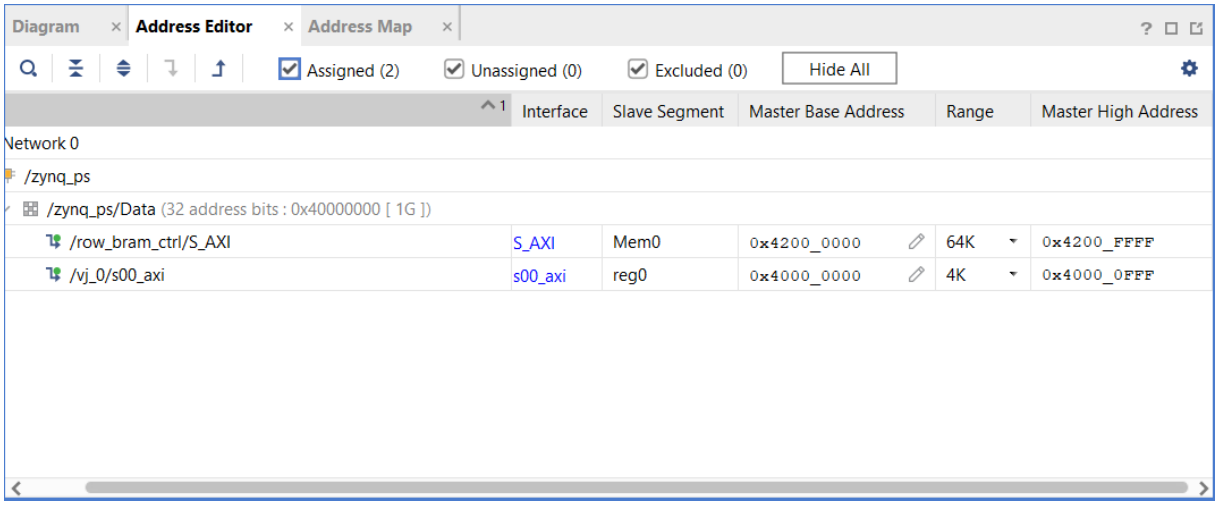
Integracija sistema

Blok dizajn i mapa adresa

Blok dizajn sadrži Zynq7 procesorski sistem, upakovano Viola-Jones IP jezgro, AXI BRAM kontroler za putanju upisa redova, AXI Interconnect i blok za reset procesorskog sistema. Prekid iz jezgra vodi se na ulaz IRQ_F2P procesorskog sistema.



Slika 33. Blok dizajn i integracija sistema



Slika 34. Izgled Address Editor prozora nakon ispravne konfiguracije

Periferija	Bazna adresa	Krajnja adresa	Veličina
vj_0 (S00_AXI)	0x40000000	0x40000FFF	4 KB (fiksirano; dekoduje se 6 adresnih bita)
row_bram_ctrl (S_AXI)	0x42000000	0x4200FFFF	64 KB

Tabela 15. Tabelarni prikaz adresne mape

Adresni prozor od 64 KB na BRAM kontroleru je neophodan i nije podrazumijevana vrijednost. Uz podrazumijevanih 8 KB signal `bram_addr_a` širok je samo 13 bita, pa se select bitovi za banke 13..9 odsijecaju i dostupne su samo banke 0–3. Simptom bi bio uvjerljive ali netačne detekcije, a ne poruka o grešci.

Taktovi i reset

U cijeloj programabilnoj logici koristi se jedan taktni domen. Signal FCLK_CLK0 iz procesorskog sistema baferuje se na globalnu taktnu mrežu i pobuđuje AXI Interconnect, BRAM kontroler i akcelerator, uključujući i takt BRAM porta A. Blok za reset procesorskog sistema generiše sinhrone resete periferija.

Parametar	**Vrijednost**	**Izvor**
Ulazna učestanost PS_CLK	33,333333 MHz	Kristal na ploči
ARM PLL	×40 → 1333,33 MHz	ps7_init.tcl, ARM_PLL_CTRL FDIV = 40
Procesor (Cortex-A9)	666,67 MHz	PCW_APU_PERIPHERAL_FREQMHZ
DDR PLL	×32 → 1066,67 MHz	DDR_PLL_CTRL FDIV = 32
IO PLL	×48 → 1600 MHz	IO_PLL_CTRL FDIV = 48
Referentni takt za UART	1600 / 16 = 100 MHz	UART_CLK_CTRL, IO PLL, djelilac 16
FCLK_CLK0 (logika)	1600 / (4×4) = 100 MHz	FPGA0_CLK_CTRL, oba djelioca 4
Ograničenje periode takta	10,000 ns	clock_util.rpt, mreža FCLK_CLK0, BUFGCTRL_X0Y15

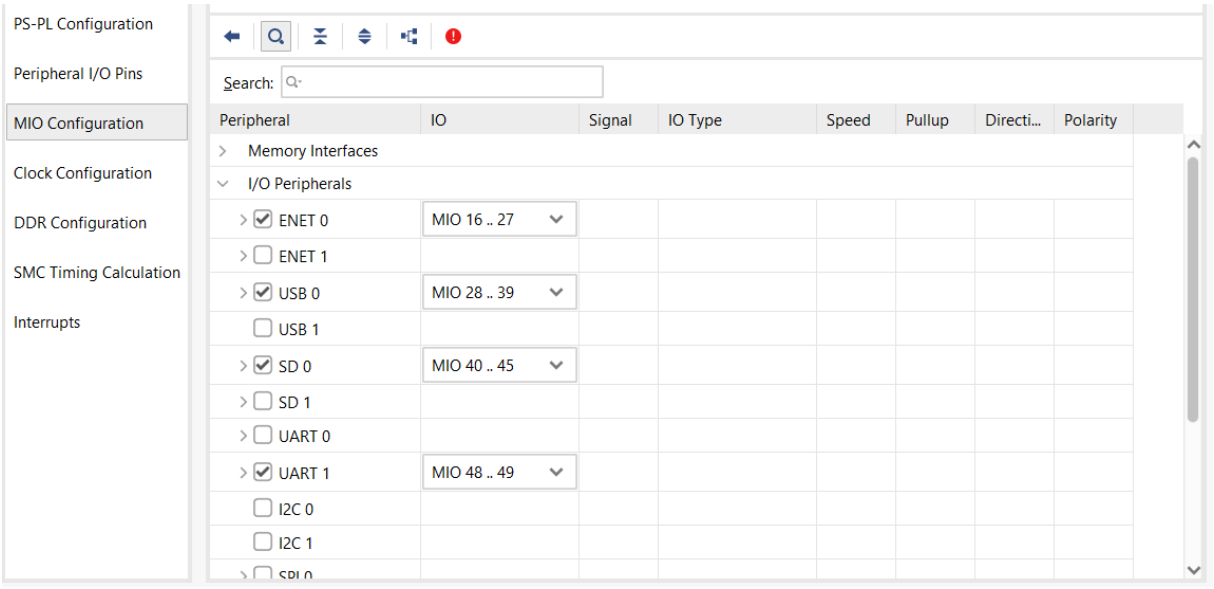
Tabela 16. Vrijednosti taktnih parametara

Sve vrijednosti u ovoj tabeli potiču iz inicijalizacionog koda i Vivado sintesisovanog izvještaja o taktnim mrežama.

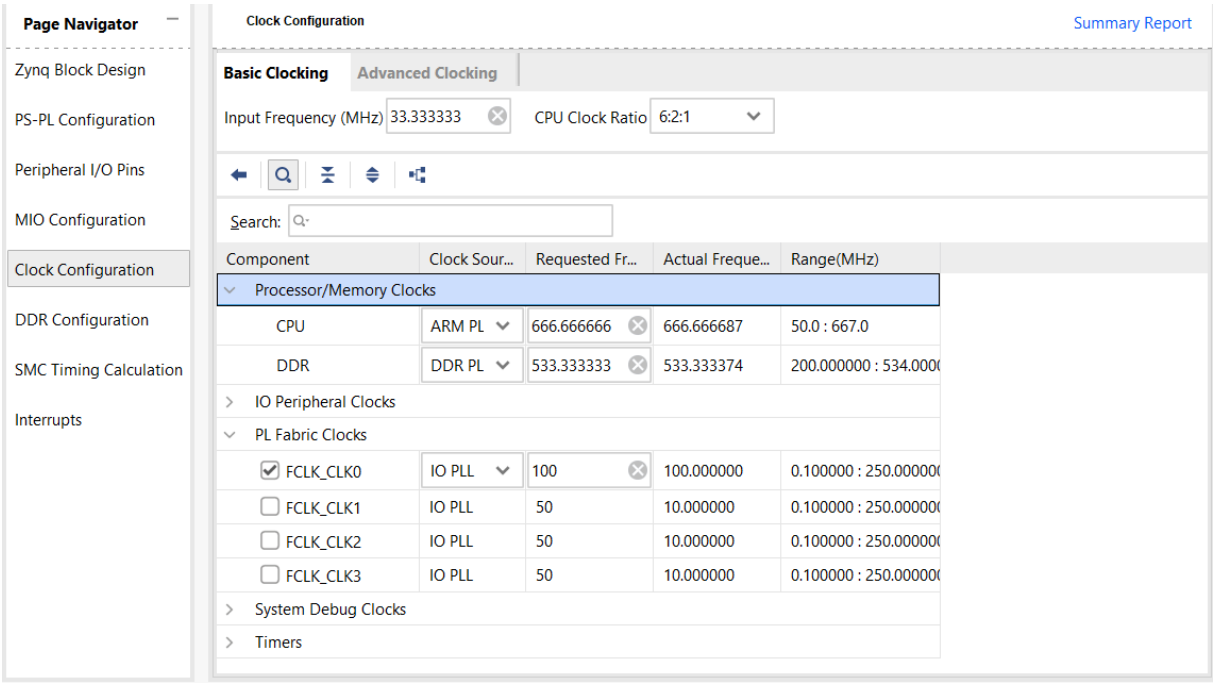
Konfiguracija periferija procesorskog sistema

Od periferija na multipleksiranim izvodima mora biti omogućen UART1. Quad SPI, SD, Ethernet i USB ne moraju biti uključeni: bare-metal aplikacija se pokreće preko JTAG-a i radi bez operativnog sistema, pa njihovo uključivanje samo dodaje ograničenja izvoda i složenost pri podizanju sistema, bez ikakve koristi.
U nastavku je ipak data konfiguracija sa uključenim SD, USB i ENET portovima, radi

pokazivanja da ne ometaju rad ovog sistema.



Slika 35. Izgled MIO Configuration prozora



Slika 36. Izgled Clock Configuration prozora

Rezultati sinteze i implementacije

Sve vrijednosti su preuzete iz Vivado izvještaja nastalih pri buildovanju. Obje kolone odnose se na kompletan blok dizajn (**vj_system_wrapper**), koji pored samog akceleratora uključuje i procesorski sistem, AXI Interconnect i BRAM kontroler; razlikuju se samo po fazi toka projektovanja: prva prikazuje stanje poslije sinteze, druga poslije razmještanja i povezivanja.

Veličina	Poslije sinteze	Poslije implementacije
Worst negative slack (WNS)	+1,822 ns	+0,751 ns
Worst hold slack (WHS)	+0,007 ns	+0,042 ns
Worst pulse width slack(WPWS)	+4,020 ns	+4,020 ns
Tačke koje ne zadovoljavaju	0 od 8.315	0 od 7.662
Implicirana Fmax pri cilju od 100 MHz	≈122,3 MHz	≈108,1 MHz
LUT	3.780 (21,48%)	3.290 (18,69%)
Flip-flopovi	2.781 (7,90%)	2.474 (7,03%)
Blokovi memorije (36 Kb)	39 od 60 (65,00%)	39 od 60 (65,00%)
DSP48E1	4 od 80 (5,00%)	4 od 80 (5,00%)

Tabela 17. Vrijednosti vremena i iskorištenosti resursa u integrisanom sistemu

Smanjenje ostvarive učestanosti između rezultata sinteze i rezultata implementacije je očekivano: procjena kašnjenja poslije sinteze ne poznaje stvarno razmještanje ćelija ni stvarne dužine veza, pa je optimistična. Nakon implementacije dizajn i dalje zadovoljava vremenska ograničenja na ciljanih 100 MHz, sa pozitivnim slackom i za vrijeme uspostavljanja i za vrijeme držanja, i svi zadati vremenski uslovi prijavljeni su kao ispunjeni. Implementacija je pritom smanjila i broj zauzetih LUT-ova za oko 13% u odnosu na sintezu, jer se dio logike sažima tek u toku optimizacije.

Raščlanjeno zauzeće resursa

Resurs	Iskorišćeno	Detalj
LUT kao logika	3.202	2.454 koristi samo O6; 748 koristi i O5 i O6
LUT kao memorija	88	Svih 88 kao pomjerački registri (106 SRL primitiva: 61 SRL16E i 45 SRLC32E); bez distribuirane memorije
Multiplekseri F7	384	4,36% od 8.800
RAMB36E1	38	Zbirni izvještaj ne razlaže zauzeće po instancama; za podjelu na banke redova i ROM klasifikatora potreban je report_utilization -hierarchical
RAMB18E1	2	Broje se kao 1 blok u ukupnih 39
DSP48E1	4	Dva množača u dizajnu: mean_r × mean_r (32×32) troši oko tri slicea, a tree_thresh × vnf (13×17) jedan; nijedan nije u putanji po pravougaoniku
BUFG	1	FCLK_CLK0, 2.625 taktnih potrošača
PS7	1	Procesorski sistem

Tabela 18. Tabelarni prikaz raščlanjenog zauzeća resursa

Kritična putanja

Putanja koja ograničava učestanost polazi od registra stanja automata i završava na dozvoli čitanja jedne od banki redova:

vj_0/U0/u_axi/u_core/u_cp/FSM_onehot_state_reg[12] TO
u_dp/u_banks/gen_banks[13].u_bank/ram_reg/ENARDEN

Ime izvornog registra potvrđuje da je Vivado automat kodovao kao „one-hot“, pa dekoder 5-na-20 iz opisa upravljačke jedinice u netlisti ne postoji kao zaseban blok - svaka linija is_<stanje> je direktno izlaz jednog flip-flopa. Sama putanja pripada mreži za generisanje adresa: iz stanja se dobija izbor reda (**bk_row_top** odnosno **bk_row_bot**), a iz njega dozvola odgovarajuće banke.

Kašnjenje putanje podataka iznosi 8,515 ns, kroz 6 nivoa logike (tri LUT5 i tri LUT6). Od toga na logiku otpada 1,262 ns (14,8%), a na povezivanje 7,253 ns (85,2%). Drugim riječima, putanju ne ograničava dubina logike nego dužina veza: signal se od registra stanja razvodi po velikom dijelu površine do banke 13. Uvođenje dodatnog stepena protočne obrade zato ne bi donijelo mnogo; djelotvornije je smanjiti fan-out i poboljšati razmještaj, na primjer replikacijom registra stanja koji upravlja dozvolama banki.

Najkritičnija putanja za vrijeme držanja je unutar protočnog niza oznaka, tag_d3 TO tag_d4, sa slackom od +0,042 ns. Iako je pozitivan, to je mala rezerva i vrijedi je provjeriti pri svakoj izmjeni razmještaja.

Zauzeće ostalih resursa nije blizu granice: 1.171 od 4.400 sliceova (26,61%), 127 jedinstvenih upravljačkih skupova i 2.474 flip-flopovaa (7,03%). Ograničavajući resurs je blok memorija sa 65% zauzeća, što je posljedica podjele slike na 27 zasebnih banki redova - svaka zauzima cijeli blok bez obzira na to što se koristi 360 od 512 kolona.

Kašnjenje i propusna moć

Oznake

Konstante ispod nisu procjene. Prilagođene su izmjerenom histogramu kašnjenja po dubini kaskade koji ispisuje test okruženje jezgra, a prilagodba pogađa svaku izmjerenu tačku unutar jednog takta.

Oznaka	Značenje	Vrijednost
Tc	Perioda takta programabilne logike	10 ns (100 MHz)
C0	Fiksna cijena: priprema prozora, normalizacija varijanse, ishod i završetak	24,3 takta
Cs	Vrijeme po fazi: ulazak u fazu, pražnjenje, evaluacija praga	3 takta
$\bar{c}$	Taktova po slabom klasifikatoru, potvrđeno	2,30 (plitke faze) 2,23 (cijela kaskada)
n_i	Broj slabih klasifikatora u fazi i	STAGE_CNT_ROM(i), 9…211
S	Indeks faze koji odlučuje o ishodu	0…24
Npr	Broj prozora u cijeloj slici, svih 13 skala	208.506
Nv	Broj pretraga redova u cijeloj slici	986

Tabela 19. Tabelarni prikaz vrijednosti u vezi s klasifikacijom

Veličina $\bar{c}$ nije jedan broj jer udio klasifikatora sa tri pravougaonika raste sa dubinom kaskade. Faze 0 i 1 ih uopšte nemaju, pa njihovi klasifikatori koštaju tačno 2 takta; usrednjeno po cijeloj kaskadi vrijednost raste na 2,23.

Kašnjenje jednog prozora

Provjereno prema histogramu koji ispisuje test okruženje jezgra (5 slučajeva, 91 prozor):

Ishod	Prozora	Izmjereno
Odbačen u fazi 0	68	48 taktova
Odbačen u fazi 1	13	88
Odbačen u fazi 2	7	152
Odbačen u fazi 3	1	229
Cijela kaskada, 2913 klasifikatora	2	6.605

Tabela 20. Tabelarni prikaz histograma odbijanja prozora

Pri 10 ns po taktu, uobičajen slučaj - odbacivanje u fazi 0, 68 od 91 prozora - košta 0,48 µs, a najgori 66,1 µs. Raspon od 138x, što kvantitativno iskazuje zašto fiksna cijena po prozoru dominira dizajnom. **Napomena o C0 = 24,3 takta:** šesnaest od njih je normalizacija varijanse, koja se izvršava jednom po prozoru bez obzira na to da li kaskada zatim odbaci prozor nakon devet klasifikatora ili izračuna svih 2913. U uobičajenom slučaju to je trećina ukupne cijene prozora - i jedini dio koji režim B ne amortizuje.

Propusna moć

U režimu B handshake sa procesorom amortizuje se na cijeli red, pa je održiva brzina recipročna prosječnoj cijeni prozora:

R = 1/(N_sr*Tc) , N_sr = 61,4 takta.

R = 1 / (61,4*10 ns) = 1,63 M prozora/s

Režim A svakom prozoru dodaje handshake sa procesorom, pa je njegova cijena isto T_prozor uvećano za konstantu:

T_A = T_prozor + T_handshake

izmjereno : 1,970 µs = 0,62 µs + 1,35 µs

Član handshakea čini pet pojedinačnih AXI4-Lite pristupa od po oko 270 ns i više je nego dvostruko veći od cijene same klasifikacije - što je i cjelokupno opravdanje za režim B.

Vrijeme obrade cijele slike

T_slika = Npr * N_sr *Tc + Nv *(T_upis+T_integral)

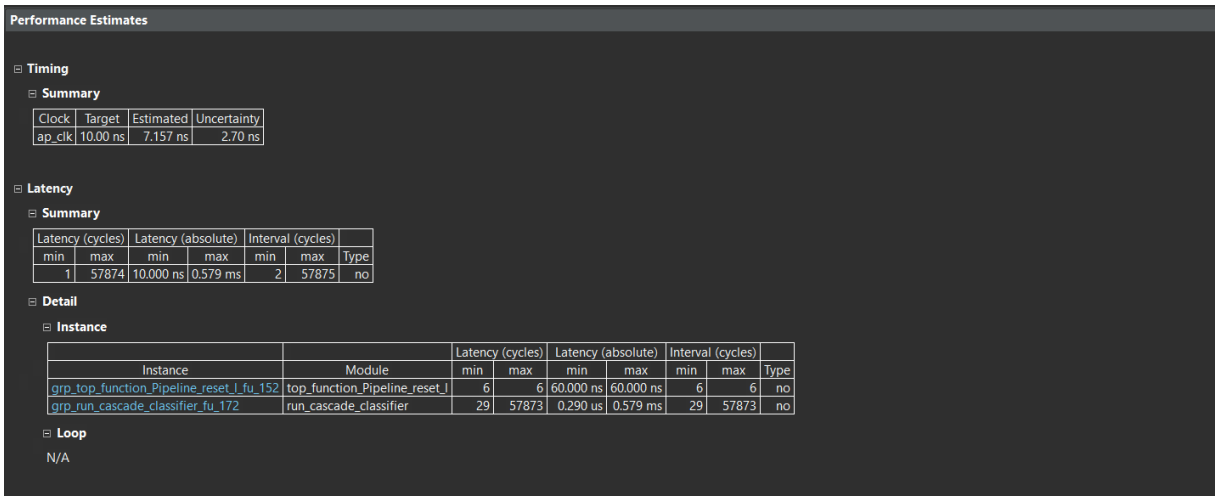
hardverski član : 208.506 * 61,4 * 10 ns = 128 ms

softverski član : 986 koraka reda, izmjereno = 283 ms

ukupno: izmjereno = 411 ms

Poređenje zauzeća resursa sa HLS

U nastavku su data predviđanja za HLS implementaciju sa kursa Projektovanje elektronskih uređaja na sistemskom nivou (prva HLS implementacija diskutovana na početku ovog dokumenta). Vidljivo je da RTL model zahtijeva **8,76 puta manje** ciklusa nego HLS implementacija, oko **2% manje LUTova**, čak **17% manje DSP jedinica**, te neznatno manji broj flipflopova. Takođe, projektovana maksimalna frekvencija za HLS bila je oko 101MHz, dok je maksimalna frekvencija rada ovog sistema nakon implementacije 108,1MHz, što rezultuje ubrzanjem za oko 6,5% u odnosu na HLS implementaciju. Time je pokazano da su arhitektonske prilagodbe i optimizacije urađene u RTLu rezultovale ubrzanjem sistema.



Performance Estimates

Timing

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00 ns	7.157 ns	2.70 ns

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
1	57874	10.000 ns	0.579 ms	2	57875	no

Detail

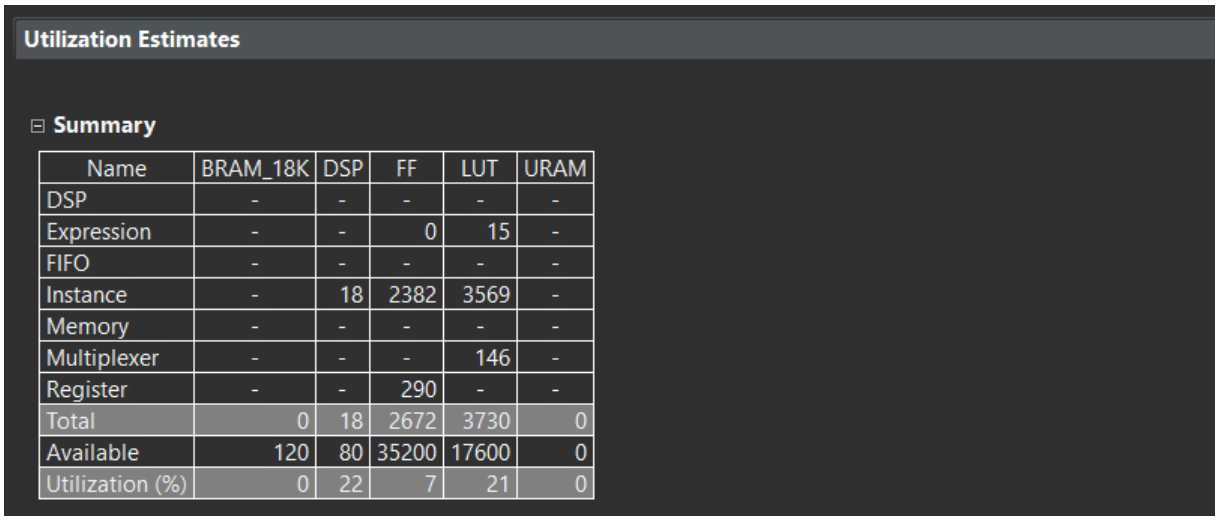
Instance

		Latency (cycles)		Latency (absolute)		Interval (cycles)		
Instance	Module	min	max	min	max	min	max	Type
grp_top_function_Pipeline_reset_l_fu_152	top_function_Pipeline_reset_l	6	6	60.000 ns	60.000 ns	6	6	no
grp_run_cascade_classifier_fu_172	run_cascade_classifier	29	57873	0.290 us	0.579 ms	29	57873	no

Loop

N/A

Slika 37. Procjena performansi sintetizovanog rješenja (top_function)



Utilization Estimates

Summary

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	15	-
FIFO	-	-	-	-	-
Instance	-	18	2382	3569	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	146	-
Register	-	-	290	-	-
Total	0	18	2672	3730	0
Available	120	80	35200	17600	0
Utilization (%)	0	22	7	21	0

Slika 38. Procjena zauzete površine sintetizovanog rješenja (top_function)

Bitstream

Generisanje bitstream-a završava se bez ijedne greške. Prijavljene su četiri informativne poruke provjere pravila projektovanja, sve iz klase DPOP-1/DPOP-2: konstatuju da DSP48

kaskada koja računa kvadrat srednje vrijednosti nema uključene unutrašnje registre MREG i PREG. To je savjet za performanse, a ne nedostatak - dizajn već zadovoljava vremenska ograničenja sa pozitivnim slackom.

Verifikacija

Verifikacija je organizovana u nezavisne nivoe. Nezavisnost je suština: svaki nivo može otkazati a da ostali ne otkažu, i svaki provjerava drugu klasu grešaka.

Nivoi koji su ponovo pokrenuti i potvrđeni

Nivo	Šta pobuđuje
1. RTL, samo jezgro (tb/tb_vj_cascade_core.vhd)	Modul vj_cascade_core neposredno, iz zlatnih vektora, preko ps_en / ps_we / ps_addr i start / done, bez ikakvog protokola magistrale. Po slučaju: 27 upisa banki, svaki prozor u režimu A uz provjeru rezultata i histogram kašnjenja po dubini kaskade, zatim pretraga u režimu B uz provjeru bafera detekcija element po element.
2. RTL, upakovano IP jezgro preko AXI (tb/tb_vj_cascade_v1_0.vhd)	Kompletno IP jezgro kroz AXI4-Lite model magistrale: čitanje registara, varijacije handshakea sa razdvojenom adresom i podatkom i odloženim BREADY / RREADY, impulsna priroda signala START, trajnost signala DONE i brisanje upisom jedinice, praćenje prekida, softverski reset, 32-bitna bajtna adresna putanja BRAM-a kroz svih 27 banki i svih pet slučajeva iz vektorske datoteke od kraja do kraja preko magistrale.
3. Izvršavanje na ploči (vj_app na Zybo Z7-10)	Cijeli sistem na stvarnom hardveru nad fotografijom: kašnjenje u režimu A, zatim potpuna detekcija u režimu B.

Tabela 21. Opis nivoa verifikacije

Zašto je dodat nivo 2

Nivo 1 pobuđuje jezgro neposredno, što ga i čini čistim testom algoritma - ali znači da se AXI omotač uopšte ne pobuđuje. U tom omotaču su skup registara, oba automata handshakea, generisanje jednotaktnih impulsa i pretvaranje bajtne adrese u adresu riječi, a u njemu postoje i oba dogovorena ograničenja. Ostaviti ga nesimuliranim bila bi najneugodnija rupa u dizajnu, a otkaz pri puštanju u rad učinio bi cijenu neverifikovanih slojeva integracije opipljivom.

Namjerno konstruisani slučajevi prihvatanja

Slučajni podaci o pikselima odbacuju se u prve dvije ili tri faze u više od 99% slučajeva, pa skup testova izveden iz slučajnih podataka nikada ne pobuđuje putanju prihvatanja niti duboke faze kaskade. Zato su dva prozora u nivou 2 i jedan u nivou 4 konstruisana uphill pretragom, upravo da bi iznudili puno prihvatanje kroz svih 25 faza, što je mnogo važnije nego što djeluje.

Ograničenja pokrivenosti RTL testbenčeva

Da bi granica onoga što je verifikovano bila jasna:

Poređenje sa virtuelnom platformom je analitičko, izvedeno iz njenog dokumentovanog vremenskog modela i izmjerenog smanjenja broja pristupa.

Sva verifikacija je funkcionalna. Mjerenje i analiza potrošnje nisu rađeni.

Poređenje sa virtuelnom platformom

SystemC/TLM virtuelna platforma izrađena u prethodnoj fazi razvoja projekta dijeli poslove isto kao i ovaj dizajn: procesorsko jezgro predaje akceleratorskom modelu jedan po jedan prozor. Virtuelna platforma dokumentuje vremenski model od 10 ns po TLM pristupu, što omogućava pošteno poređenje brojanjem pristupa.

Metoda

Broj pristupa na strani hardvera dobijen je iz strukture RTL koda i potvrđen prema konačnom automatu, a na strani virtuelne platforme iz obrasca pristupa memoriji referentne implementacije. Porede se identične jedinice posla: jedan slabi klasifikator i jedan prosječan prozor, gde se pod „prosječnim" podrazumijeva izmjerena vrednost od 14,6 klasifikatora po prozoru.

Jedinica posla	Virtuelna platforma	Ovaj dizajn (RTL)	Ubrzanje
Jedan slabi klasifikator	290 ns	20,2 ns	14,4×
Jedan prozor (prosjek)	4,31 µs	0,62 µs	7,0×

Tabela 22. Poređenje preko broja pristupa

Dva odnosa se razlikuju, i sama ta razlika je provera dosljednosti. Vrijednost po klasifikatoru izdvaja unutrašnju petlju, gdje optimizacione transformacije imaju pun efekat: sedamnaest čitanja postaje jedno, dvanaest adresnih množenja nestaje, množenja težinama nestaju. Vrijednost po prozoru uključuje fiksnu cijenu pripreme - čitanja ćoškova, računanje varijanse i kvadratni korijen - koja je po prirodi ista u obje implementacije i zato umanjuje ubrzanje u odnos na VP.

Smanjenje transfera

Druga osa poređenja, ovoga puta mjerena a ne izvedena, jeste količina prenosa sa procesora ka akceleratoru za cijelu sliku:

Veličina	Pristup virtuelne platforme	Ovaj dizajn
Riječi po koraku reda	18.000	1.080
Upisa u BRAM, cijela slika	11.151.750	714.753
Rukovanja PS/PL, cijela slika	208.506 (po prozoru)	986 (po redu)

Tabela 23. Poređenje broja prenosa

Propusnost

U režimu B dizajn obrađuje približno 1,63 miliona prozora u sekundi, što odgovara obradi cijele slike 360×240, kroz svih 13 skala i 208.506 prozora, za oko 0,14s pri burst prenosima. Od toga je približno 11 ms upis redova, a približno 128 ms rad akceleratora.

Potvrda na hardveru

Izvođenje iz prethodnih odjeljaka sada je provjereno na ploči. Aplikacija mjeri režim A nad 200 uzastopnih poziva sa jednim prozorom, pa vrijednost koju prijavljuje uključuje i full handshake sa procesorom, koji analitički model namjerno izostavlja.

Veličina	Vrijednost	Izvor
Virtuelna platforma, po prozoru	4,31 µs	Analitički, 10 ns po pristupu
Izmjereno, režim A, po prozoru	1,970 µs	Ploča, 200 poziva, globalni tajmer
Samo klasifikacija u hardveru	0,62 µs	Simulacija, svih 13 skala
Implicirano rukovanje sa procesorom	≈ 1,35 µs	Izmjereno - simulirano
Ubrzanje prema virtuelnoj platformi	2,2× izmjereno, 7,0× bez rukovanja	-

Tabela 24. Rezultati hardverskog poređenja

Ostatak od oko 1,35 µs slaže se sa onim što režim A zaista radi po prozoru: pet AXI4-Lite pristupa (dva upisa koordinata, jedan upravljački upis, najmanje jedno ispitivanje statusa i jedno čitanje rezultata), svaki kao pojedinačna transakcija preko opštenamjenskog porta. Pri otprilike 270ns po pristupu to objašnjava razliku bez potrebe da se pretpostavi da nešto nije u redu. Ništa u mjerenju ne ukazuje da se hardver ponaša drugačije nego u simulaciji, pa vrijednosti od 14,4 puta po klasifikatoru i 7 puta po prozoru ostaju kako su izvedene. Poštena glavna vrijednost je 2,2 puta end-to-end u režimu A, jer je to ono što je izmjereno. Vrijednost od 7 puta odnosi se na akcelerator izolovano i predstavlja pošteno poređenje samo ako se i na strani virtuelne platforme izuzme njena sopstvena cijena handshakea. Režim B handshake izbjegava u potpunosti i predstavlja konfiguraciju koju bi stvarna aplikacija koristila.

Puštanje u rad na ploči

Tok builda i izvoza

Buildovanje u alatu Vivado može se podijeliti u četiri faze, koje poziva **scripts/vj_project.tcl**:

Faza	Radnja
check	Sinteza jezgra akceleratora van konteksta; daje izvještaje o vremenu i resursima za samo jezgro.
ip	Pakovanje modula vj_cascade_v1_0 u lokalno IP skladište, uz kreiranje interfejsa AXI4-Lite i BRAM_PORTA.
bd	Buildovanje blok dizajna: procesorski sistem, akcelerator, AXI BRAM kontroler,interconnect, reset blok, vođenje prekida, dodjela adresa.
impl	Sinteza, implementacija, generisanje bitstream-a, izvoz izvještaja.

Tabela 25. Faze izvršavanja TCL skripte

Hardver se zatim izvozi zajedno sa bitstream-om, nad dobijenom XSA datotekom builduje se se Vitis platformski projekat, a aplikacija se gradi nad tom platformom. Dodatno, postoji i izdvojena **sim** faza za kreiranje simulacionog projekta, koja omogućava pokretanje testbencha kako za jezgro tako i za čitav sistem. Odabir faza vrši se dodavanjem **-tclargs <ime faze>** argumenta u komandnu liniju pri pokretanju skripte. Po defaultu, bez argumenata se

pokreću sve faze buildovanja, osim simulacione, kako bi se izbjeglo preklapanje pri izgradnji simulacionog projekta.

U okviru TCL skripte ostavljena je mogućnost uvezivanja sa dodatnom **vj_golden.tcl** skriptom koja sadrži konfiguraciju ploče na kojoj je aplikacija uspješno radila. Skripta nije neophodna, ali je ostavljena radi debagovanja i provjere ispravnosti, baš kao i faze buildovanja.

Aplikacija na procesoru

Bare-metal aplikacija radi tri stvari redom: mjeri kašnjenje pojedinačnog prozora u režimu A, izvršava potpunu detekciju u režimu B i ispisuje pronađene koordinate pravougaonika lica.

```
int main(void)
{
      setvbuf(stdout,NULL,_IONBF,0);
    XTime t0,t1;
    vj_rect_t faces[VJ_MAX_FACES];
    xil_printf("\r\n");
    xil_printf("=======================================================\r\n");
    xil_printf(" Viola-Jones sistem za detekciju lica\r\n");
    xil_printf(" slika %dx%d, scaleFactor 1.2, shiftStep %d\r\n",VJ_IMG_WIDTH,
VJ_IMG_HEIGHT, SHIFT_STEP);
    xil_printf("=======================================================\r\n");
    vj_init(VJ_CTRL_BASE, VJ_BRAM_BASE);

    /*
     *Trajanje detekcije jednog prozora, mod A.
     *Uporedivo je s virtuelnom platformom zbog istog nacina rada, vecina
     *prozora se odbija u fazi 0, pa raspodjela trajanja nije uniformna
     *pa je minimalno kasnjenje pozeljno uz prosjek.*/
    {
        static int32_t sum[VJ_INT_ROWS*VJ_IMG_WIDTH];
        static int32_t sqs[VJ_INT_ROWS*VJ_IMG_WIDTH];
        vj_integral_first(test_image,sum,sqs,VJ_IMG_WIDTH,VJ_IMG_HEIGHT);
        vj_set_width(VJ_IMG_WIDTH);
        vj_set_row_base(0);
        for(int r=0;r<VJ_INT_ROWS;r++)//redovi integralne slike
            vj_upload_row(r,&sum[r*VJ_IMG_WIDTH],VJ_IMG_WIDTH);
        vj_upload_row(VJ_BANK_SQ0,&sqs[0*VJ_IMG_WIDTH],VJ_IMG_WIDTH);//dva coska
kvadrirane integralne slike, relevantna za detekciju
        vj_upload_row(VJ_BANK_SQ23,&sqs[23*VJ_IMG_WIDTH],VJ_IMG_WIDTH);
        const int N=200;
        XTime_GetTime(&t0);
        for(int i=0;i<N;i++)
            (void)vj_run_window(i);
        XTime_GetTime(&t1);
        double us=ticks_to_us(t1-t0)/N;
        xil_printf("\r\n=Mod A, detekcija za jedan prozor  =\r\n");
        printf("   prosjek po prozoru : %.3f us (sa uracunatim handshakeom)\r\n",
us);
    }
    /*
     *Kompletna detekcija, mod B.*/
    xil_printf("\r\n= Mod B, kompletna detekcija =\r\n");
    XTime_GetTime(&t0);
```

```
    int n=vj_detect(test_image,SCALE_FACTOR,SHIFT_STEP,faces,VJ_MAX_FACES);
    XTime_GetTime(&t1);
    printf("   proteklo : %.1f ms\r\n",ticks_to_us(t1-t0)/1000.0);
    xil_printf("   broj lica   : %d\r\n\r\n\r\n",n);
    for(int i=0;i< n;i++)
        xil_printf("   lice %2d :  x=%3d  y=%3d  w=%3d
h=%3d\r\n",i,faces[i].x,faces[i].y,faces[i].w,faces[i].h);
    xil_printf("\r\ndone.\r\n");
    return 0;
}
```

Listing 9. main funkcija bare-metal aplikacije

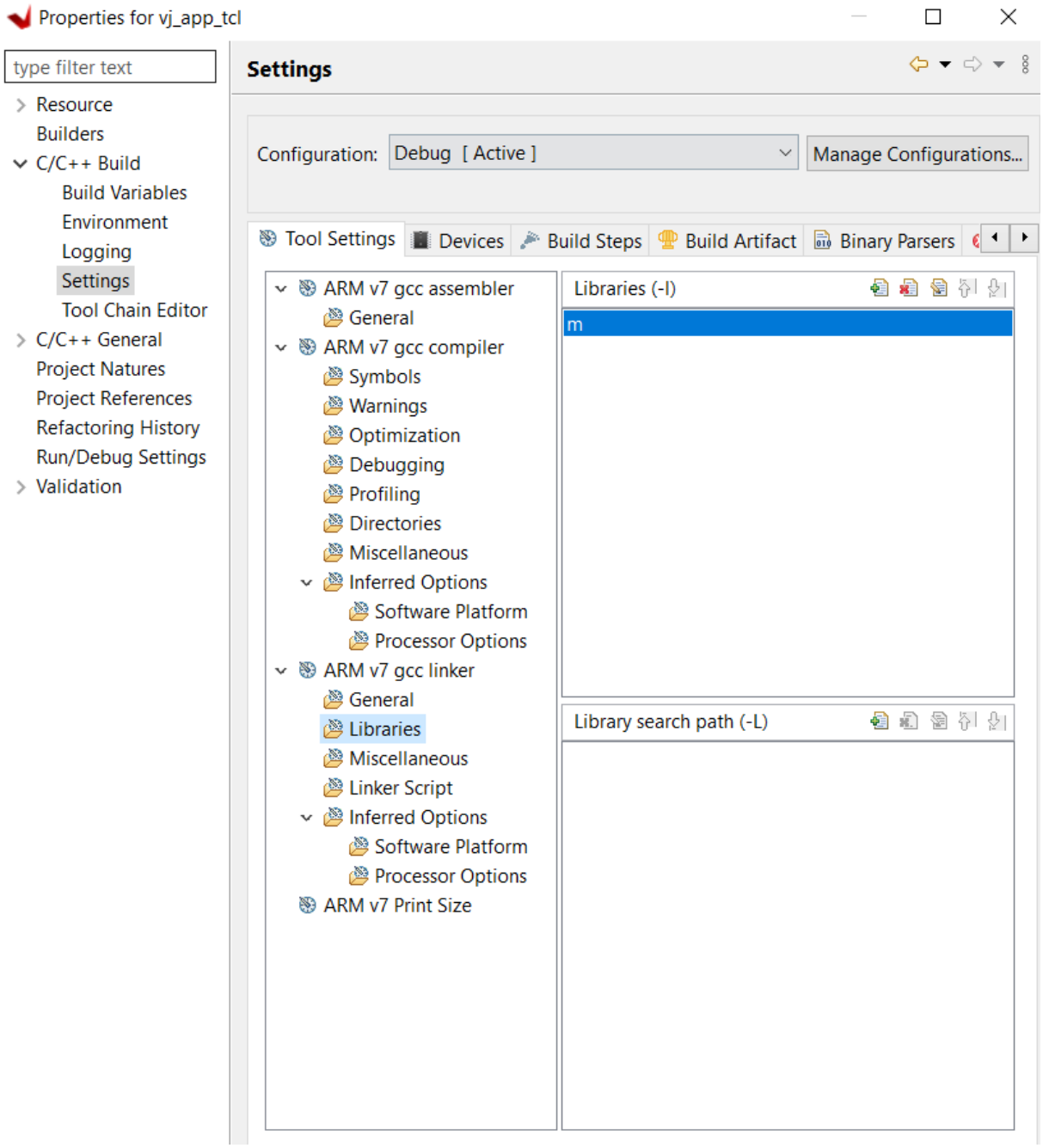
Mjerenje vremena koristi globalni tajmer procesora Zynq preko funkcije XTime, čija je učestanost brojanja polovina takta procesora. Testna slika ugrađena je kao zaglavlje koje generiše skripta **img_to_header.py**; piramida skala builduje se u toku rada, uz istu kvantizaciju u fiksnom zarezu koju koristi virtuelna platforma, kako bi detekcije bile uporedive.

Konfiguracija ploče i softvera

Korektivne vrednosti za konfiguraciju ploče preuzete su iz zvanične konfiguracije proizvođača Digilent za i primjenjuju se eksplicitno, tako da build ne zavisi od toga da li su datoteke opisa ploče instalirane. Nju vrši pomenuta TCL skripta, pa je stoga na korisniku ostavljena mogućnost eventualne provjere unutar blok dizajna u vezi sa UART portovima i taktovima u sistemu.

Za rad bare-metal aplikacije nakon kreiranja platforme i aplikativnog projekta neophodno je uključiti biblioteku **math.h** kako bi se mogla koristiti floor funkcija potrebna za kreiranje integralne slike. Najlakši način za to jeste **desni klik na aplikaciju -> Properties -> C/C++ Build -> Settings -> ARM v7 gcc linker -> Libraries -> Add… i nakon toga upisati slovo m u**

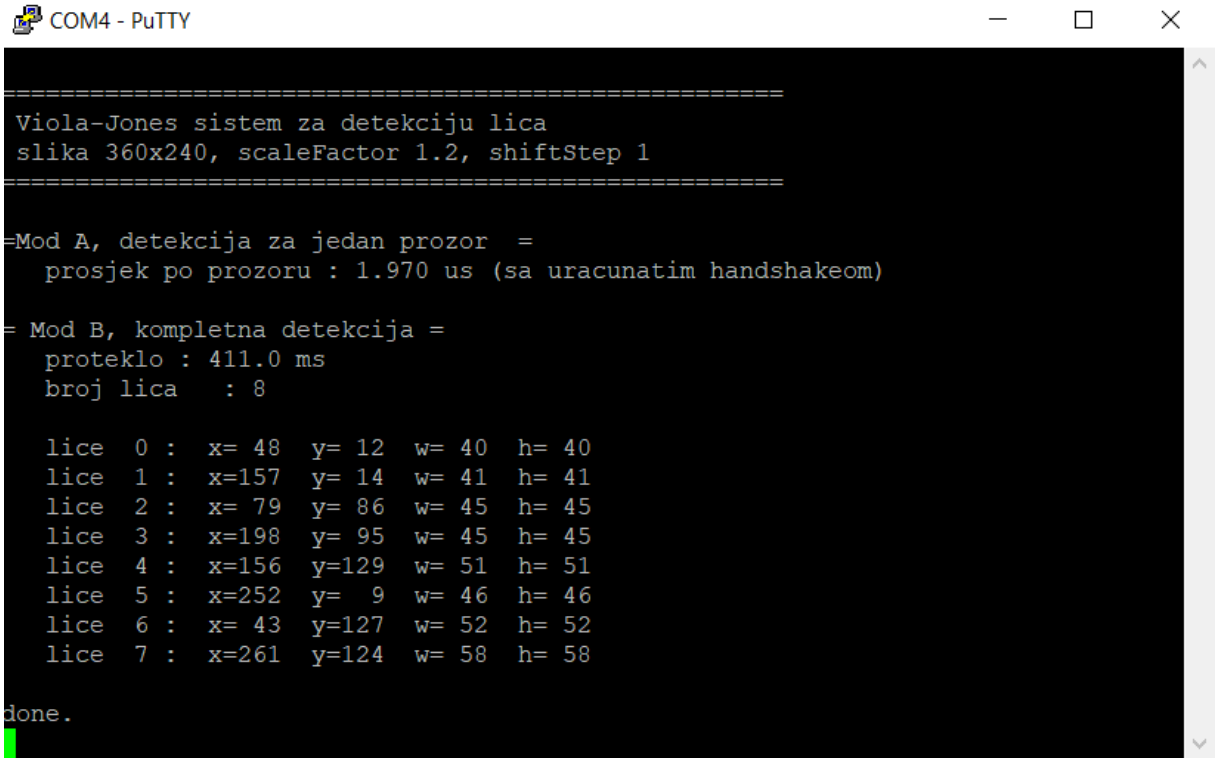
novootvoreni text box.



Slika 39. Izgled Properties prozora aplikacije nakon konfiguracije biblioteke

Izmjereni rezultati na ploči

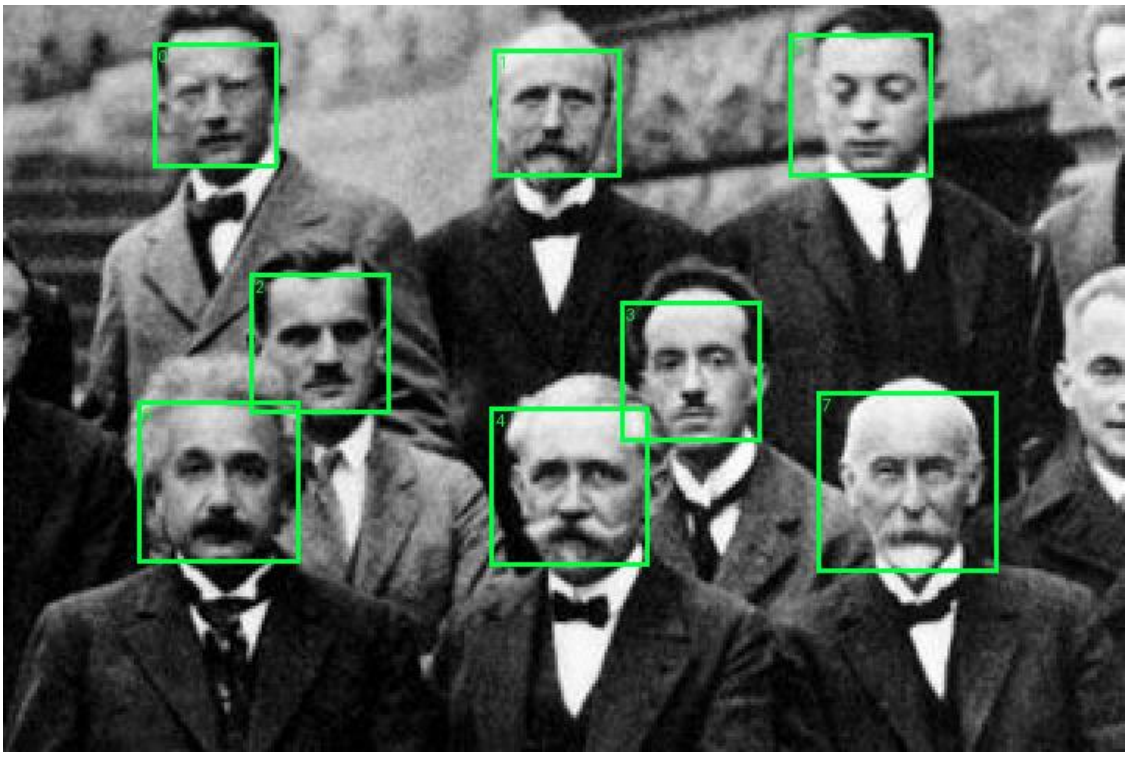
U nastavku je ostavljen serijski ispis bare-metal aplikacije iz Putty terminala.



Slika 40. Ispis u PuTTY terminalu nakon pokretanja bare-metal aplikacije

Kvalitet detekcije

Testna slika je isječak grupne fotografije 360 × 240, sa 11 vidljivih lica, od kojih je nekoliko presječeno ivicom kadra. Svih osam prijavljenih pravougaonika pada na lica.



Slika 41. Rekonstrukcija pravougaonika ispisanih u terminalu na test_image.h

Odsustvo lažnih detekcija je također bitan dio rezultata. Izmiješani integralni podaci ne daju nula detekcija; daju detekcije rasute po pozadini. To što ih nema neposredan je dokaz da su obje neprimjetne putanje otkaza ispravne na hardveru: adresni prozor od 64 KB doprema upise u svih 27 banki, a softver i hardver se slažu o tome koja fizička banka čuva koji logički red.

Odziv je osam od jedanaest. Promašena su lica presječena lijevom i desnom ivicom, koja se po konstrukciji ne mogu detektovati - prozor mora u cjelini stati u kadar, a pretraga ide samo do x_last = sz_w - 24 - 1.

Vrijeme režima B u odnosu na predviđanje

Izmjerenih 411 ms je 2,9 puta više od približno 140 ms koliko je analitički predviđeno. To predviđanje je bilo pogrešno i razlog je isključivo softverske prirode.

Komponenta	**Predviđeno**	**Ponovna procjena**	**Razlog**
Rad akceleratora	≈128 ms	≈128 ms	Nepromjenjeno; režim A potvrđuje cijenu po prozoru
Upis redova	≈11 ms	≈100 ms	Vrijednost od 11 ms pretpostavljala je burst prenose, a vj_upload_row izdaje pojedinačne Xil_Out32 upise u oblast mapiranu kao uređaj
Integralni proračun na procesoru	nije uračunato	ostatak	vj_integral_lastrow izvodi dva memmove poziva po koraku reda kroz 986 koraka, uz petlju presamplinga i akumulacije

Tabela 26. Poređenje analitičke procjene s mjerenim vrijednostima

Kako su obje stavke na strani softvera, u slučaju potrebe mogu se dalje optimizovati bez dodirivanja RTL koda.

Dodaci za pomoć pri radu

Podešavanja serijskog terminala

Postavka	**Vrijednost**
Brzina prenosa	115200
Bitova podataka	8
Stop bitova	1
Parnost	Nema
Upravljanje tokom	Nema
Port	COMx (iz Device Managera)

Tabela 27. Podešavanja PuTTY terminala pri radu sa bare-metal aplikacijom

Terminal mora biti povezan prije pokretanja programa. UART ne baferuje ispis za terminal koji se poveže naknadno, pa se najraniji ispis nepovratno gubi ako se terminal otvori kasnije.

Referentna konfiguracija procesorskog sistema

Vrijednosti ispod su one koje za ovu ploču postavlja zvanična konfiguracija proizvođača Digilent. Zabilježene su ovdje kako bi se svako buduće odstupanje moglo uočiti pregledom.

Multipleksirani izvodi i naponi

Parametar	Vrijednost
PCW_UART1_PERIPHERAL_ENABLE	1
PCW_UART1_UART1_IO	MIO 48 .. 49
PCW_UART1_BAUD_RATE	115200
PCW_UART_PERIPHERAL_CLKSRC	IO PLL
PCW_UART_PERIPHERAL_FREQMHZ	100
PCW_PRESET_BANK0_VOLTAGE	LVCMOS 3.3V
PCW_PRESET_BANK1_VOLTAGE	LVCMOS 1.8V
PCW_CRYSTAL_PERIPHERAL_FREQMHZ	33.333333
PCW_FPGA0_PERIPHERAL_FREQMHZ	100

Tabela 28. Podešavanja izvoda i napona

Memorija

Parametar	Vrijednost
PCW_UIPARAM_DDR_MEMORY_TYPE	DDR 3 (Low Voltage)
PCW_UIPARAM_DDR_PARTNO	MT41K256M16 RE-125
PCW_UIPARAM_DDR_DEVICE_CAPACITY	4096 MBits
PCW_UIPARAM_DDR_DRAM_WIDTH	16 Bits
PCW_UIPARAM_DDR_BUS_WIDTH	32 Bit
PCW_UIPARAM_DDR_SPEED_BIN	DDR3_1066F
PCW_UIPARAM_DDR_FREQ_MHZ	533.333333
PCW_UIPARAM_DDR_CL / CWL	7 / 6
PCW_UIPARAM_DDR_T_RCD / T_RP	7 / 7
PCW_UIPARAM_DDR_T_RC	48.75
PCW_UIPARAM_DDR_T_RAS_MIN	35.0
PCW_UIPARAM_DDR_T_FAW	40.0
Adresni biti reda / kolone / banke	15 / 10 / 3
Kašnjenja vodova 0..3	0.221 / 0.222 / 0.217 / 0.244
Pomjeraji DQS prema taktu 0..3	−0.050 / −0.044 / −0.035 / −0.100
Podešavanje: nivo upisa, kapija čitanja, podatkovno oko	Sve uključeno
Dobijeno PCW_DDR_RAM_HIGHADDR	0x3FFFFFFF (1 GiB)

Tabela 29. Podešavanja DDR memorije

Namjerno nepromijenjeno podešavanje takta

Ovaj dizajn izvršava Cortex-A9 na 666,67 MHz (divider faktor ARM PLL-a 40); konfiguracija proizvođača koristi 650 MHz. Obje vrijednosti su dozvoljene. Zadržana je viša zato što iz nje slijedi učestanost brojanja globalnog tajmera, a time i svaka vremenska vrijednost koju aplikacija prijavljuje.

Literatura

P. Viola i M. Jones, „Rapid Object Detection using a Boosted Cascade of Simple Features", Proc. CVPR 2001

Digilent, „Zybo Z7 Reference Manual"

AMD/Xilinx, UG585, „Zynq-7000 SoC Technical Reference Manual"

AMD/Xilinx, PG078, „AXI BRAM Controller v4.1 Product Guide“

ARM, „AMBA AXI and ACE Protocol Specification"

AMD/Xilinx, UG994 / UG1118, Vivado IP Integrator i „Creating and Packaging Custom IP"

P. P. Chu, „RTL Hardware Design using VHDL", Wiley-Interscience, 2006

Fakultet tehničkih nauka, Katedra za elektroniku, materijali sa predmeta „Projektovanje složenih digitalnih sistema".